TECHNICAL DOCUMENTATION

KrugWorld

A gamified personal life app: money, calendar, meals, training and ideas as an RPG character sheet, with an AI steward named Gaby.

Spring Boot 3.3 · Java 17 · PostgreSQL 16 · Flyway · React 19 · TypeScript · OpenAI · Claude · ESP32 desk device

Version 0.1.0 · 23 October 2026

krugworld.veteryr.online

Contents

Twelve chapters, from what the app does to how it runs in production.

01 **Overview**
What KrugWorld is and how these documents fit together

02 **Features & Game Rules**
Every screen, plus XP, levels, streaks and badges

03 **Flows in Pictures**
Sign-in, password reset, pairing, import and a diagram index

04 **Architecture**
Packages, layers, events, scheduling, security, frontend

05 **Database**
Migrations, tables, relationships and derived numbers

06 **API Reference**
Every endpoint with bodies, responses and error codes

07 **Gaby, the Steward**
The AI helper: tool loop, language rule, limits, voice

08 **Desk Device API**
The ESP32 desk Gaby and its token-based API

09 **Development**
Running locally, tests, and adding a feature

10 **New PC Setup**
Getting a fresh machine ready

11 **Operations**
Releases, troubleshooting, rollback, backups, lessons

12 **First-time Deployment**
Setting up the server from nothing

CHAPTER 01

Overview

What KrugWorld is and how these documents fit together

KrugWorld is a gamified personal life app styled as an RPG character sheet: money, calendar, meals, training and ideas all earn XP, and the AI steward **Gaby** can act on all of it, in the browser or on a desk device.

Document	Read it when you want to…
Flows in Pictures	see it in pictures: an index of every diagram, plus sign-in, password reset, pairing and import flows
Features & Game Rules	know what every screen does, and the game rules (XP, levels, streaks, badges)
Architecture	understand how the pieces fit: packages, layers, events, security, frontend
Database	look up a table, the migrations, or the rules for changing the schema
API Reference	call the browser API: every endpoint, body, response and error code
Gaby, the Steward	understand or extend the AI steward: tools, providers, limits, voice
Desk Device API	work on the desk device (ESP32) and its API
Development	run it locally, run the tests, add a feature the house way
New PC Setup	set up a fresh machine
Operations	release, check, back up, roll back or fix the live site
First-time Deployment	set the server up for the first time

At a glance

- **Live:** https://krugworld.veteryr.online
- **Stack:** Spring Boot 3.3 (Java 17) + PostgreSQL 16 (Flyway) + React 19 / TypeScript / Vite, shipped as one jar in Docker behind nginx.
- **Tests:** backend integration tests against real Postgres (Testcontainers); frontend lint at zero warnings; CI on every push.
- **Design ideas worth knowing:** balances and debts are summed from history, never stored; rewards are idempotent through a unique `(user, kind, ref)` key; every lookup is scoped to the user; the nightly job can't double-book; Gaby's tools go through the same services as the screens.

Known gap

On 2026-10-22, four releases were deployed from another PC and aren't in this repository yet: **push notifications** (`V10__push.sql`), **meal money**, **one language** and **push urgency**. Once that code is merged, document them in FEATURES, API and DATABASE (DATABASE.md already reserves the V10 row).

CHAPTER 02

Features & Game Rules

Every screen, plus XP, levels, streaks and badges

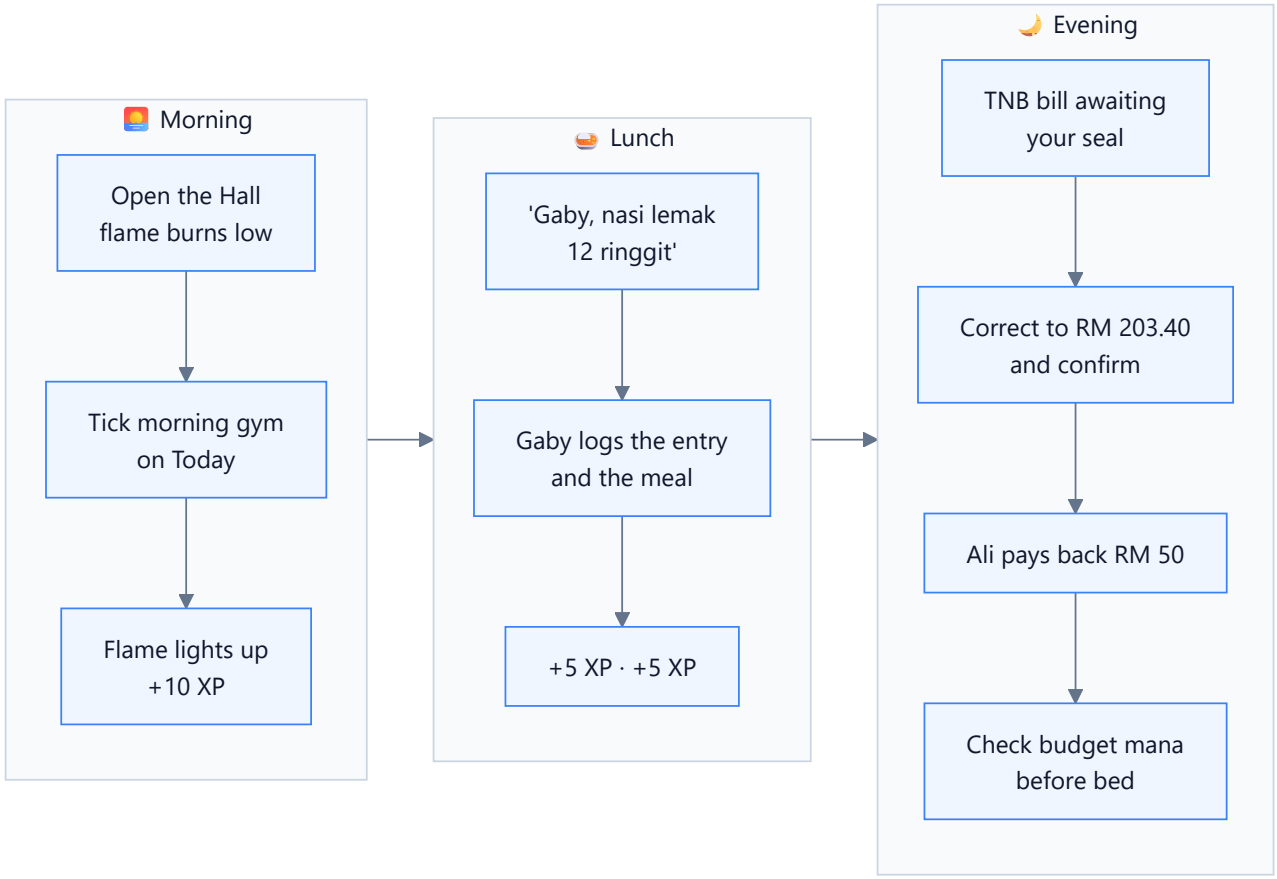
A tour of every screen, then the rules of the game: how XP, levels, badges and streaks work. Navigation is grouped as **Hall · Finance · Life · Arcane · Settings**; on phones each group is a tab in the bottom bar.

FIGURE 1 · FEATURES & GAME RULES



A day in KrugWorld

FIGURE 2 · A DAY IN KRUGWORLD



The Hall

The landing page: your life as an RPG character sheet. It fits one laptop screen (1366×657 and up) with Gaby's dock closed. Read it like a sheet:

- **Left: who you are.** The character card: an avatar ring that fills with XP towards the next level, your level and title, the XP bar, and the badge grid (locked badges are dashed). The **streak flame** sits on the corner of the avatar with the day count inside it:
  - **burning** (flickers): you've already done something today;
  - **low** (dim, breathing): the streak is alive but today is still empty;
  - **cold** (grey, 0): no streak. Hovering shows your best streak and how many of the last 7 days you were active.

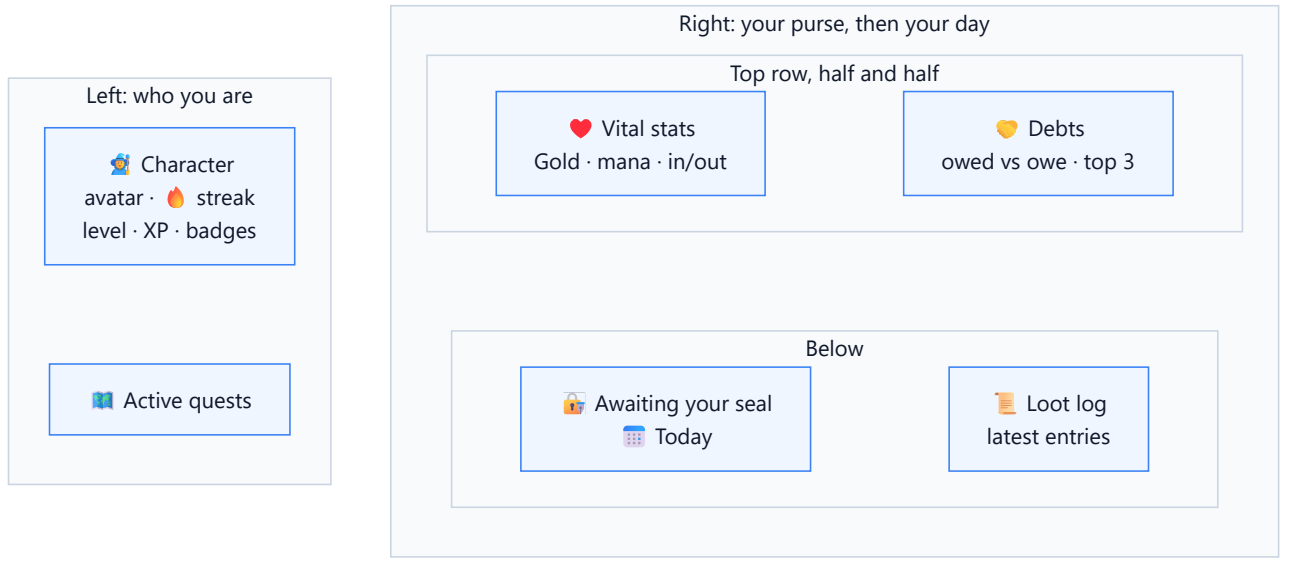
  Below it, **Active quests**: up to three savings goals with progress bars.
- **Top row: the state of your purse**, two halves side by side:
  - **Vital stats**: *Gold* (total balance) as the HP bar against this month's money, *Budget mana* (what's left of this month's budgets, with days left), and this month's money in and out.
  - **Debts**: what you're owed and what you owe, a tug-of-war bar between them, and the three most pressing debts (overdue first, then soonest due).
- **Below: what wants doing, then what happened.**

- **Awaiting your seal** (only when something is due): recurring entries set to "wait for my confirm" whose day has come. The wax-seal button opens the confirm dialog.
- **Today**: calories against the goal, today's planned workouts (tick them off), upcoming events and ideas in progress.
- **Loot log**: the latest ledger entries.

The title bar shows the greeting, the weather for your saved location (Open-Meteo, called from the browser) and the **Log entry** button.

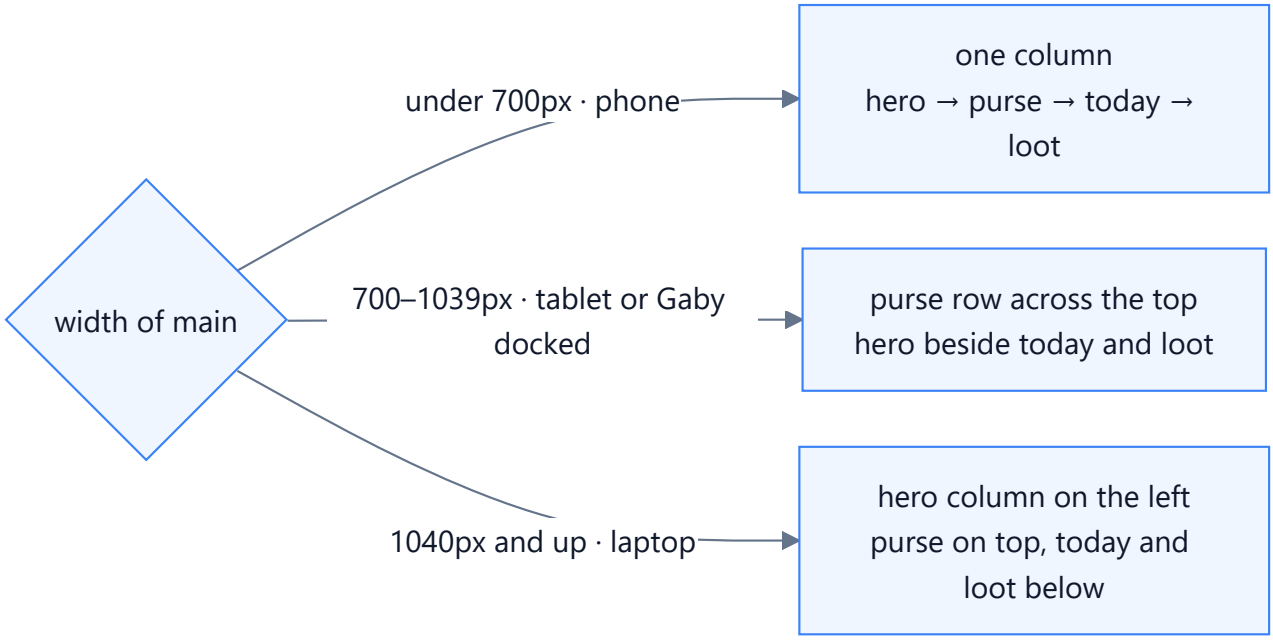
On medium widths (for example with Gaby docked) the purse row goes across the top and the hero sits under it; on phones everything is one column.

FIGURE 3 · THE HALL



The breakpoints, by the room the page actually has:

FIGURE 4 · THE HALL



Finance

Ledger

Every movement of money, month by month: income, expenses and transfers between accounts. Filter by account or category; see the month's income, spending and net. Logging an entry earns XP (see below). Entries made by a recurring rule are marked.

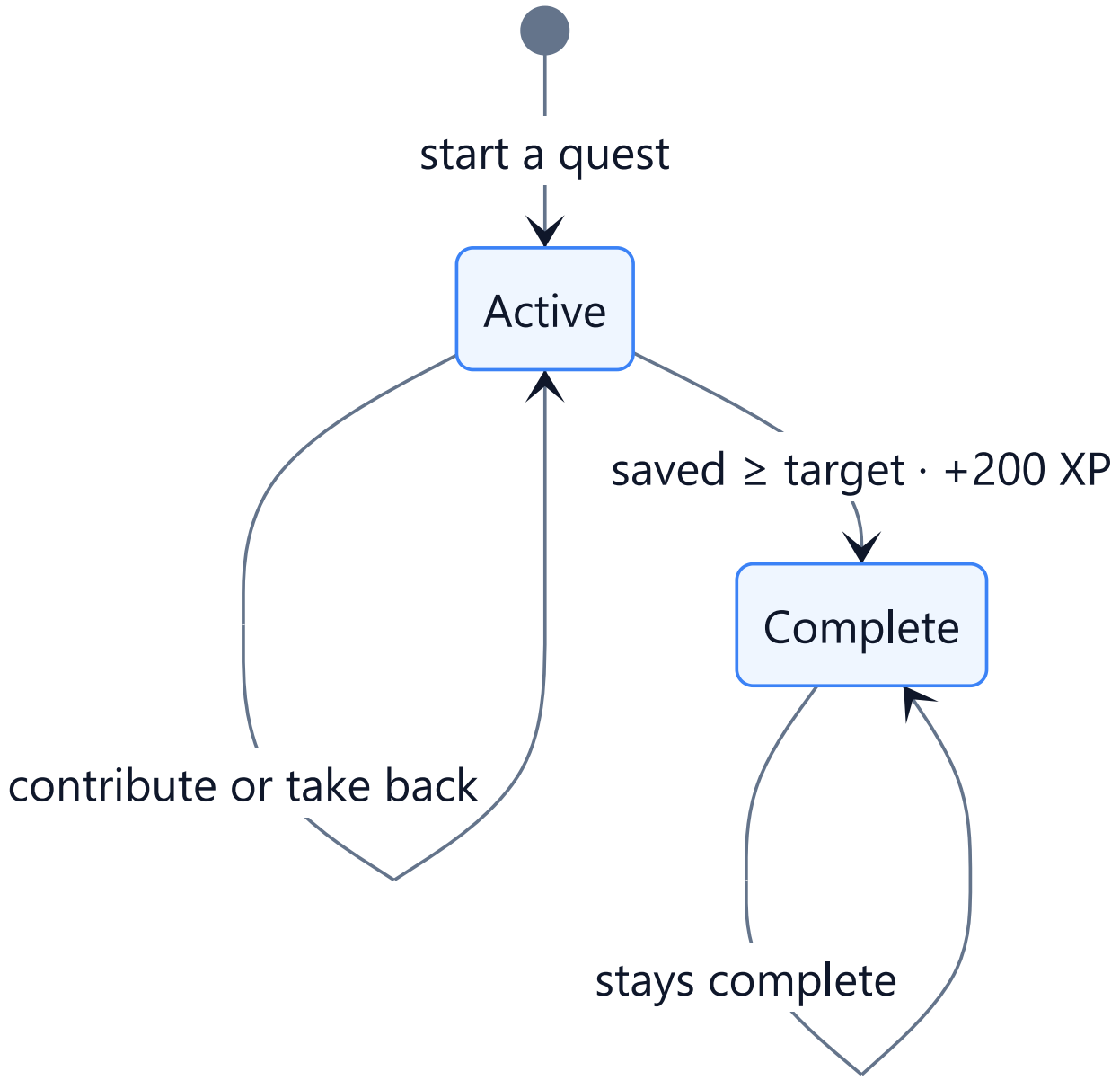
Budgets ("mana")

Give an expense category a monthly budget and it appears here as a mana bar: budget, spent and left, plus the month's total and anything spent outside a budget. Finishing a month within a category's budget earns +50 XP for that budget.

Quests

Savings goals: a name, an icon, a target and an optional deadline. Put money towards a quest (or take some back); reaching the target completes it for +200 XP and the *Quest complete* badge. A completed quest stays completed.

FIGURE 5 · QUESTS



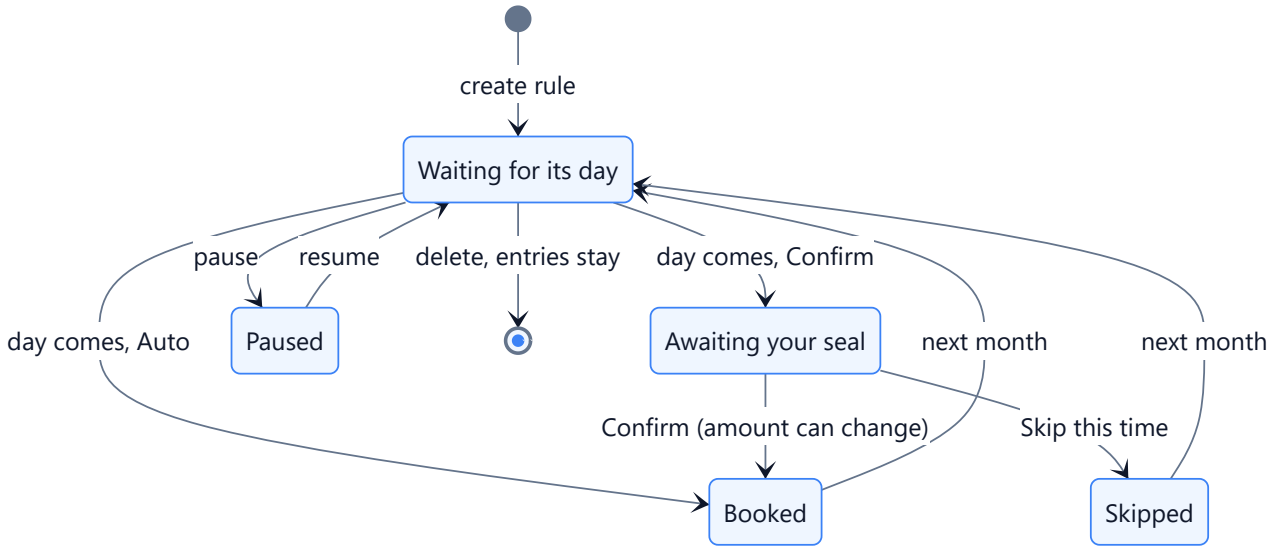
Recurring

Salary, rent, subscriptions: things that happen on the same day every month. Each one says how it's logged **on the day**:

- **Log it by itself** (*Auto*): the nightly job books it at 00:05. For fixed amounts like salary or rent.
- **Wait for my confirm** (*Confirm*): on its day it shows up in the Hall under *Awaiting your seal*. Open it to check or correct this month's amount, then **Confirm** (it's booked and earns entry XP), or **Skip this time** (nothing is booked; it moves to next month). For bills that vary, like electricity, or anything you want to check before it's recorded. If several months were missed, they come up one at a time, oldest first.

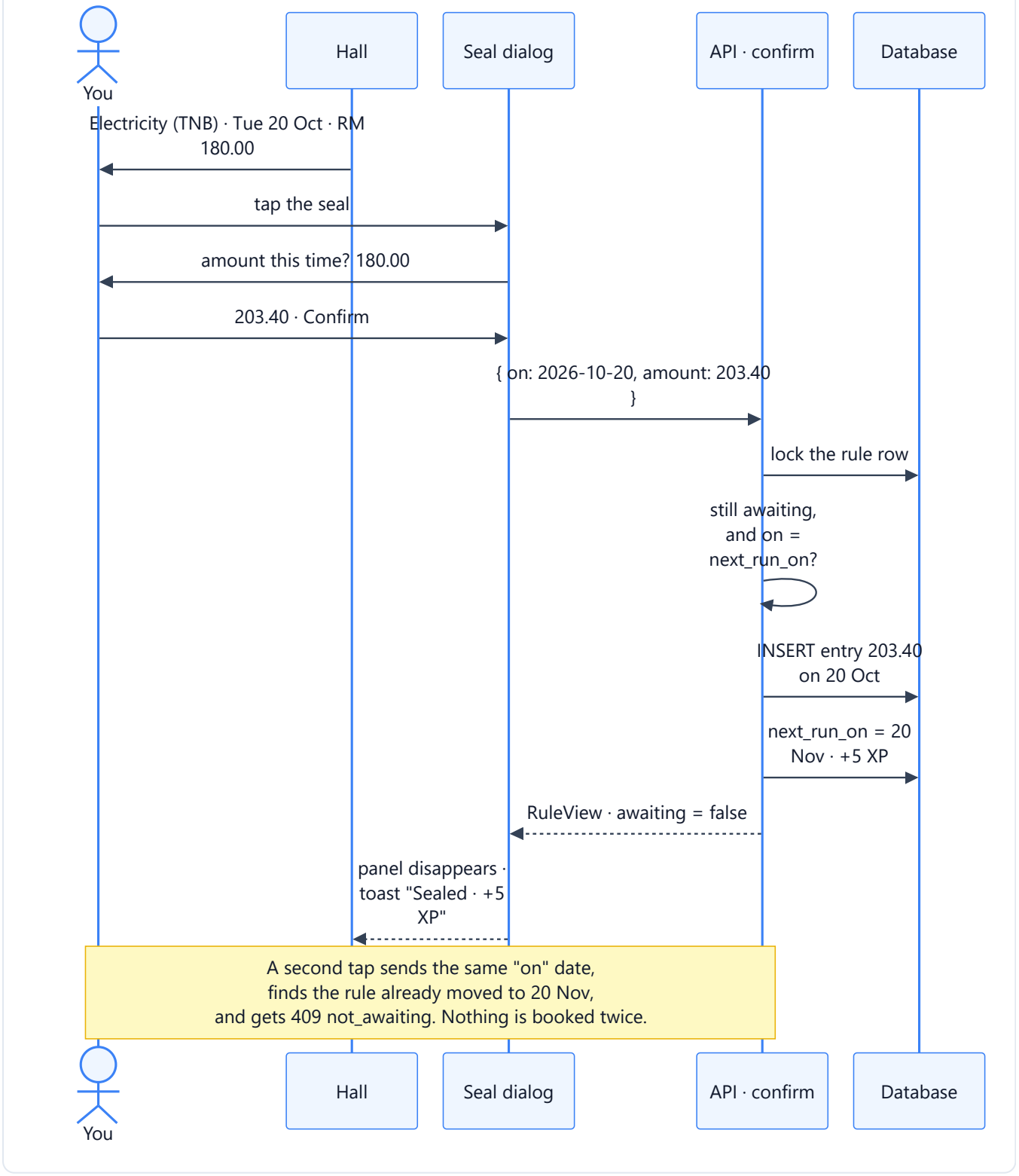
Days 29–31 fall on the last day of shorter months. Pausing a rule stops it; deleting it keeps the entries it already made. If the server was down over a payday, auto rules catch up when it starts.

FIGURE 6 · RECURRING



Sealing one, step by step:

FIGURE 7 · RECURRING



Debts

A memory of money between you and other people, kept apart from the ledger (it's about who owes whom, not money in an account).

- **Owed to you** and **You owe** columns, each debt with the person, what it was for, a repaid bar, and a due date (red when overdue).

- **They paid / I paid** records a repayment, in parts if needed. Paying the last of it settles the debt: +10 XP the first time and the *Clean slate* badge.
- **Let the rest go** settles it without the rest being paid (forgiven, written off). It can be opened again.
- Repayments recorded by mistake can be removed; a debt that's owed again reopens.
- Settled debts move to a collapsible *Settled* list.

FIGURE 8 · DEBTS

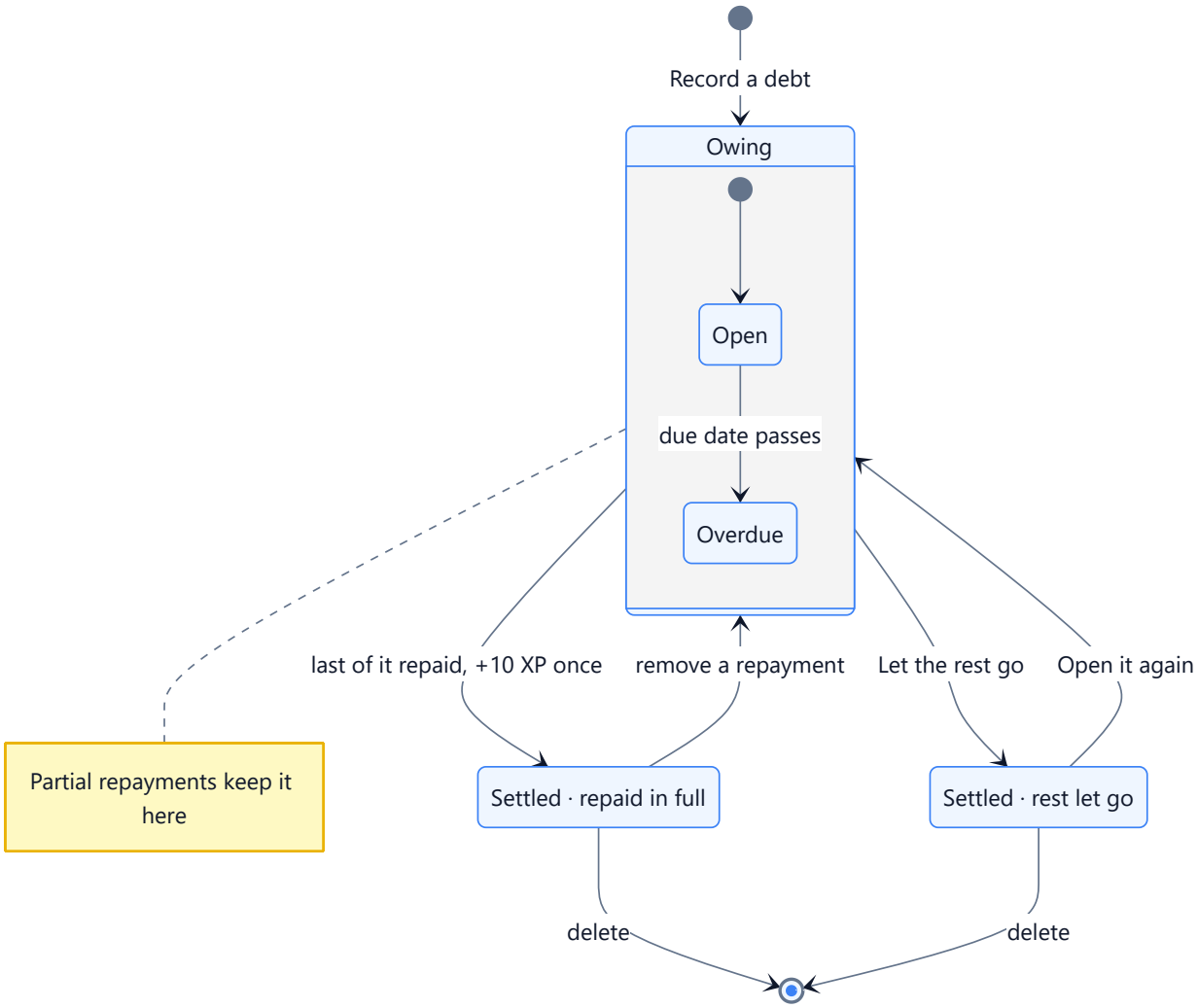
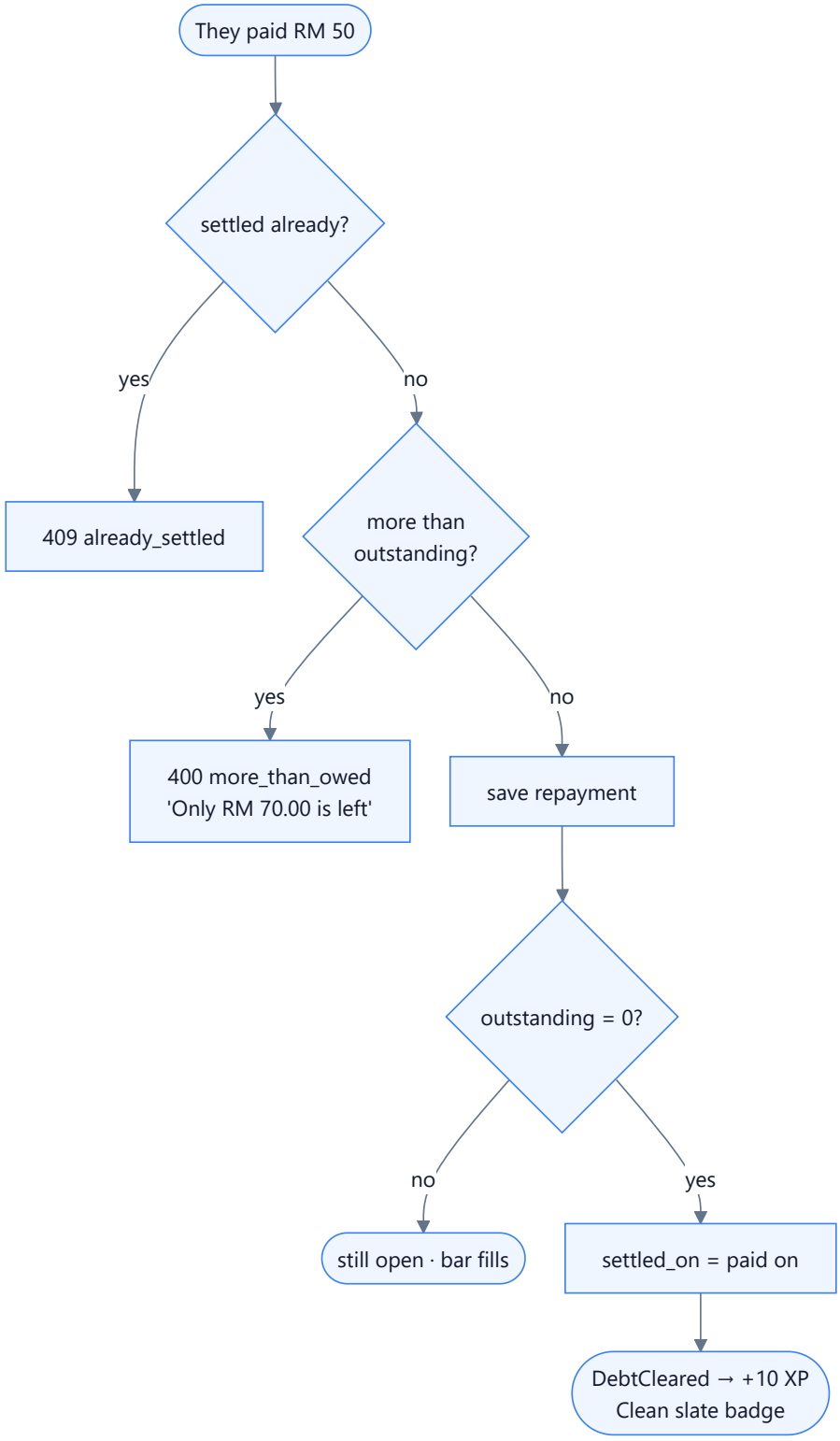


FIGURE 9 · DEBTS



Accounts

Where the money is kept: bank, cash, e-wallet, card, other, each with an opening balance. Balances are always computed from the entries. Deleting an account with history archives it instead (hidden from pickers, history kept); it can be restored.

Life

Calendar

A month grid with dots on busy days; pick a day to see and edit its events. Events can be all-day or at a time. The next few appear in the Hall's Today panel.

Vitals

- **The feast**: meals with calories against your daily goal, and the last 7 days as a bar chart. Logging any meal on a day earns +5 XP once for that day.
- **Training grounds**: workouts planned on certain weekdays (or any day). Ticking one earns +10 XP, once per workout per day. Each workout keeps a streak of *planned* days done in a row, so resting on an unplanned day doesn't break it.

Workshop

An idea board in three columns: to do, doing, done. Finishing an idea earns +15 XP.

Arcane

Steward (Gaby)

Chat with Gaby, the realm's steward, who can read and change everything above through tools: log spending, check budgets, add a recurring bill, confirm one, record that Ali paid you back, plan a workout. She answers in the mix of English and Malay you use, with a light medieval flavour, plus a one-line English subtitle. On wide screens she can be docked in a right-hand column on any page. See Gaby, the Steward.

Oracle's Eye

Shows what your Anthropic organisation spent on Claude, day by day. It calls the Anthropic Admin API straight from the browser with an admin key you paste in; the key is kept in your browser only and never reaches KrugWorld's server.

Settings

- **Skin**: *Classic Krugger* (the old Tavern exactly: pixel fonts, teal stone, square borders, scanlines) or *KrugWorld* (Pirata One + Barlow Condensed) with four backgrounds: Knight's steel (default), oak, forest, parchment.
- **Recovery email**: needed to reset a forgotten passphrase by email.
- **Desk Gaby**: make a pairing code, see and remove paired devices.
- **Import from Krugger's Tavern**: one-off import of the old app's data.
- **Weather**: your location and °C/°F.

Game rules

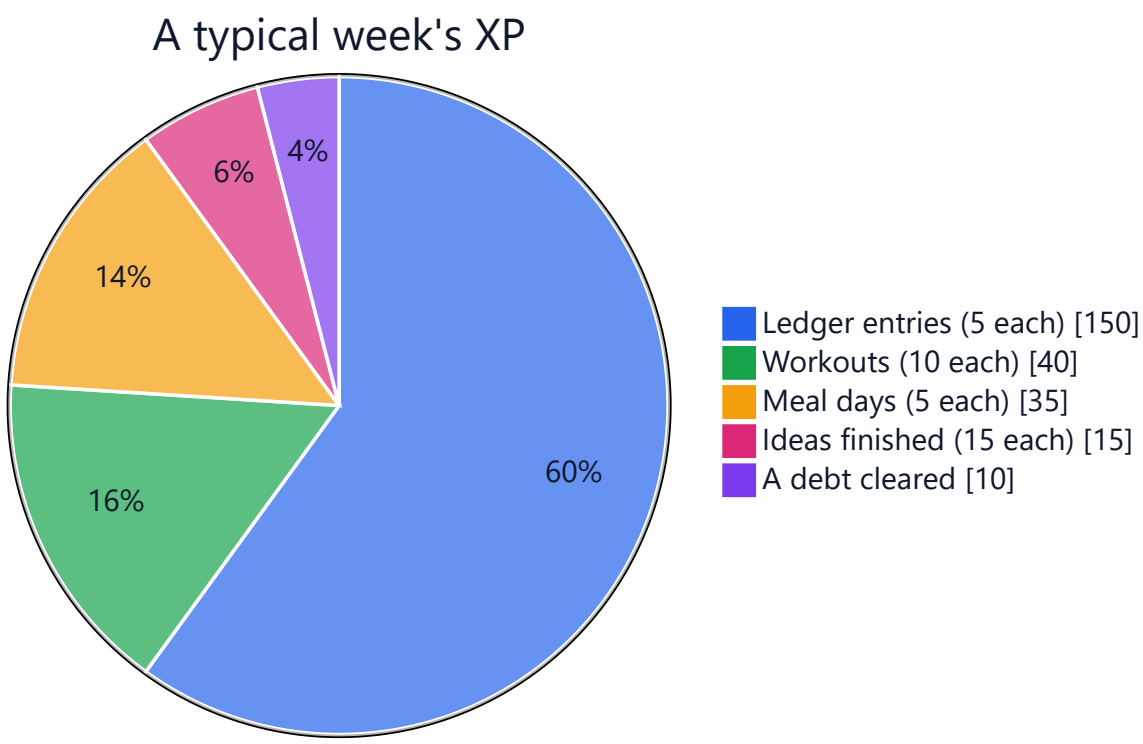
XP

Action	XP	Limit
Log a ledger entry (incl. confirming a recurring one)	5	first 10 entries a day
Log a meal	5	once per day
Tick a workout	10	once per workout per day (unticking and re-ticking pays nothing)
Finish an idea	15	once per idea
Keep a budget for a finished month	50	per budgeted category per month, up to a year back
Complete a quest	200	once per quest
Clear a debt in full	10	once per debt

Entries booked automatically by the nightly job earn nothing: XP is for things you did. Every reward is recorded once in `xp_event`, so nothing can be paid twice.

Where a typical active week's XP comes from (an illustration, not a rule):

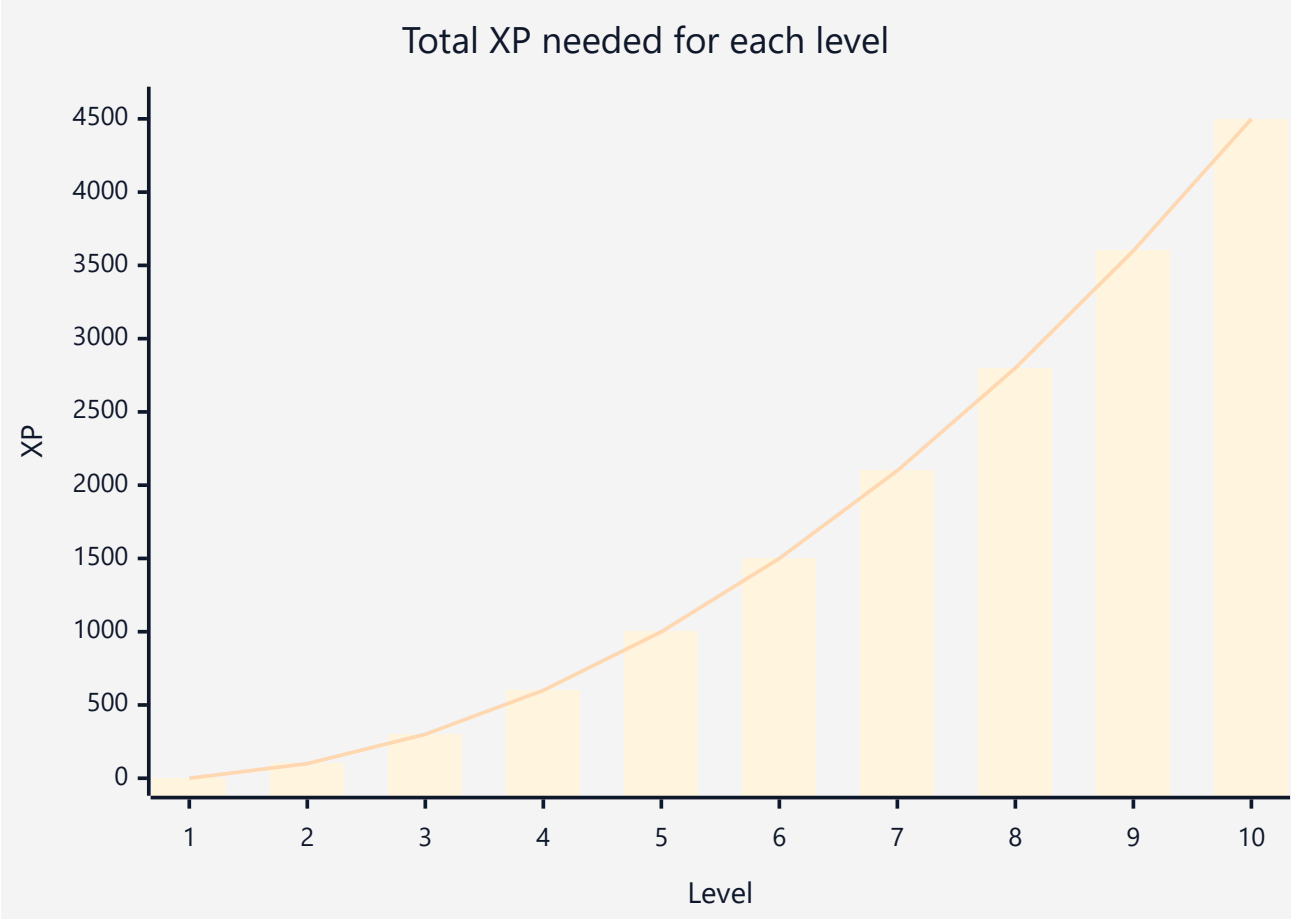
FIGURE 10 · XP



Levels

Reaching level *n* takes `50 × n × (n − 1)` XP in total: 0, 100, 300, 600, 1000, 1500, … Each level needs 100 more than the one before, so early levels come fast and later ones need real habits.

FIGURE 11 · LEVELS



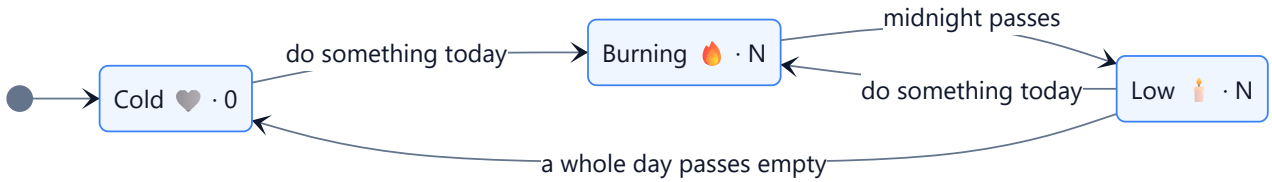
Level	Title
1	Wanderer
2	Villager
3	Apprentice
4	Journeyman
5	Adept
6	Veteran
7	Champion
8	Keeper of the Realm
9+	Legend of KrugWorld

Streaks

A day counts if you earned any *activity* XP that day: an entry, a meal, a workout or a finished idea. The streak is the number of days in a row up to today; today still counts as "in progress", so the streak doesn't drop to 0 until a whole day passes empty. Days follow the app's time zone (Asia/Kuala_Lumpur by default).

The flame on your avatar shows which state you're in:

FIGURE 12 · STREAKS



Badges

Badge	How
First coin	Log your first entry
Week warrior	Be active 7 days in a row
Iron habit	Be active 30 days in a row
Budget keeper	Finish a month within a budget
Quest complete	Reach a savings goal
No-spend Sunday	Spend nothing on a Sunday you were tracking (something logged within 3 days)
Hoard of a thousand	Hold RM 1,000 across your accounts
First feast	Log a meal
Iron body	Finish 10 workouts
Idea smith	Finish an idea in the Workshop
Clean slate	Have a debt paid back in full
Adept	Reach level 5

CHAPTER 03

Flows in Pictures

Sign-in, password reset, pairing, import and a diagram index

Every diagram in the documentation, plus the journeys that don't belong to one page. The diagrams are Mermaid, which GitHub renders straight from the Markdown (in VS Code, install a Mermaid preview extension).

Diagram index

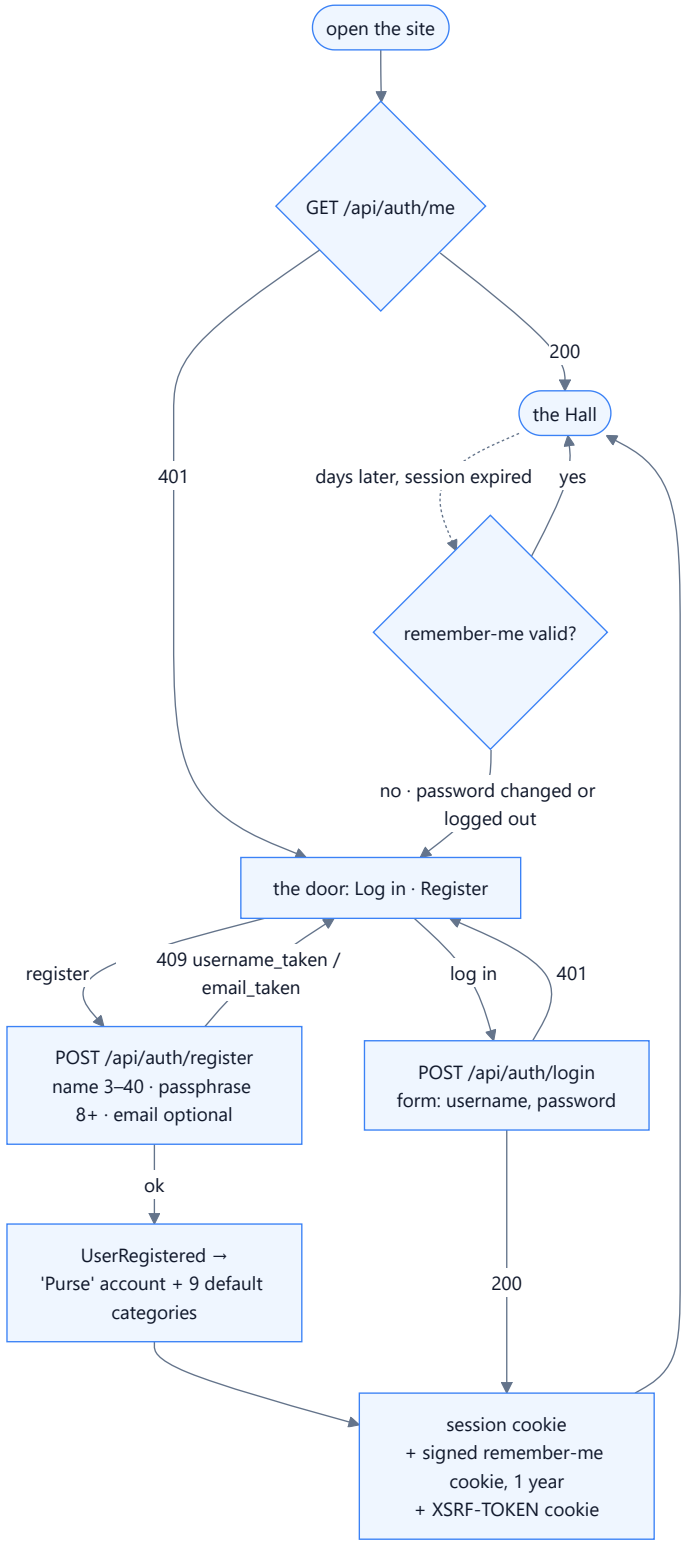
Topic	Diagram	In
The whole system	flowchart: clients → nginx → Spring → Postgres / AI	ARCHITECTURE
Logging an entry end to end	sequence	ARCHITECTURE
Layers	flowchart: controller → service → JPA / JdbcTemplate	ARCHITECTURE
How XP is paid	flowchart: events → Rewards → `xp_event`	ARCHITECTURE
Nightly recurring job	flowchart	ARCHITECTURE
Which security chain	decision tree	ARCHITECTURE
Frontend writes and structure	flowcharts	ARCHITECTURE
Schema history	timeline	DATABASE
Every table	4 ER diagrams: ledger · debts and quests · game and life · security	DATABASE
Derived numbers	flowchart: balance, outstanding, mana, level, streak	DATABASE
App map	mind map	FEATURES
A day in KrugWorld	user journey	FEATURES
The Hall's layout and breakpoints	block diagram + flowchart	FEATURES
Recurring entry lifecycle	state diagram	FEATURES
Sealing a bill	sequence	FEATURES
Quest lifecycle	state diagram	FEATURES
Debt lifecycle and repayment rules	state diagram + flowchart	FEATURES
Where XP comes from	pie	FEATURES
Level curve	XY chart	FEATURES
Streak flame	state diagram	FEATURES
Gaby's tool loop	sequence	GABY

Topic	Diagram	In
Gaby's language rule	flowchart	GABY
Choosing the AI provider	flowchart	GABY
Spend guards	flowchart	GABY
Voice on the desk device	sequence	GABY
Local dev setup	flowchart	DEVELOPMENT
How the tests are wired	flowchart	DEVELOPMENT
Adding a feature	flowchart	DEVELOPMENT
The server	flowchart	OPERATIONS
A release	flowchart: PC → server → health	OPERATIONS
"Not healthy" troubleshooting	decision tree	OPERATIONS
The 23 Oct incident	git graph + timeline	OPERATIONS
Push-to-deploy (proposed)	flowchart	OPERATIONS

The rest are below.

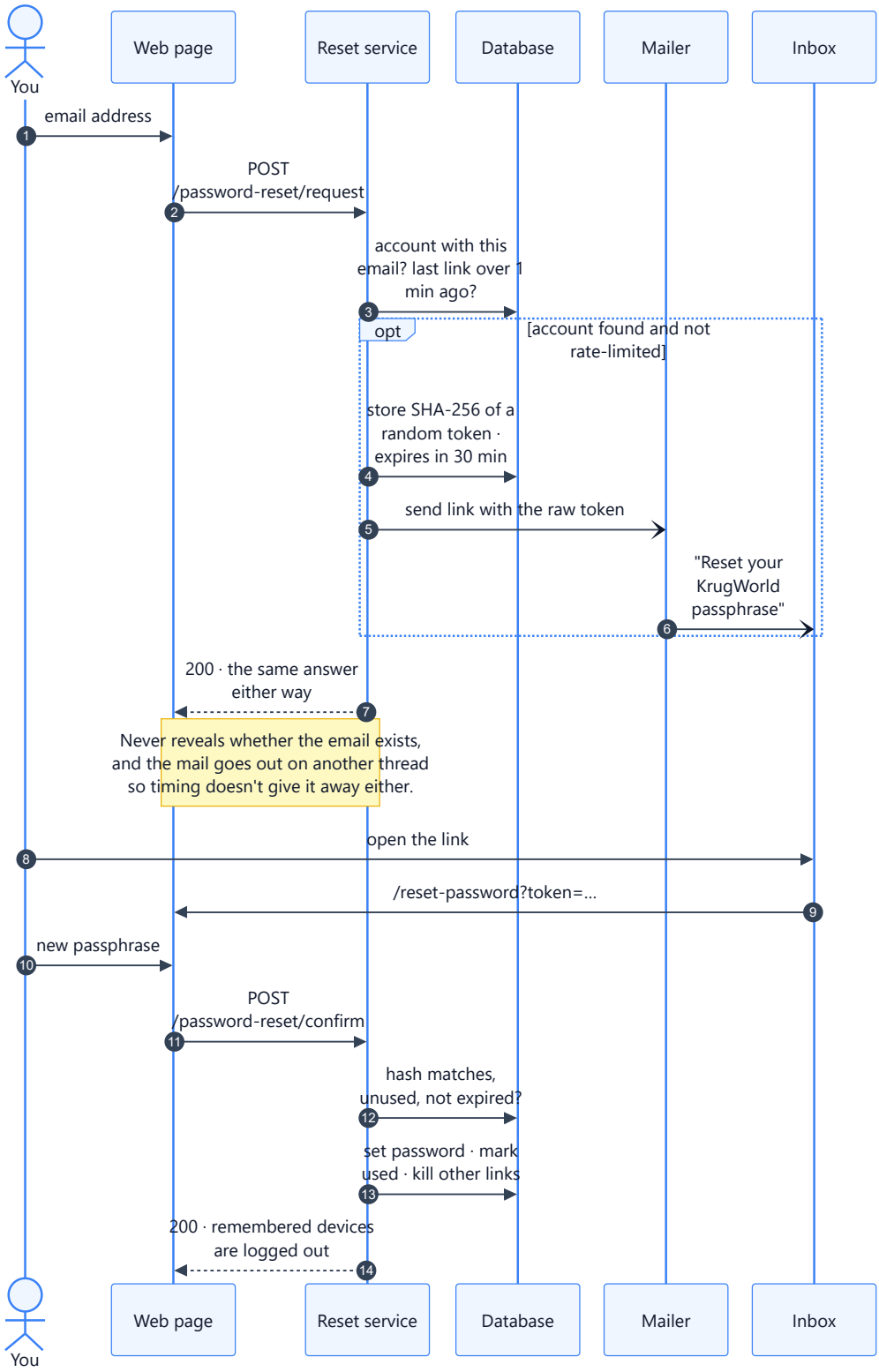
Signing up and staying signed in

FIGURE 13 · SIGNING UP AND STAYING SIGNED IN



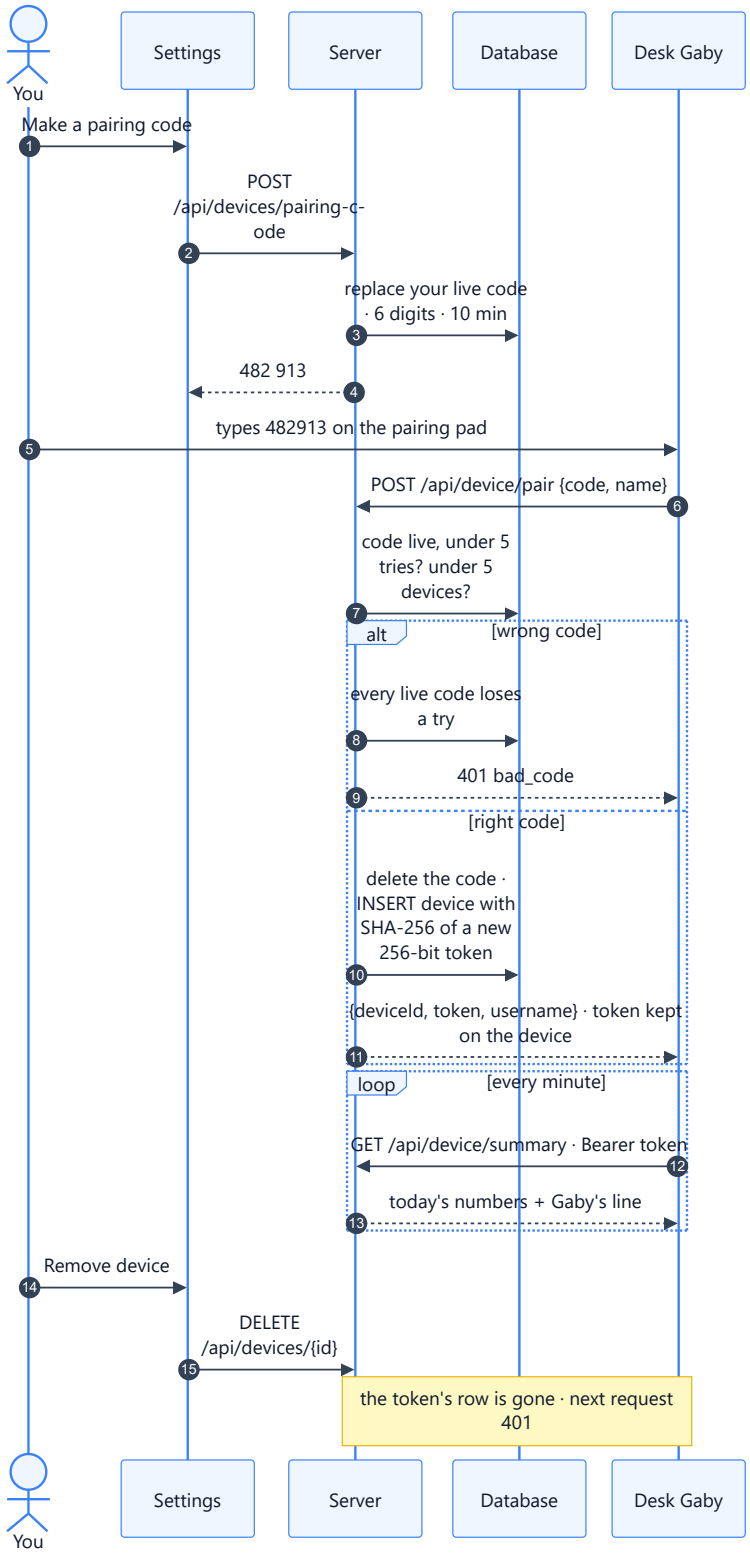
Forgot the passphrase

FIGURE 14 · FORGOT THE PASSPHRASE



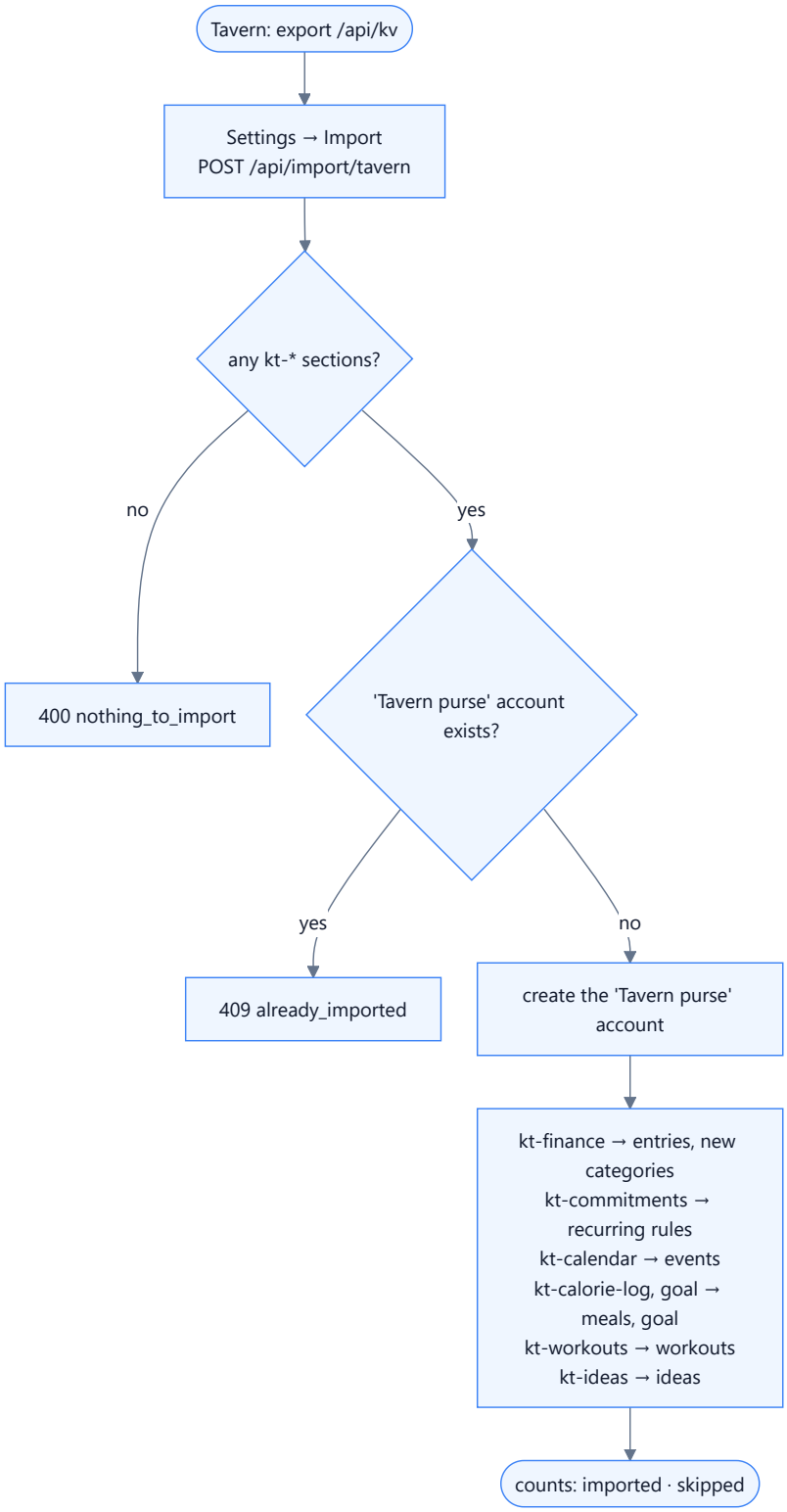
Pairing a desk Gaby

FIGURE 15 · PAIRING A DESK GABY



Importing from Krugger's Tavern

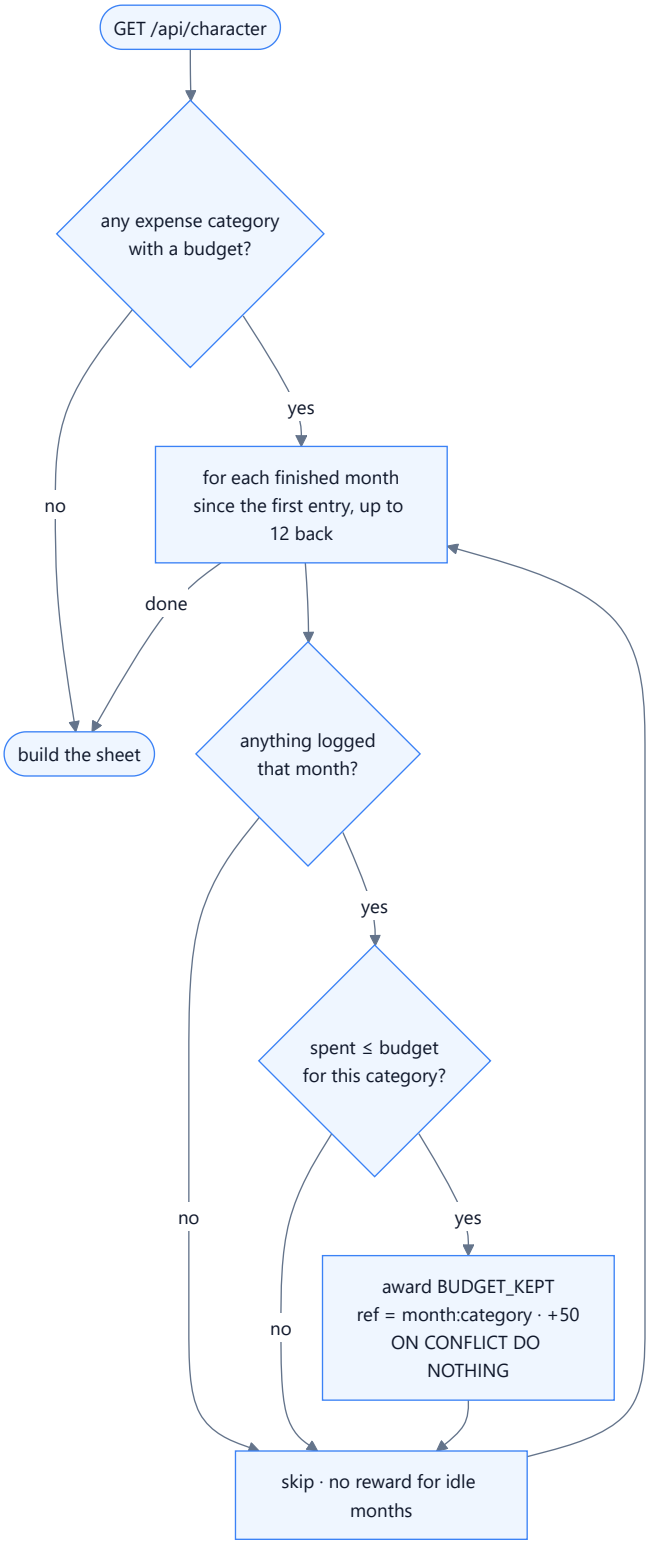
FIGURE 16 · IMPORTING FROM KRUGGER'S TAVERN



"Budget kept" XP, checked lazily

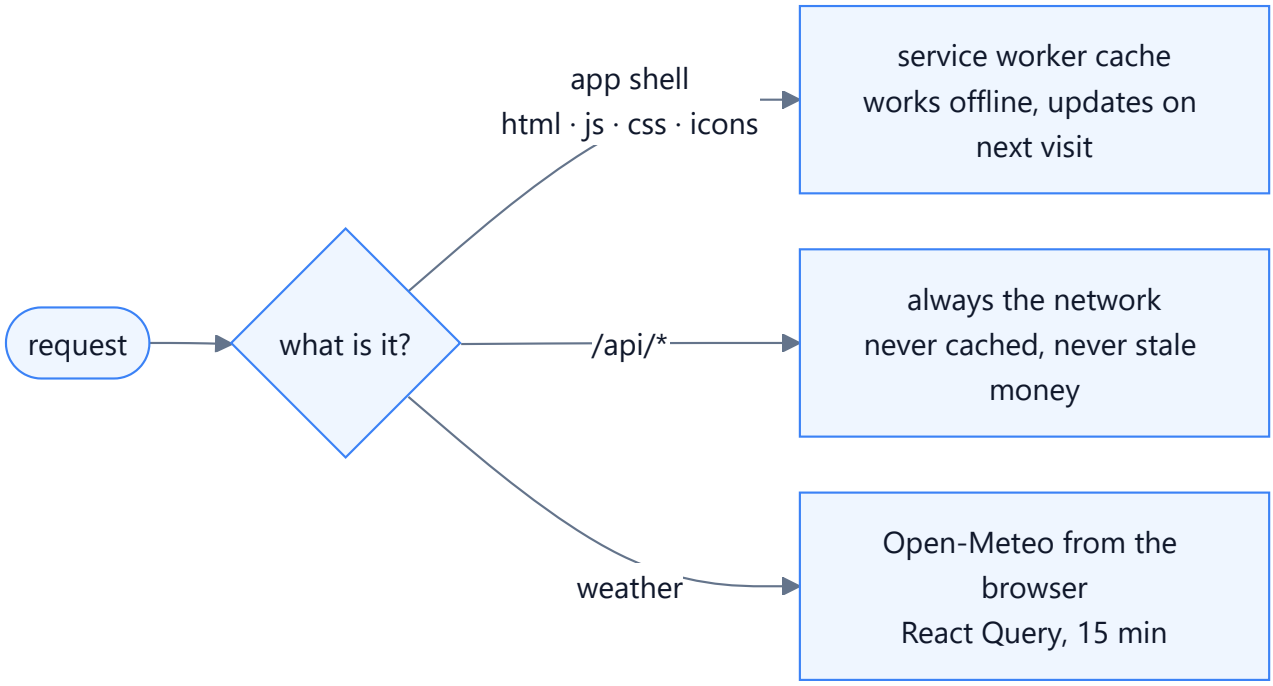
There's no month-end job: the reward is re-checked every time the Hall loads, and the unique key on `xp_event` makes re-checking free.

FIGURE 17 · BUDGET KEPT XP, CHECKED LAZILY



Offline and the PWA

FIGURE 18 · OFFLINE AND THE PWA



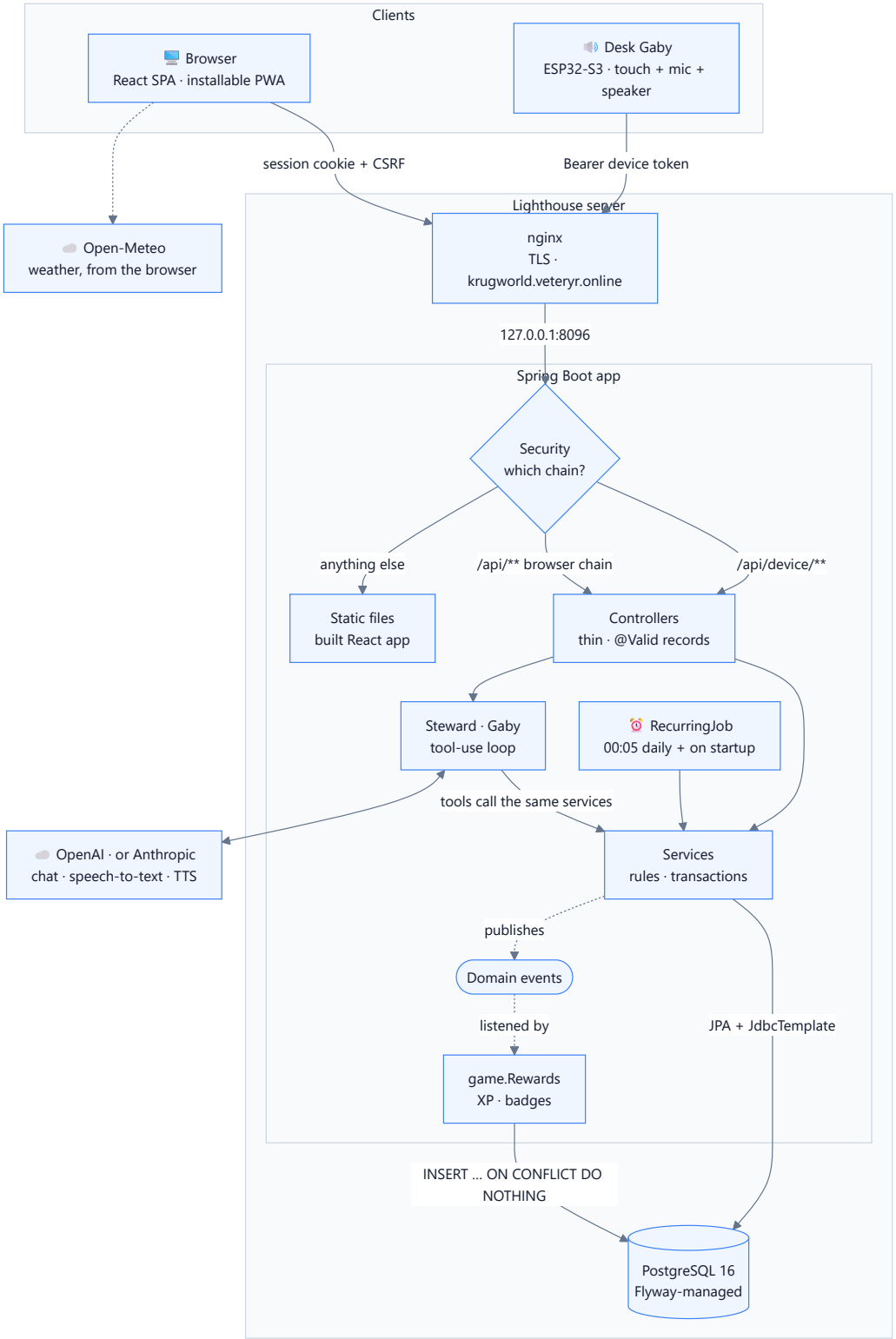
CHAPTER 04

Architecture

Packages, layers, events, scheduling, security, frontend

KrugWorld is one Spring Boot application that serves both the JSON API and the built React app, backed by one PostgreSQL database. A desk device (an ESP32 "desk Gaby") talks to the same server through its own small API.

FIGURE 19 · ARCHITECTURE



Why one jar

Shipping the frontend inside the backend jar means one artifact, one container, one port, and no CORS: the app and the API share an origin, so the session cookie and the CSRF cookie just work. In development Vite serves the frontend and proxies `/api` to Spring, which keeps the same single-origin shape.

Backend

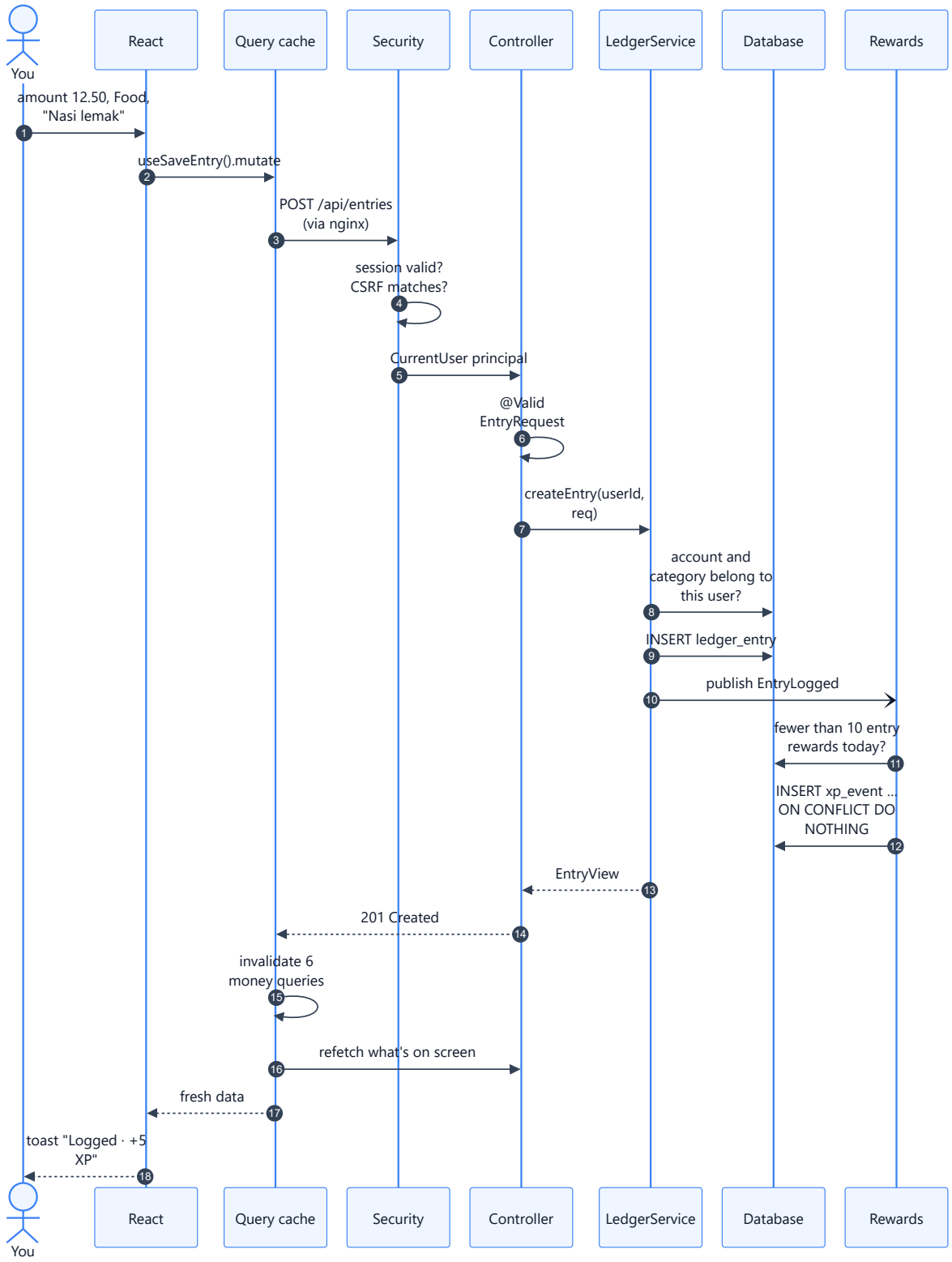
Spring Boot 3.3 on Java 17. Each package is one feature and holds its own controller, service, entities and repositories:

Package	What it owns
`auth`	Users, login/registration, remember-me, preferences, password reset by email
`ledger`	Accounts, categories, ledger entries, month summaries
`budget`	Monthly budgets per expense category ("mana")
`recurring`	Monthly recurring entries, the nightly job, confirm/skip
`debt`	Debts between the user and other people, repayments
`quest`	Savings goals ("quests")
`game`	XP log, levels, badges, streaks, the character sheet
`calendar`	Calendar events
`vitals`	Meals/calories and the training plan
`workshop`	The idea board
`life`	"Today" aggregate (meals, workouts, upcoming events, ideas)
`steward`	Gaby: chat providers, tools, usage limits, speech
`device`	Desk device pairing, device API, Gaby's daily line
`importer`	One-off import from Krugger's Tavern
`web`	Forwards SPA routes to `index.html`
`status`	`/api/status` health and schema version
`common`	API errors, clock/time zone, scheduling switch

Life of a request

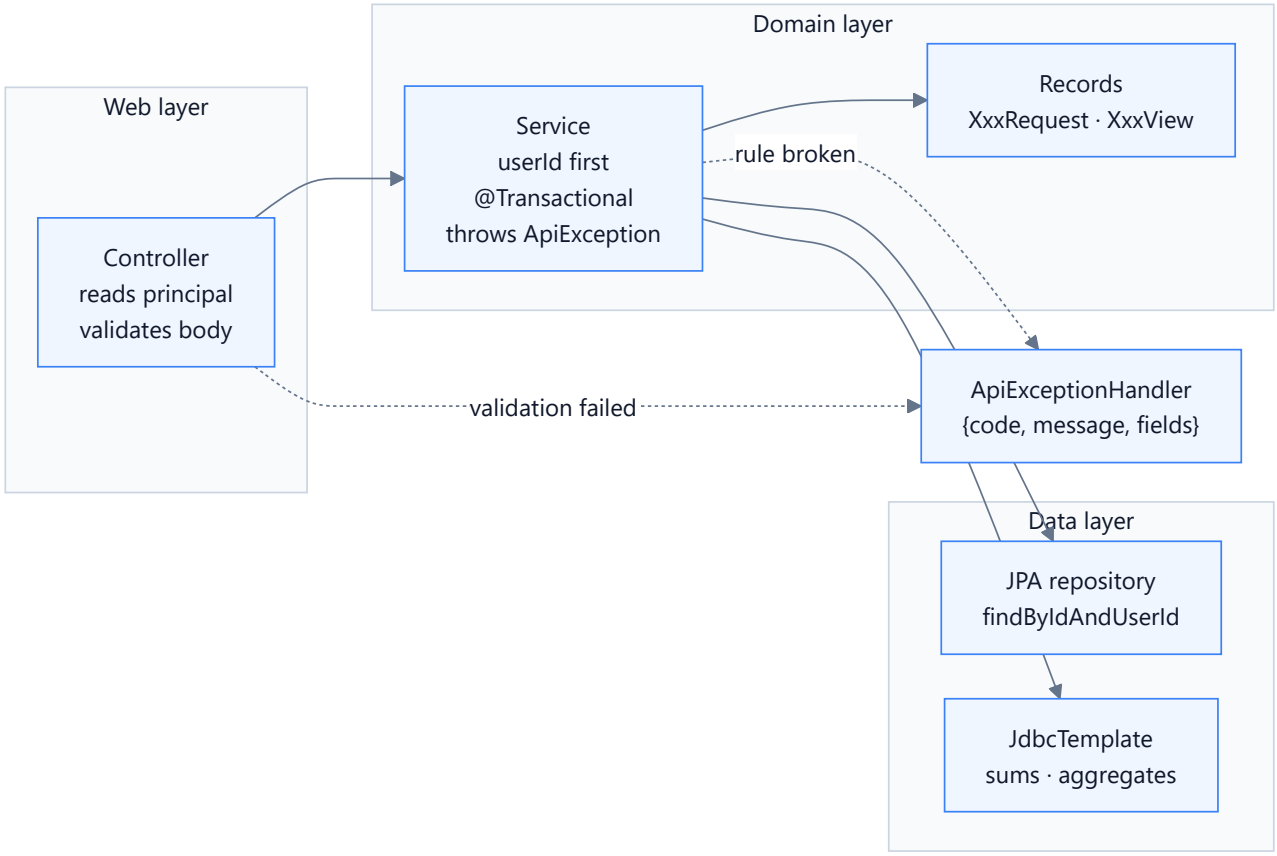
What happens when you log a RM 12.50 nasi lemak:

FIGURE 20 · LIFE OF A REQUEST



Layers and conventions

FIGURE 21 · LAYERS AND CONVENTIONS

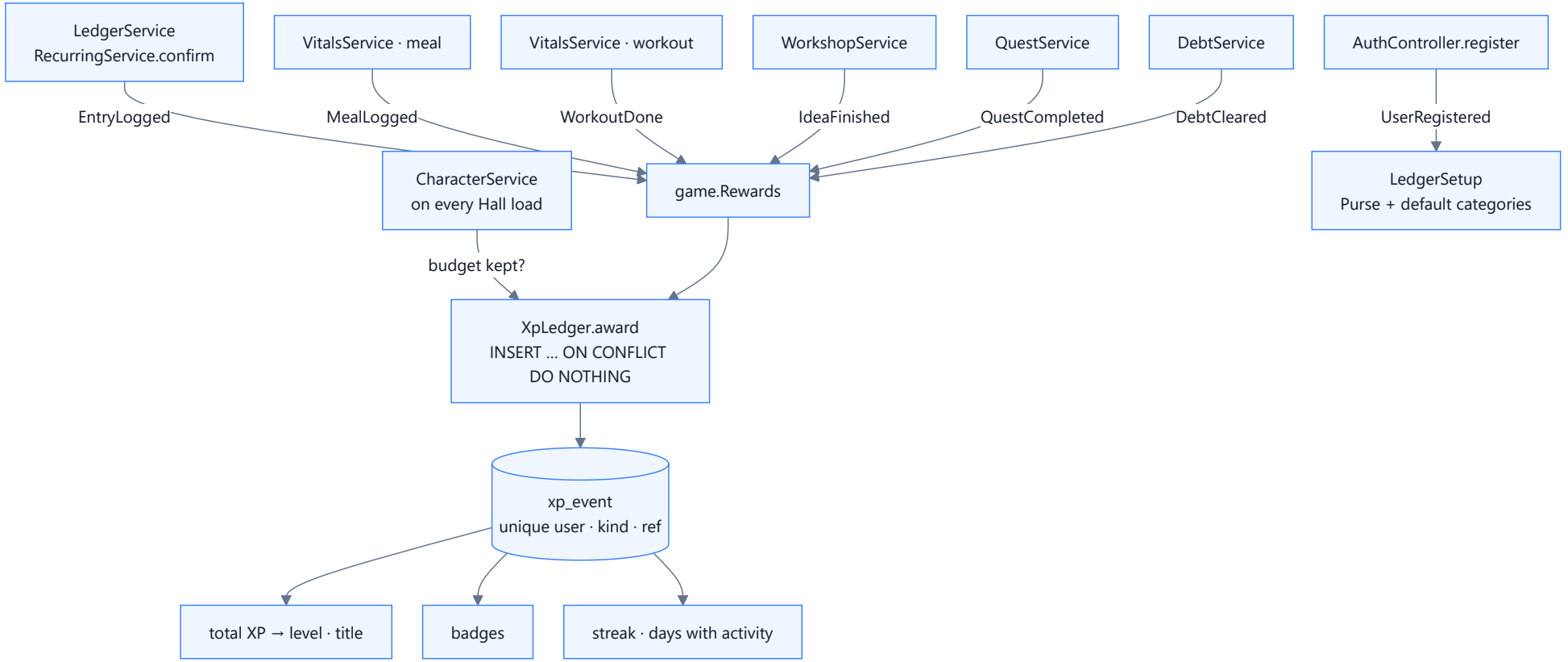


- **Controllers** are thin: they read the principal, validate the body (`@Valid` on records) and call one service method.
- **Services** own the rules and the transaction (`@Transactional` on the class). Every method takes the user id first, and every lookup by id also filters by user (`findByIdAndUserId`), so another user's row looks exactly like a missing one (404).
- **Request/response shapes** are Java records next to the service (`DebtService.DebtRequest` , `DebtService.DebtView`). Entities never leave the service.
- **JPA** handles single-row writes; **JdbcTemplate** handles aggregates (balances, totals by category, repaid amounts), where plain SQL is clearer and faster than loading entities.
- **Money** is `NUMERIC(14,2)` in the database and `BigDecimal` in Java. The frontend sends amounts as two-decimal strings, so no floating-point rounding reaches the database.
- **Derived numbers are never stored.** Account balances are opening balance plus entries; a debt's outstanding amount is the amount minus repayments. Both are summed in SQL on read, so they can't drift from the history.
- **Errors** are always `{ "code", "message", "fields"? }` (`ApiExceptionHandler`). Services throw `ApiException.badRequest / notFound / conflict` ; the frontend switches on `code` and shows `message` .

Domain events and rewards

Features don't call the game directly. They publish a small event and `game.Rewards` listens:

FIGURE 22 · DOMAIN EVENTS AND REWARDS



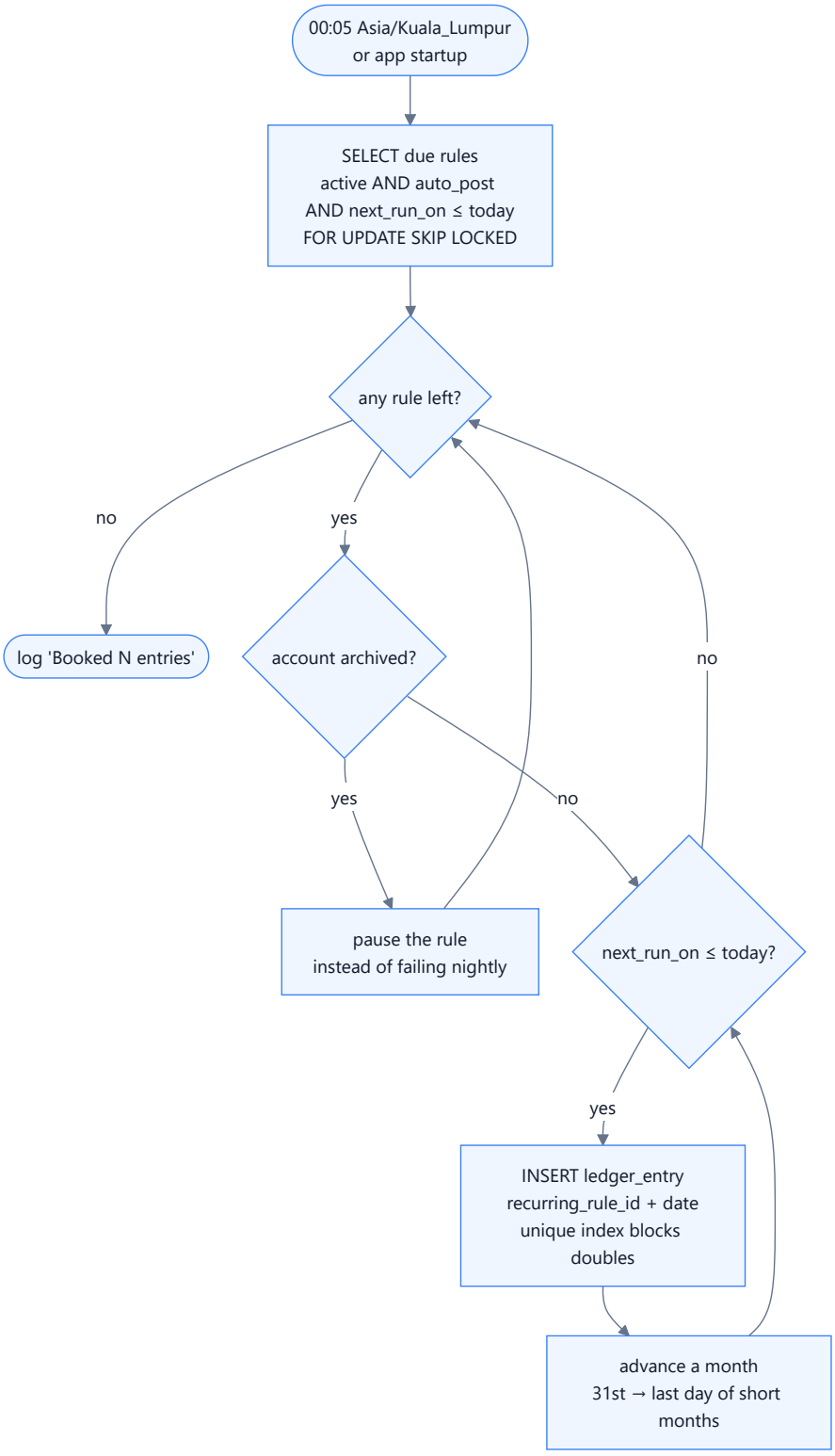
Event	Published when	Reward
`EntryLogged`	A person logs an entry (or confirms a recurring one)	+5 XP, first 10 a day
`MealLogged`	A meal is logged	+5 XP, once per day
`WorkoutDone`	A workout is ticked	+10 XP, once per workout per day
`IdeaFinished`	An idea reaches DONE	+15 XP, once per idea
`QuestCompleted`	A quest reaches its target	+200 XP, once per quest
`DebtCleared`	A debt is repaid in full	+10 XP, once per debt
`UserRegistered`	An account is created	default account and categories (`LedgerSetup`)

Every reward is an `INSERT … ON CONFLICT DO NOTHING` into `xp_event`, whose unique key is `(user_id, kind, ref)`. That makes paying out **idempotent**: re-checking, a double click or a retried request can never pay twice. "Budget kept" XP (+50 per budget per finished month) uses the same trick: it is re-checked every time the character sheet is opened instead of by a month-end job.

Scheduling

`RecurringJob` runs at 00:05 in the app's time zone (`krugworld.zone`, default `Asia/Kuala_Lumpur`) and once at startup to catch up after downtime. It locks due rules with `FOR UPDATE SKIP LOCKED`, and a partial unique index on `(recurring_rule_id, occurred_on)` makes a double booking impossible even if two instances run at once. Rules set to "wait for my confirm" are skipped by the job.

FIGURE 23 · SCHEDULING



Time is always read through an injected `Clock` (`common/TimeConfig`), so "today", month boundaries and streaks follow the configured zone, not the server's, and tests can control it.

Security

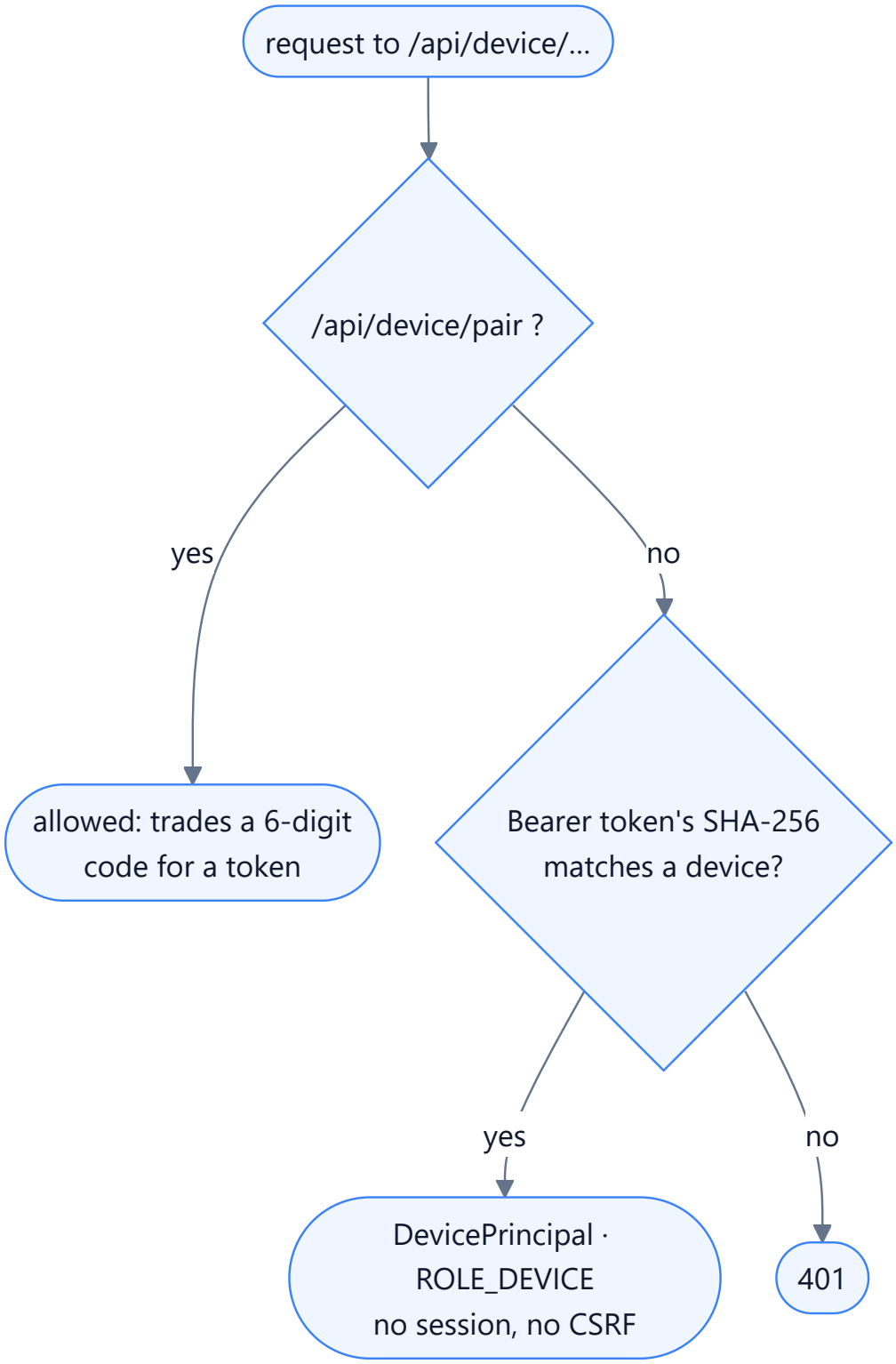
Two filter chains (`auth/SecurityConfig`):

1. **Device chain**, `/api/device/**`, checked first: stateless, bearer token only (`DeviceTokenFilter`), principal `DevicePrincipal`, role `ROLE_DEVICE`. No cookies, so no CSRF. Device tokens are random; only their SHA-256 is stored.
2. **Browser chain**, everything else: session cookie plus a signed, year-long remember-me cookie (its key lives in `app_secret` so deploys don't log everyone out), CSRF via the `XSRF-TOKEN` cookie echoed in the `X-XSRF-TOKEN` header. `/api/**` needs a login except login, register and password reset. A missing login is a bare 401, never a redirect.

Non-API paths are public static files; the SPA asks `/api/auth/me` who you are.

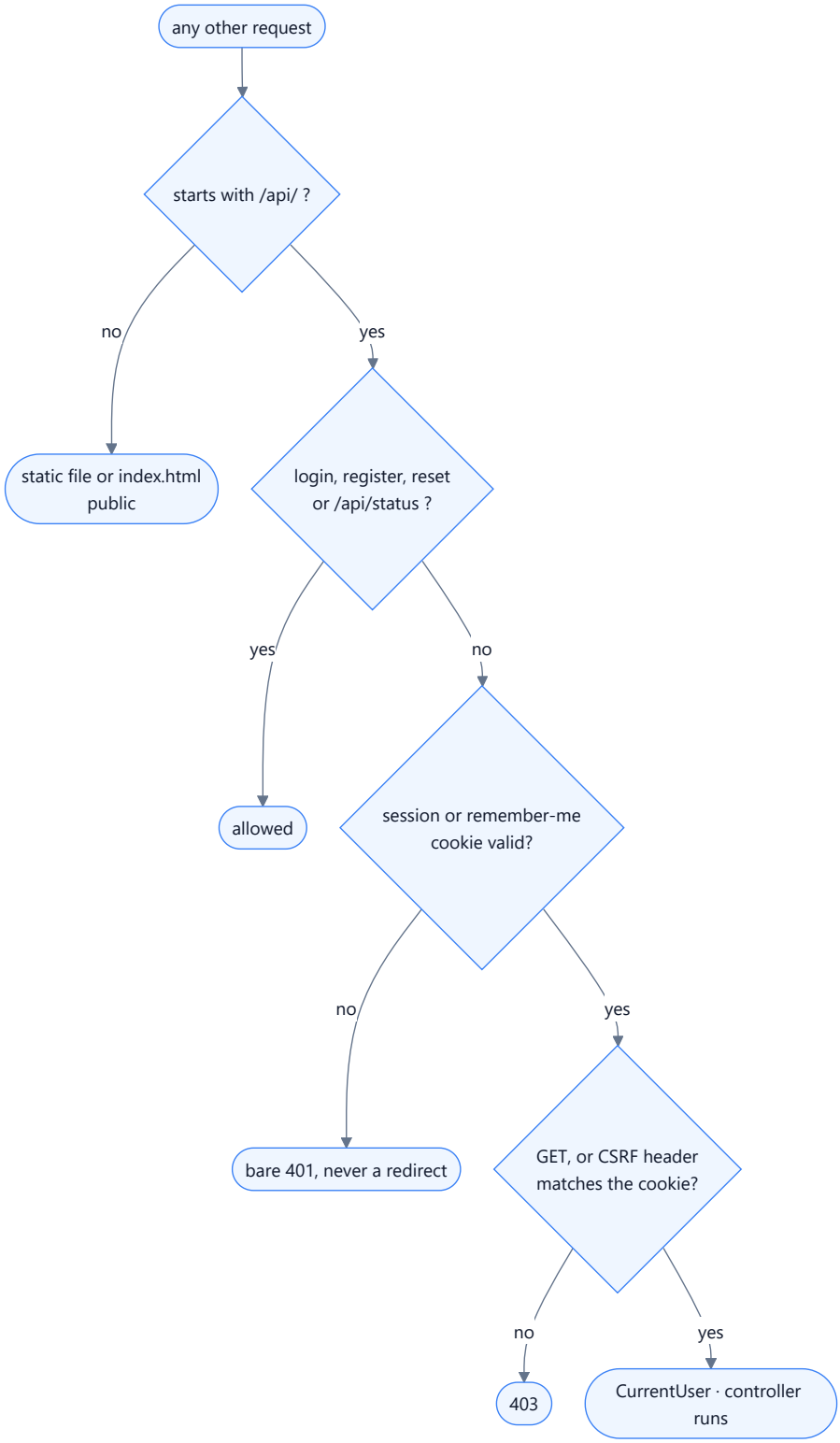
The device chain, checked first:

FIGURE 24 · SECURITY



Everything else, the browser chain:

FIGURE 25 · SECURITY



Gaby (the Steward)

See Gaby, the Steward. In short: a stateless tool-use loop. The browser sends the recent conversation; the server lets the model call up to 8 rounds of tools (thin wrappers over the same services the screens use, so the same rules and XP apply) and returns the reply plus an English subtitle.

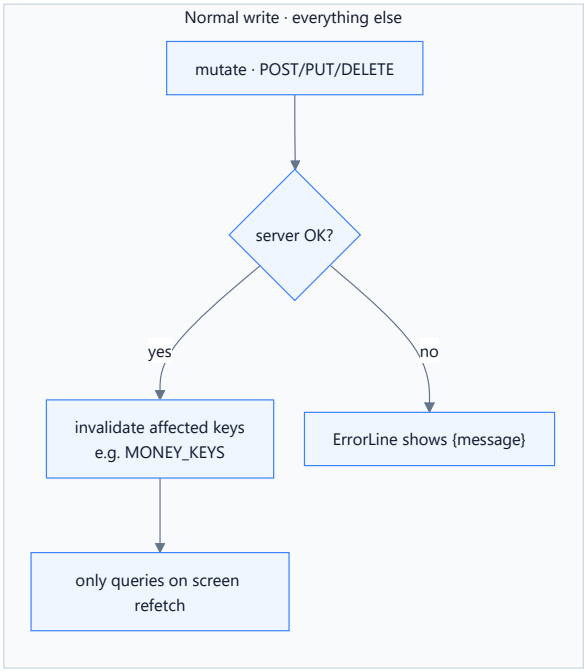
Frontend

React 19 + TypeScript + Vite 8, React Query for server state, react-router, Tabler icons. `npm run lint` (oxlint) is kept at zero warnings.

```
frontend/src/
  App.tsx            routes; every page except the Hall is a lazy() chunk
  api/client.ts      the only fetch wrapper: CSRF header, error -> ApiError
  api/types.ts       TypeScript shapes of the API
  api/hooks.ts       React Query hooks for money features (+ invalidation)
  api/lifeHooks.ts   hooks for calendar, vitals, workshop, steward
  components/        Layout, nav, ui primitives, Hall widgets, dialogs
  pages/             one file per page
  steward/           Gaby's portrait, chat and dock
  skin/skin.ts       applies the account's skin/theme to <html>
  styles/skins.css   skin variables; app.css uses only those variables
```

- **Server state** lives in React Query. After a write, hooks invalidate the keys whose data changed (an entry changes entries, accounts, summary, budgets, the character sheet and recurring). A few writes are **optimistic** (`onMutate` + rollback): ticking a workout and deleting an entry.

FIGURE 26 · FRONTEND



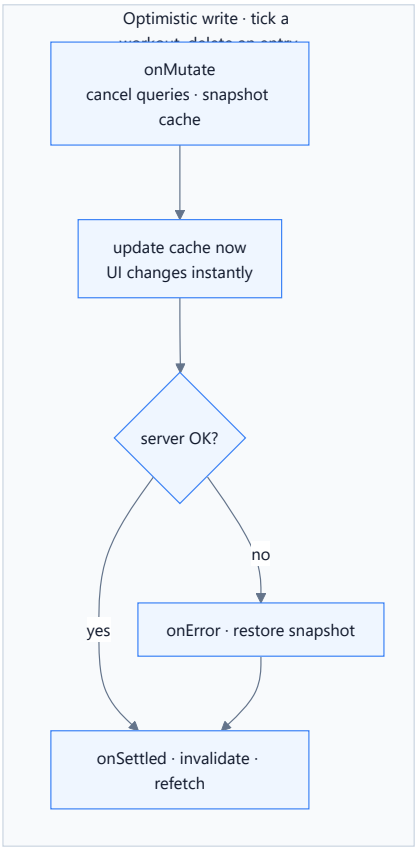
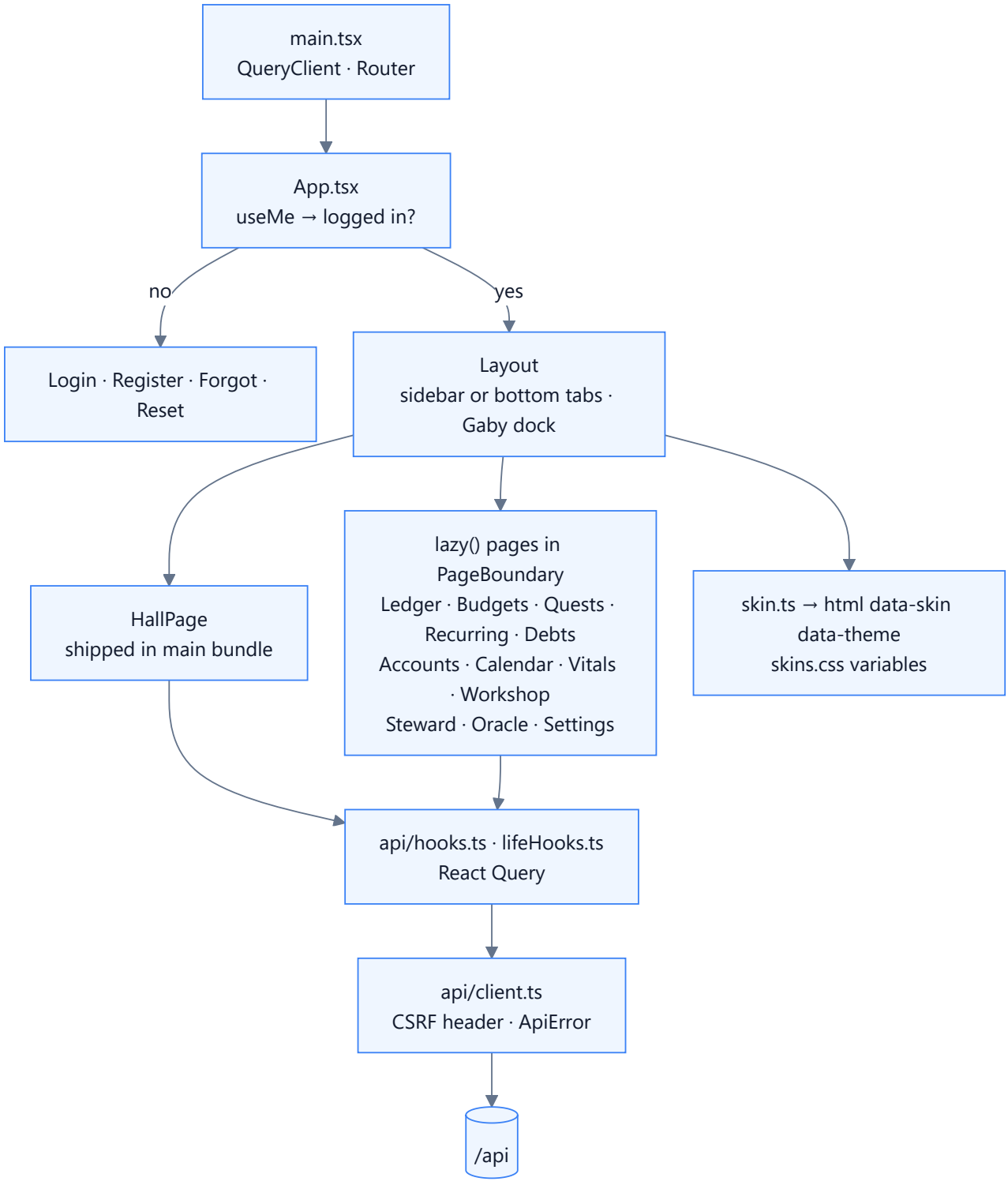


FIGURE 27 · FRONTEND



- **Skins** are CSS variables on `<html data-skin data-theme>`. Components only use the variables, so a new theme is a block of variables, not a component change. *Classic Krugger* reproduces the old Tavern exactly; the KrugWorld skin has four backgrounds.
- **Layout**: grouped navigation (Hall · Finance · Life · Arcane · Settings); on phones each group is a tab in a bottom bar. The Hall's grid uses container queries on `main`, so it adapts to the room actually left when Gaby's dock is open.
- **PWA**: `vite-plugin-pwa` caches the app shell only. `/api` is never cached, so the app never shows stale money.

- **SPA routes**: any new route must also be added to `web/SpaController` so a refresh on that URL serves `index.html` instead of a 404.

Desk device

`firmware/` holds the PlatformIO + Arduino + LVGL firmware for the Waveshare ESP32-S3-Touch-LCD-4B. It pairs with a six-digit code from Settings, shows a glance screen (Gaby's face, today's numbers, workouts to tick) and does tap-to-talk voice. Its API is documented in Desk Device API.

Deployment shape

One `postgres:16` container and one app container (`eclipse-temurin:17-jre` running the jar) per `deploy/docker-compose.yml`, on a shared Lighthouse server behind the host's nginx and Let's Encrypt. The jar is built on a developer machine; nothing is compiled on the server. See Operations and `../DEPLOY.md`.

CHAPTER 05

Database

Migrations, tables, relationships and derived numbers

PostgreSQL 16. The schema is owned by **Flyway** (`backend/src/main/resources/db/migration`); Hibernate runs with `ddl-auto=validate` , so it only checks that the entity classes match what Flyway built and refuses to start if they don't.

Rules for changing the schema

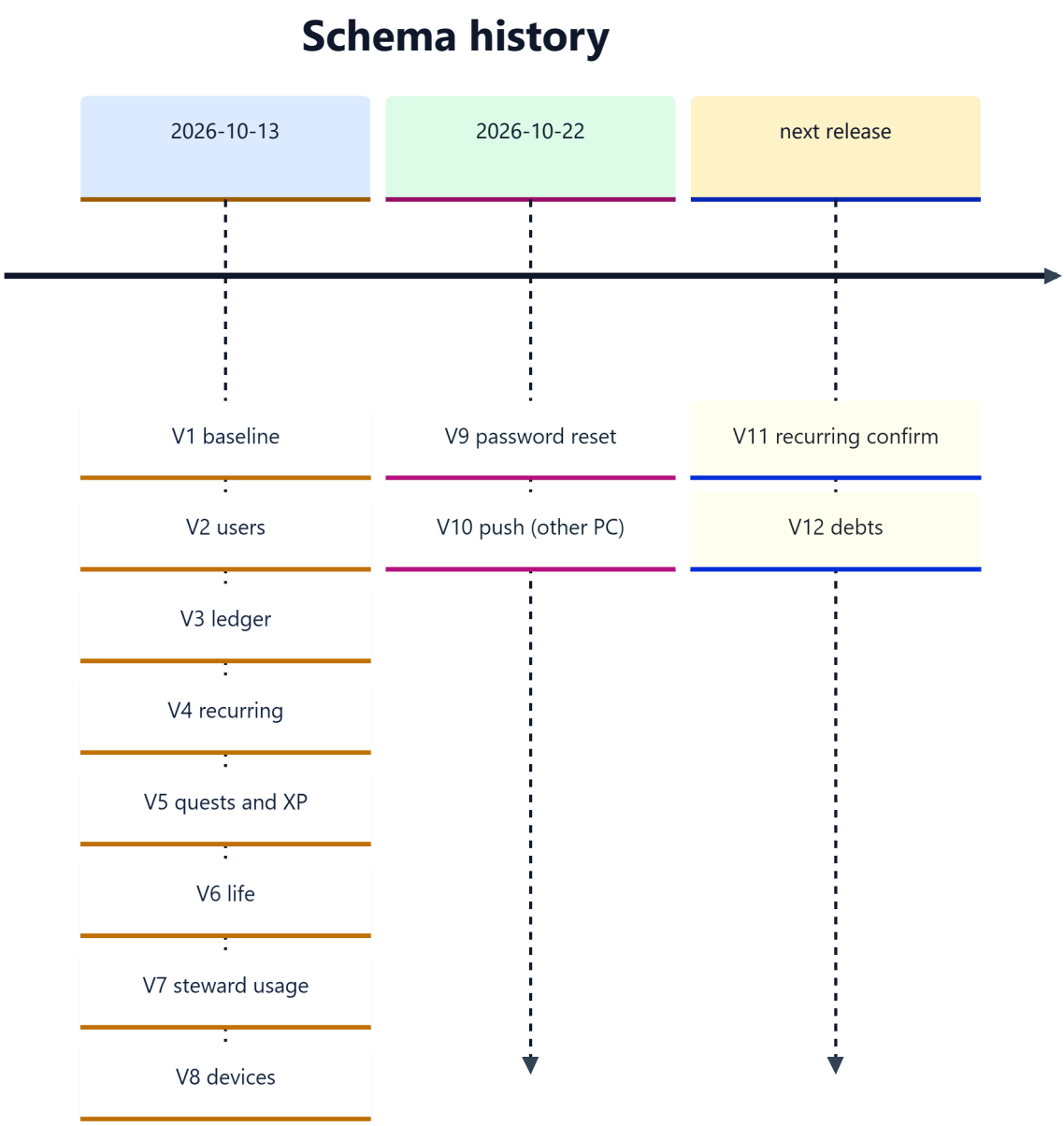
- **Never edit a migration that has run anywhere** (your PC, the server). Flyway stores a checksum of every applied file and will refuse to start if one changes. Add a new `V<n>__what.sql` instead.
- **Pick the next number from production, not from your checkout.** Two machines that each add "the next" migration will both create the same `V<n>` , and the second deploy fails with *checksum mismatch* (this happened on 2026-10-23; see Operations). Pull first; check `SELECT max(version::int) FROM flyway_schema_history` on the server if in doubt.
- Put the "why" in a comment at the top of each migration, and enforce rules in the database too (CHECK constraints, unique indexes), not only in Java.

Migrations

Version	File	Adds
1	`V1__baseline.sql`	`app_meta`
2	`V2__users.sql`	`app_user` , `app_secret`
3	`V3__ledger.sql`	`account` , `category` , `ledger_entry`
4	`V4__recurring.sql`	`recurring_rule` , `ledger_entry.recurring_rule_id`
5	`V5__quests_and_xp.sql`	`goal` , `xp_event`
6	`V6__life.sql`	`calendar_event` , `idea` , `meal` , `workout` , `workout_completion` , user settings
7	`V7__steward_usage.sql`	`steward_usage`
8	`V8__devices.sql`	`device` , `device_pairing`
9	`V9__password_reset.sql`	`app_user.email` , `password_reset`
10	`V10__push.sql`	push notifications (`push_subscription`). **Deployed from the other PC, not yet in this repo**; document it here when that code is merged.
11	`V11__recurring_confirm.sql`	`recurring_rule.auto_post`
12	`V12__debts.sql`	`debt` , `debt_payment`

Every table that belongs to a person has `user_id … REFERENCES app_user (id) ON DELETE CASCADE`, so deleting an account removes everything it owned.

FIGURE 28 · MIGRATIONS

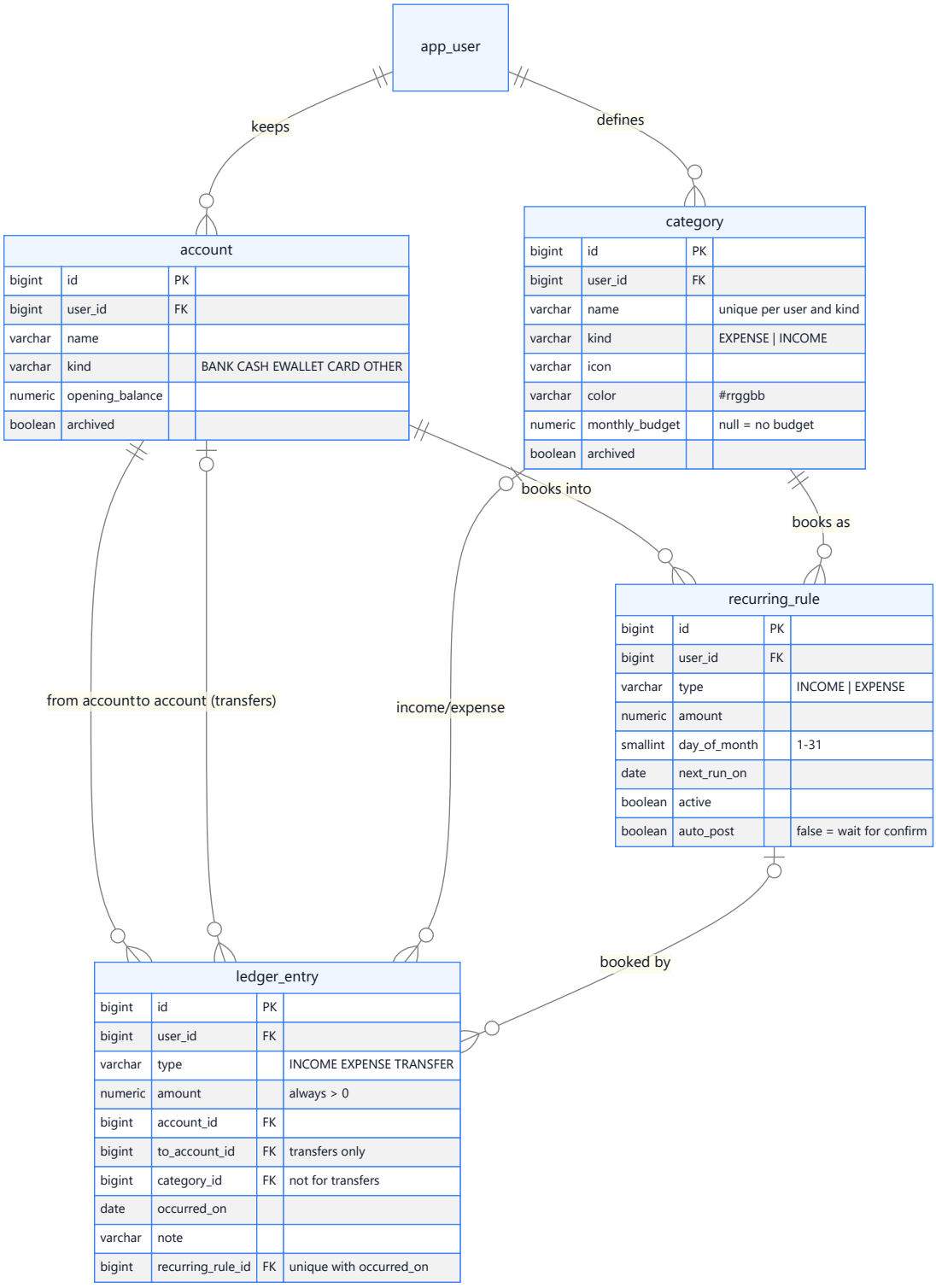


Entity relationships

Every table hangs off `app_user`. Split into three pictures so each stays readable.

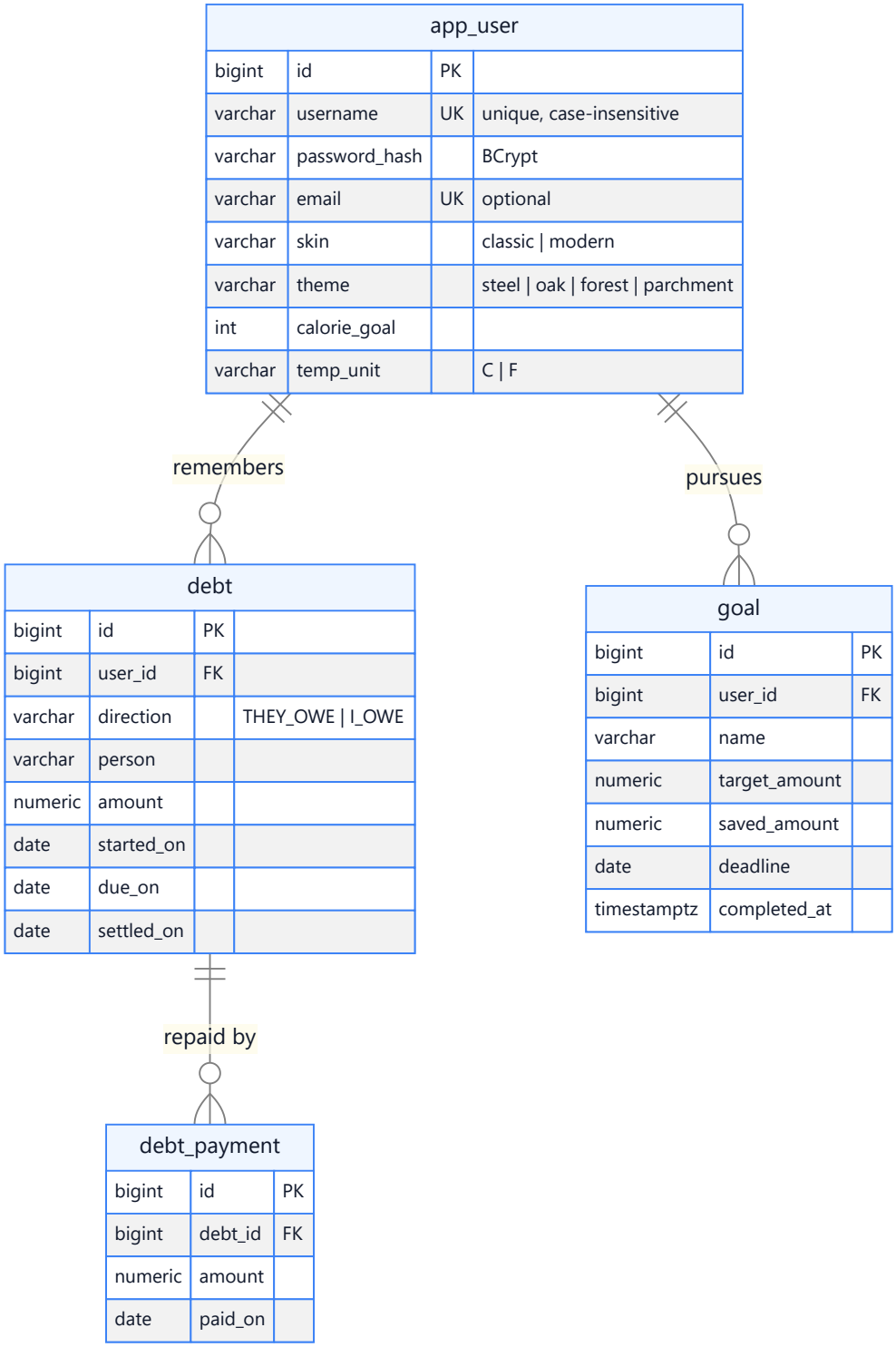
Ledger and recurring

FIGURE 29 · LEDGER AND RECURRING



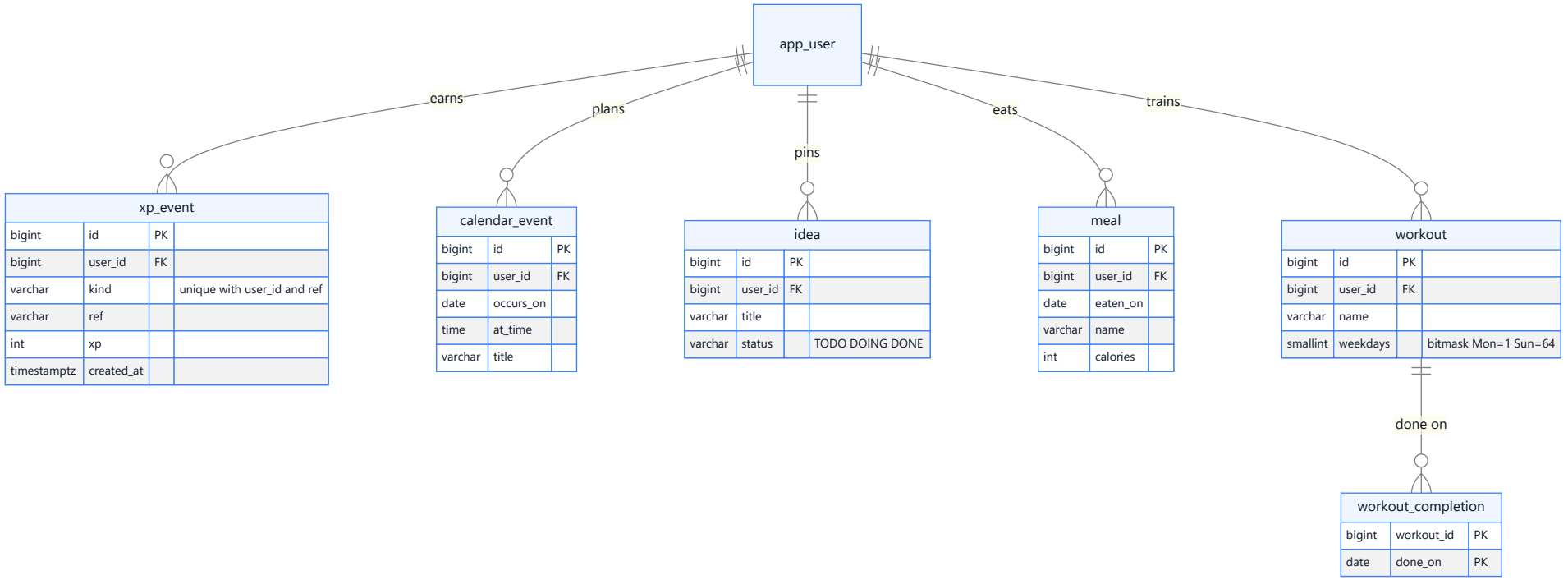
Debts and quests

FIGURE 30 · DEBTS AND QUESTS



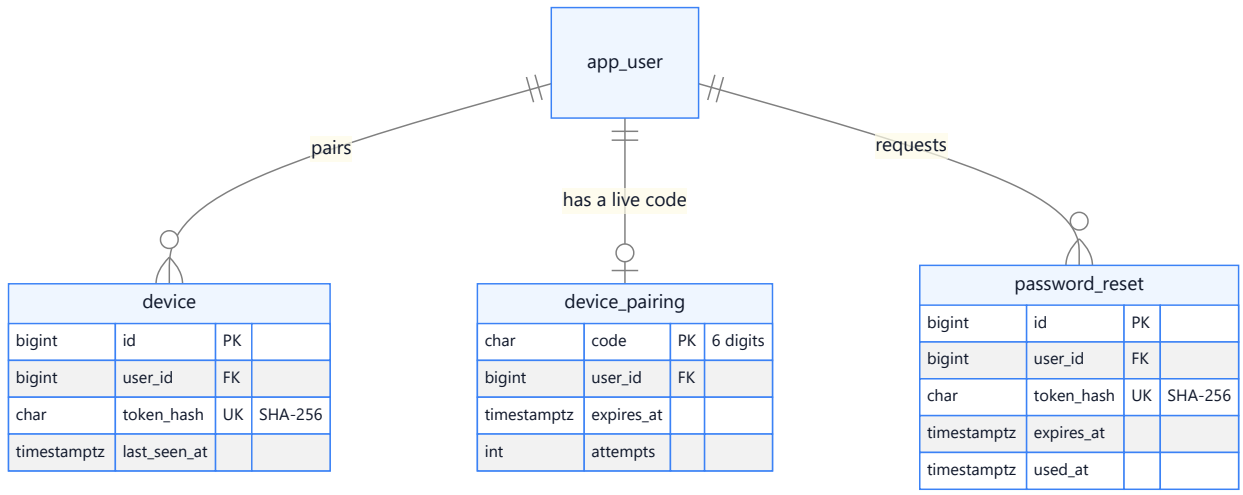
Game and life

FIGURE 31 · GAME AND LIFE



Accounts, devices and security

FIGURE 32 · ACCOUNTS, DEVICES AND SECURITY

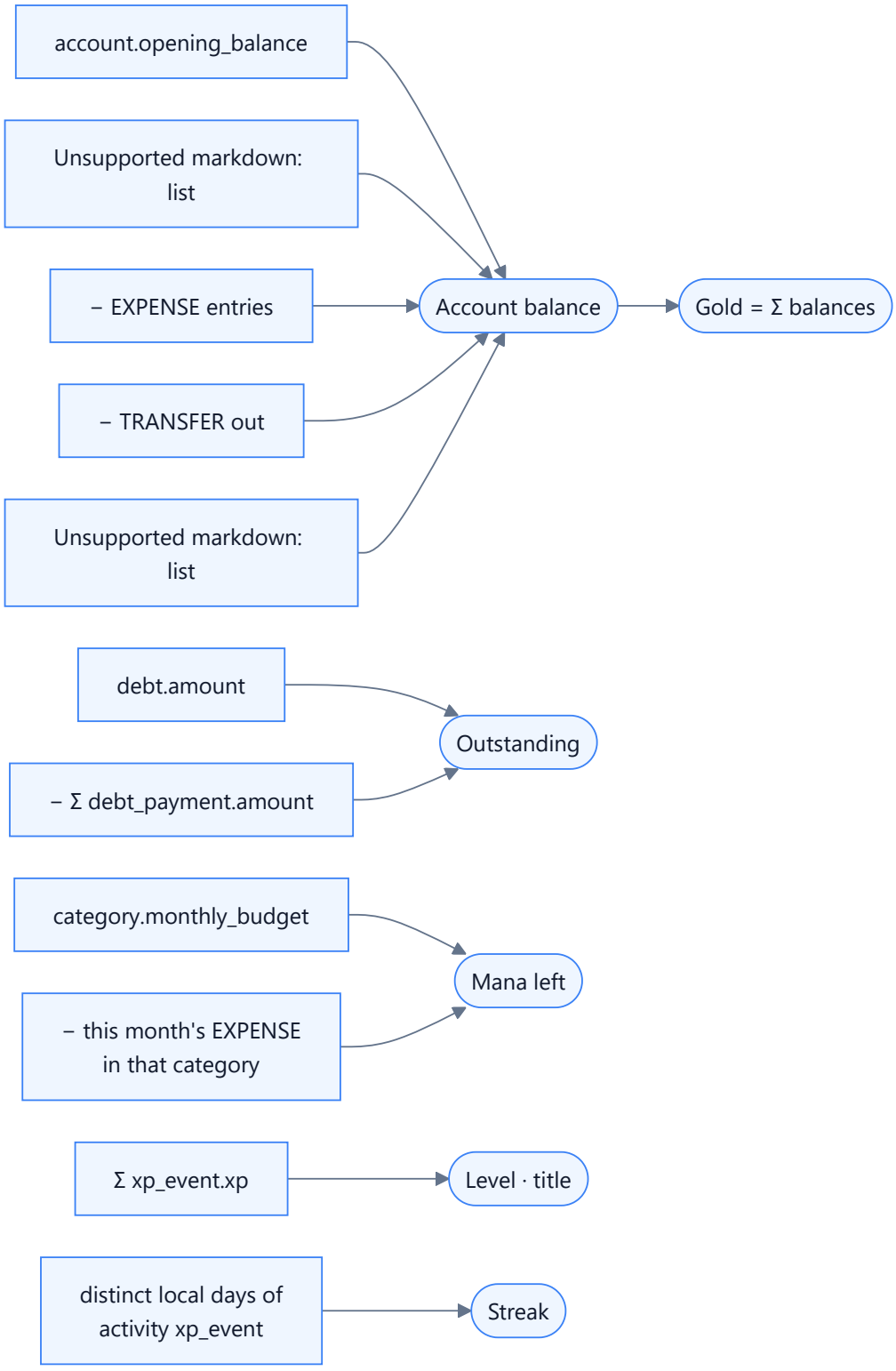


`app_secret`, `steward_usage` and `app_meta` stand alone (no foreign keys).

How derived numbers are computed

Nothing below is stored; each is summed from history on read.

FIGURE 33 · HOW DERIVED NUMBERS ARE COMPUTED



Tables

Accounts and security

`app_user`: one row per login.

Column	Type	Notes
`id`	BIGSERIAL PK	
`username`	VARCHAR(40)	unique ignoring case (`ux_app_user_username` on `lower(username)`)
`password_hash`	VARCHAR(100)	BCrypt
`email`	VARCHAR(254) NULL	optional, unique ignoring case; needed only for password reset
`skin`	VARCHAR(20)	`classic` or `modern`
`theme`	VARCHAR(20)	`steel` , `oak` , `forest` , `parchment`
`calorie_goal`	INTEGER	500–10000, default 2000
`weather_place` , `weather_lat` , `weather_lon`		optional location for the Hall's weather
`temp_unit`	VARCHAR(1)	`C` or `F`
`created_at`	TIMESTAMPTZ	

`app_secret` : secrets generated on first boot (e.g. the remember-me signing key), kept in the database so redeploys don't log everyone out.

`password_reset` : one row per reset link. Only the SHA-256 of the token is stored; `used_at` makes a link single-use and `expires_at` limits it to 30 minutes.

`device` / `device_pairing` : paired desk devices (token stored as SHA-256 only) and short-lived six-digit pairing codes (10 minutes, limited attempts, one live code per user).

`steward_usage` : Gaby's daily message counters, `(scope, day)` with scope `user:<id>` or `global` .

Money

`account` : where money lives. `kind` is `BANK` , `CASH` , `EWALLET` , `CARD` or `OTHER` ; `opening_balance` is the balance before the first entry. `archived` hides it from pickers but keeps its history. Deleting an account with entries archives it instead.

`category` : what money was for. `kind` is `EXPENSE` or `INCOME` ; `icon` is one of a fixed set of keys; `color` is `#rrggbb` ; `monthly_budget` (expense only, NULL = no budget) drives Budgets. Unique per user on `(kind, lower(name))` . Deleting a used category archives it and clears its budget.

`ledger_entry` : every movement of money.

Column	Notes
`type`	`INCOME` , `EXPENSE` or `TRANSFER`
`amount`	NUMERIC(14,2), always > 0; the type carries the direction
`account_id`	source account
`to_account_id`	transfers only, must differ from `account_id`
`category_id`	income/expense only; transfers never have one

Column	Notes
`occurred_on`	a calendar DATE, never an instant
`recurring_rule_id`	set when a recurring rule made it

`ck_entry_shape` enforces the transfer/category rule in the database. The ledger screen's query (one user, one month, newest first) is served by `ix_entry_user_date`.

Balance of an account = `opening_balance` + income − expenses − transfers out + transfers in, summed by `LedgerQueries` on read.

`recurring_rule`: monthly income/expense templates.

Column	Notes
`type`, `amount`, `account_id`, `category_id`, `note`	copied onto each entry
`day_of_month`	1–31; 29–31 fall on the last day of shorter months
`next_run_on`	the next date it's due
`active`	paused rules do nothing
`auto_post`	`true`: the nightly job books it. `false`: it waits for the user to confirm or skip

`ux_entry_rule_date` (unique on `(recurring_rule_id, occurred_on)` where set) guarantees one entry per rule per date, whoever books it.

`goal`: savings quests: `target_amount`, `saved_amount`, optional `deadline`, `completed_at` (set once, never unset).

`debt` / `debt_payment`: promises between the user and other people.

`debt` column	Notes
`direction`	`THEY_OWE` (someone owes the user) or `I_OWE`
`person`	free text, e.g. "Ali"
`amount`	the original amount, > 0
`note`, `started_on`, `due_on`	`due_on` optional
`settled_on`	set when repaid in full or when the user lets the rest go

`debt_payment` holds repayments (`amount > 0`, `paid_on`, `note`), cascading with the debt. **Outstanding** = `amount - SUM(debt_payment.amount)`, computed in SQL. Debts are kept apart from the ledger on purpose: they record who owes whom, not money in an account.

Game

`xp_event`: an append-only log of every reward earned.

Column	Notes
`kind`	`ENTRY_LOGGED`, `MEAL_LOGGED`, `WORKOUT_DONE`, `IDEA_DONE`, `QUEST_COMPLETE`, `BUDGET_KEPT`, `DEBT_CLEARED`
`ref`	what it was for, e.g. an entry id, `2026-09:12` (month:category), `7:2026-10-23` (workout:day)
`xp`	> 0
`created_at`	its local date is what streaks count

`ux_xp_event (user_id, kind, ref)` makes every reward payable once. Total XP, level, badges and streaks are all derived from this table on read.

Life

`calendar_event`: `occurs_on` DATE, optional `at_time`, `title`.

`idea`: workshop board; `status` `TODO` → `DOING` → `DONE`.

`meal`: `eaten_on`, `name`, `calories` (0–20000).

`workout` + `workout_completion`: a workout's planned weekdays are a bitmask (Mon = 1 … Sun = 64, 0 = any day); a completion is `(workout_id, done_on)` as the primary key, so ticking twice on one day is impossible.

Handy queries

```
-- Which migrations have run
SELECT installed_rank, version, description, installed_on, success
FROM flyway_schema_history ORDER BY installed_rank;

-- One user's account balances
SELECT a.name,
       a.opening_balance
       + COALESCE(SUM(CASE WHEN e.type = 'INCOME'  AND e.account_id = a.id THEN e.amount END), 0)
       - COALESCE(SUM(CASE WHEN e.type IN ('EXPENSE', 'TRANSFER') AND e.account_id = a.id THEN e.amount END),
0)
       + COALESCE(SUM(CASE WHEN e.type = 'TRANSFER' AND e.to_account_id = a.id THEN e.amount END), 0) AS
balance
FROM account a
LEFT JOIN ledger_entry e ON e.account_id = a.id OR e.to_account_id = a.id
WHERE a.user_id = :user
GROUP BY a.id;

-- Open debts with what's still owed
SELECT d.person, d.direction, d.amount - COALESCE(SUM(p.amount), 0) AS outstanding
FROM debt d LEFT JOIN debt_payment p ON p.debt_id = d.id
WHERE d.user_id = :user AND d.settled_on IS NULL
GROUP BY d.id;
```

Backups

See Operations. Take one before every release that adds a migration.

CHAPTER 06

API Reference

Every endpoint with bodies, responses and error codes

The browser API: everything under `/api` except `/api/device/**`, which the desk device uses with its own token (see Desk Device API).

Conventions

- **Auth.** A session cookie set by login, plus a signed remember-me cookie. All `/api` calls need a login except: `POST /api/auth/login`, `POST /api/auth/register`, `POST /api/auth/password-reset/request|confirm`, `GET /api/status`. Without a login the answer is a bare `401`.
- **CSRF.** Every non-GET request must send the `XSRF-TOKEN` cookie's value in an `X-XSRF-TOKEN` header (`frontend/src/api/client.ts` does this).
- **JSON** in and out. Dates are `YYYY-MM-DD` (local calendar dates), months `YYYY-MM`, times `HH:MM[:SS]`.
- **Money** comes back as JSON numbers and should be sent as two-decimal strings (`"12.50"`). Amounts are always positive; the entry `type` carries the direction.
- **Ownership.** Ids belonging to another user behave exactly like ids that don't exist: `404 not_found`.
- **Status codes.** `200` read/update, `201` create, `204` delete, `400` invalid, `401` not logged in, `404` not found, `409` conflict (state has moved on), `429` Gaby's daily limit, `502` AI provider failed, `503` feature not configured.

Errors

```
{ "code": "more_than_owed", "message": "Only RM 70.00 is left to pay." }
```

Validation failures add the failing fields:

```
{ "code": "invalid", "message": "Some fields need another look.",
  "fields": { "amount": "must be greater than or equal to 0.01" } }
```

`code` is stable and meant for code to switch on; `message` is for showing to people.

Auth and account

Method	Path	Body	Returns
POST	`/api/auth/login`	form: `username`, `password`	`200`, or `401`
POST	`/api/auth/logout`		`200`
POST	`/api/auth/register`	`{username, password, email?}`	`Me`, logged in. `409 username_taken` / `email_taken`

Method	Path	Body	Returns
GET	`/api/auth/me`		`Me`
PATCH	`/api/auth/me/preferences`	any of `{skin, theme, calorieGoal, tempUnit, email, weather}`	`Me`
POST	`/api/auth/password-reset/request`	`{email}`	always `200` (never reveals whether the email exists). `503 reset_unavailable` without SMTP
POST	`/api/auth/password-reset/confirm`	`{token, password}`	`200` . `400 reset_invalid`

`username` : 3–40 letters, digits, `_` or `-` , unique ignoring case. `password` : 8–100 characters. `skin` : `classic|modern` . `theme` : `steel|oak|forest|parchment` . `calorieGoal` : 500–10000. `tempUnit` : `C|F` . `email` : blank clears it. `weather` : `{place, lat, lon}` .

```
// Me
{ "id": 1, "username": "krugger", "email": null, "skin": "modern", "theme": "steel",
  "calorieGoal": 2000, "tempUnit": "C",
  "weather": { "place": "Kuala Lumpur, Malaysia", "lat": 3.139, "lon": 101.6869 } }
```

Character sheet

`GET /api/character` : everything the Hall shows, derived on read. Opening it also awards any "budget kept" XP due for finished months.

```
{ "username": "krugger", "level": 3, "title": "Apprentice", "xp": 340,
  "levelXp": 40, "levelXpNeeded": 300,
  "gold": 6097.60, "incomeThisMonth": 6200.00, "spentThisMonth": 102.40,
  "budget": { /* MonthBudget, see Budgets */ },
  "streak": { "current": 4, "longest": 12, "last7": [false, true, true, true, false, true, true] },
  "badges": [ { "id": "first_coin", "name": "First coin", "description": "Log your first entry.",
                "icon": "coin", "earned": true } ],
  "quests": [ /* up to 3 active QuestView */ ],
  "loot": [ /* 8 most recent EntryView */ ] }
```

`gold` is the total balance across accounts. `last7` ends with today.

Accounts

Method	Path	Body	Returns
GET	`/api/accounts`		`AccountView[]` with live balances
POST	`/api/accounts`	`{name, kind, openingBalance}`	`201 AccountView`

Method	Path	Body	Returns
PUT	`/api/accounts/{id}`	same	`AccountView`
DELETE	`/api/accounts/{id}`		`204` ; archived instead if it has entries
POST	`/api/accounts/{id}/restore`		`AccountView`

`kind` : `BANK|CASH|EWALLET|CARD|OTHER` . `AccountView` : `{id, name, kind, openingBalance, balance, archived}` .

Categories

Method	Path	Body	Returns
GET	`/api/categories`		`CategoryView[]`
POST	`/api/categories`	`{name, kind, icon, color, monthlyBudget?}`	`201` . `409 category_exists`
PUT	`/api/categories/{id}`	same	`CategoryView`
DELETE	`/api/categories/{id}`		`204` ; archived (budget cleared) if used
POST	`/api/categories/{id}/restore`		`CategoryView`

`kind` : `EXPENSE|INCOME` . `color` : `#rrggbb` . `monthlyBudget` only applies to expense categories. `icon` is one of: `food transport bills fun shopping health home education gift salary savings travel pet phone coffee coin` (`400 unknown_icon` otherwise).

Entries (the ledger)

Method	Path	Body / query	Returns
GET	`/api/entries`	`?month=YYYY-MM&accountId=&categoryId=` (month defaults to now)	`EntryView[]` , newest first
POST	`/api/entries`	`EntryRequest`	`201 EntryView` , +5 XP (first 10 a day)
PUT	`/api/entries/{id}`	`EntryRequest`	`EntryView`
DELETE	`/api/entries/{id}`		`204`
GET	`/api/ledger/summary`	`?month=YYYY-MM`	`MonthSummary`

```
// EntryRequest
{ "type": "EXPENSE", "amount": "12.50", "accountId": 3, "categoryId": 7,
  "toAccountId": null, "occurredOn": "2026-10-23", "note": "Nasi lemak" }
```

Rules: income/expense need a category of the matching kind (`category_required` , `category_kind_mismatch`); a transfer needs `toAccountId` different from `accountId` and no category (`transfer_needs_two_accounts`); archived accounts can't take new entries (`account_archived`).

`EntryView` : `{id, type, amount, occurredOn, note, account: {id, name}, toAccount: {id, name} | null, category: {id, name, icon, color} | null, recurring}` .

`MonthSummary` : `{month, income, expense, net, spendingByCategory: [{category, total}]}` .

Budgets

`GET /api/budgets?month=YYYY-MM` returns a `MonthBudget` :

```
{ "month": "2026-10", "daysLeft": 8, "totalBudget": 1200.00, "totalSpent": 840.10,
  "left": 359.90, "unbudgetedSpent": 52.00,
  "categories": [ { "category": {"id":7,"name":"Food","icon":"food","color":"#e5606a"},
                    "budget": 600.00, "spent": 512.30, "left": 87.70 } ] }
```

A budget is set through the category's `monthlyBudget` .

Recurring

Method	Path	Body	Returns
GET	`/api/recurring`		`RuleView[]` by day of month
POST	`/api/recurring`	`RuleRequest`	`201 RuleView` ; a past `startOn` catches up at once (auto rules)
PUT	`/api/recurring/{id}`	`RuleRequest`	`RuleView`
DELETE	`/api/recurring/{id}`		`204` ; entries it made stay
POST	`/api/recurring/{id}/confirm`	`{on, amount?}`	`RuleView` ; books the awaiting entry, +5 XP
POST	`/api/recurring/{id}/skip`	`{on}`	`RuleView` ; moves on without booking

```
// RuleRequest
{ "type": "EXPENSE", "amount": "180.00", "accountId": 3, "categoryId": 9,
  "dayOfMonth": 20, "startOn": "2026-10-01", "note": "Electricity (TNB)",
  "active": true, "autoPost": false }

// RuleView
{ "id": 4, "type": "EXPENSE", "amount": 180.00, "account": {"id":3,"name":"Maybank"},
  "category": {"id":9,"name":"Bills","icon":"bills","color":"#e2b453"},
  "note": "Electricity (TNB)", "dayOfMonth": 20, "nextRunOn": "2026-10-20",
  "active": true, "autoPost": false, "awaitingConfirmation": true }
```

- `type` is `INCOME` or `EXPENSE` (`400 transfer_not_recurring`).
- `autoPost` defaults to `true` (logged by itself on the day). With `false` , the rule becomes `awaitingConfirmation` on its day until confirmed or skipped.
- **confirm/skip** must send `on` = the `nextRunOn` the user saw. If the rule isn't awaiting or `on` doesn't match (a double tap, a second tab), the answer is `409 not_awaiting` and nothing is booked twice. `amount` overrides the amount for that one occurrence (bills that vary).
- Switching a rule from confirm to auto books anything that was awaiting.

Debts

Method	Path	Body	Returns
GET	`/api/debts`		`DebtView[]` : open first (soonest due on top), then settled (latest first)
POST	`/api/debts`	`DebtRequest`	`201 DebtView`
PUT	`/api/debts/{id}`	`DebtRequest`	`DebtView` . `400 amount_below_repaid`
DELETE	`/api/debts/{id}`		`204` (repayments go with it)
POST	`/api/debts/{id}/payments`	`{amount, paidOn?, note?}`	`201 DebtView` ; paying the rest settles it, +10 XP once
DELETE	`/api/debts/{id}/payments/{paymentId}`		`DebtView` ; reopens it if something is owed again
POST	`/api/debts/{id}/settle`		`DebtView` ; lets the rest go (no XP)
POST	`/api/debts/{id}/reopen`		`DebtView` . `409 fully_repaid`

```
// DebtRequest
{ "direction": "THEY_OWE", "person": "Ali", "amount": "120.00",
  "note": "Concert ticket", "startedOn": "2026-10-10", "dueOn": "2026-10-27" }

// DebtView
{ "id": 2, "direction": "THEY_OWE", "person": "Ali", "amount": 120.00,
  "repaid": 50.00, "outstanding": 70.00, "note": "Concert ticket",
  "startedOn": "2026-10-10", "dueOn": "2026-10-27", "settledOn": null, "overdue": false,
  "payments": [ { "id": 5, "amount": 50.00, "paidOn": "2026-10-20", "note": "" } ] }
```

- `direction` : `THEY_OWE` (someone owes the user) or `I_OWE` .
- `startedOn` defaults to today; `dueOn` is optional and can't be before it (`due_before_start`).
- A repayment can't exceed what's outstanding (`400 more_than_owed`) or be added to a settled debt (`409 already_settled`).
- `outstanding` is `0` once settled; `amount − repaid` on a settled debt is what was let go.

Quests (savings goals)

Method	Path	Body	Returns
GET	`/api/quests`		`QuestView[]`
POST	`/api/quests`	`{name, icon, targetAmount, deadline?}`	`201 QuestView`
PUT	`/api/quests/{id}`	same	`QuestView`
DELETE	`/api/quests/{id}`		`204`
POST	`/api/quests/{id}/contributions`	`{amount}` (negative takes back)	`QuestView`; reaching the target completes it, +200 XP once

`QuestView`: `{id, name, icon, targetAmount, savedAmount, deadline, completedAt, xpReward}`. A completed quest stays completed. `400 zero_amount` for a zero contribution.

Calendar

Method	Path	Body / query	Returns
GET	`/api/events`	`?from=YYYY-MM-DD&to=YYYY-MM-DD` (inclusive, about a year at most)	`EventView[]`
POST	`/api/events`	`{occursOn, atTime?, title}`	`201 EventView`
PUT	`/api/events/{id}`	same	`EventView`
DELETE	`/api/events/{id}`		`204`

`EventView`: `{id, occursOn, atTime, title}`; `atTime` null means all day.

Vitals

Method	Path	Body / query	Returns
GET	`/api/meals`	`?date=` (default today)	`MealDay`
POST	`/api/meals`	`{name, calories, eatenOn}`	`201`; +5 XP once per day with a meal
DELETE	`/api/meals/{id}`		`204`
GET	`/api/workouts`	`?date=`	`WorkoutView[]`
POST	`/api/workouts`	`{name, days}`	`201 WorkoutView`
PUT	`/api/workouts/{id}`	same	`WorkoutView`
DELETE	`/api/workouts/{id}`		`204`
POST	`/api/workouts/{id}/done`	`{date, done}`	`WorkoutView`; +10 XP once per workout per day

`MealDay` : `{date, goal, total, meals: [{id, name, calories}], week: [{date, calories}]}` . `days` : weekday names (`MONDAY` … `SUNDAY`); empty means any day. `WorkoutView` : `{id, name, days, scheduled, done, streak, totalDone}` ; `streak` counts planned days in a row that were done.

Workshop

Method	Path	Body	Returns
GET	`/api/ideas`		`IdeaView[]` : doing, to do, done
POST	`/api/ideas`	`{title, notes?, status?}`	`201 IdeaView`
PUT	`/api/ideas/{id}`	same	`IdeaView` ; reaching `DONE` gives +15 XP once
DELETE	`/api/ideas/{id}`		`204`

Today

`GET /api/life/today` : `{date, calories, calorieGoal, workouts: [{id, name, done}], upcoming: EventView[], ideasInProgress, ideasWaiting}` ; the Hall's Today panel.

Gaby

Method	Path	Body	Returns
GET	`/api/steward/status`		`{ready, provider, remaining, limit}`
POST	`/api/steward/chat`	`{message, history: [{role, text}]}`	`{reply, subtitle, remaining}`

`message` : at most 1000 characters (`400 too_long` , `400 empty`). The server keeps no chat state: the browser sends the recent `history` (`role` is `user` or `assistant`). `503 not_configured` without an API key, `429 user_limit` / `global_limit` when the day's allowance is used, `502 steward_failed` if the provider fails. See Gaby, the Steward.

Desk devices (managed from the browser)

Method	Path	Returns
GET	`/api/devices`	`[{id, name, createdAt, lastSeenAt}]`
POST	`/api/devices/pairing-code`	`{code, expiresAt}` : six digits, 10 minutes
DELETE	`/api/devices/{id}`	`204` ; the device's token stops working at once

Import and status

Method	Path	Body	Returns
POST	`/api/import/tavern`	Krugger's Tavern `/api/kv` export	`{entries, recurring, categoriesCreated, events, meals, workouts, ideas, skipped}`. `409 already_imported`, `400 nothing_to_import`
GET	`/api/status`		`{app, status, schemaVersion}` (public)
GET	`/actuator/health`		`{"status":"UP"}`; used by `redeploy.sh`

CHAPTER 07

Gaby, the Steward

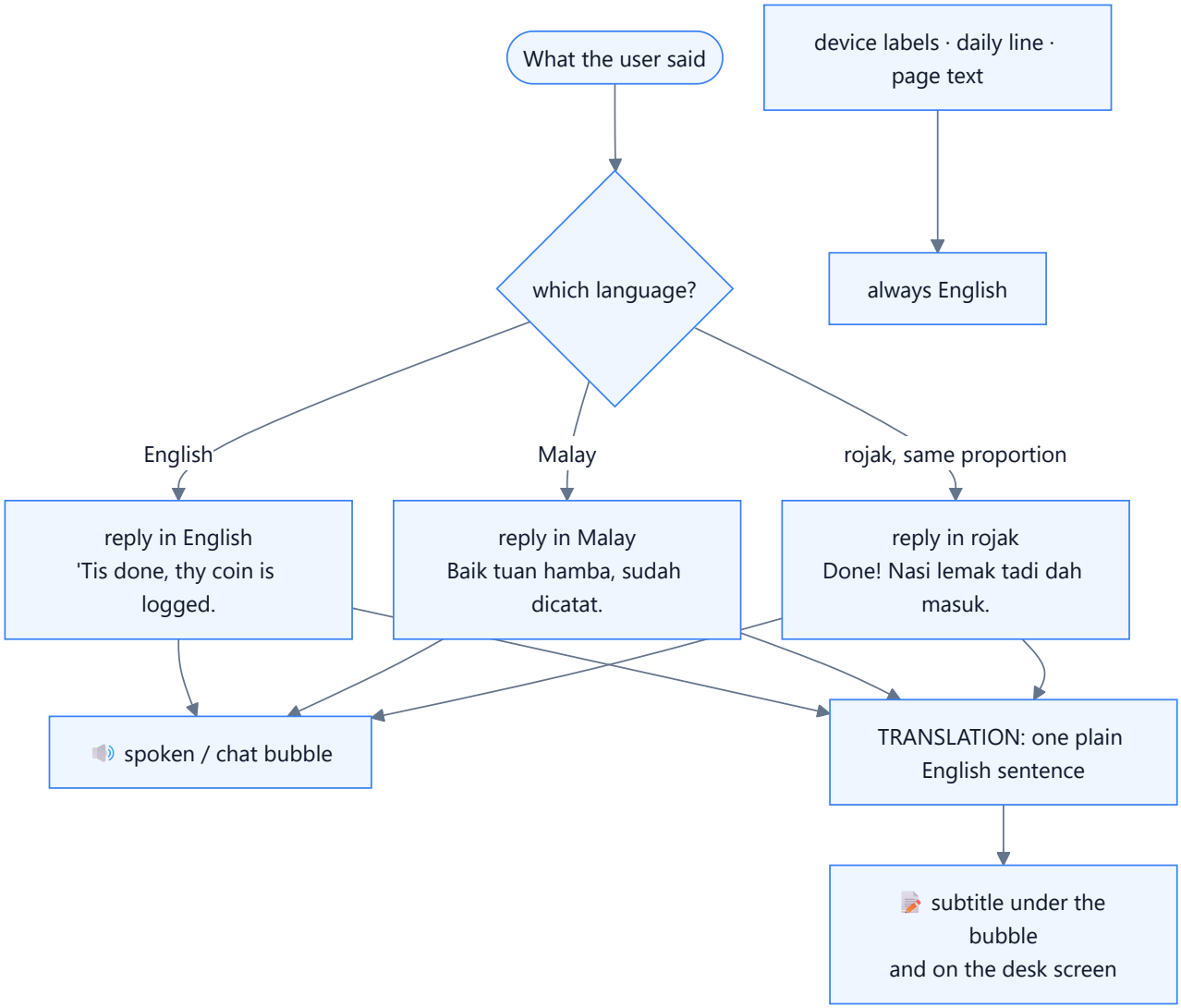
The AI helper: tool loop, language rule, limits, voice

Gaby is KrugWorld's AI helper: a cheerful young steward in a medieval keep who keeps the realm's books. She can read and change everything in the app through tools, in the web chat, in the dock beside any page, and by voice on the desk device.

Character

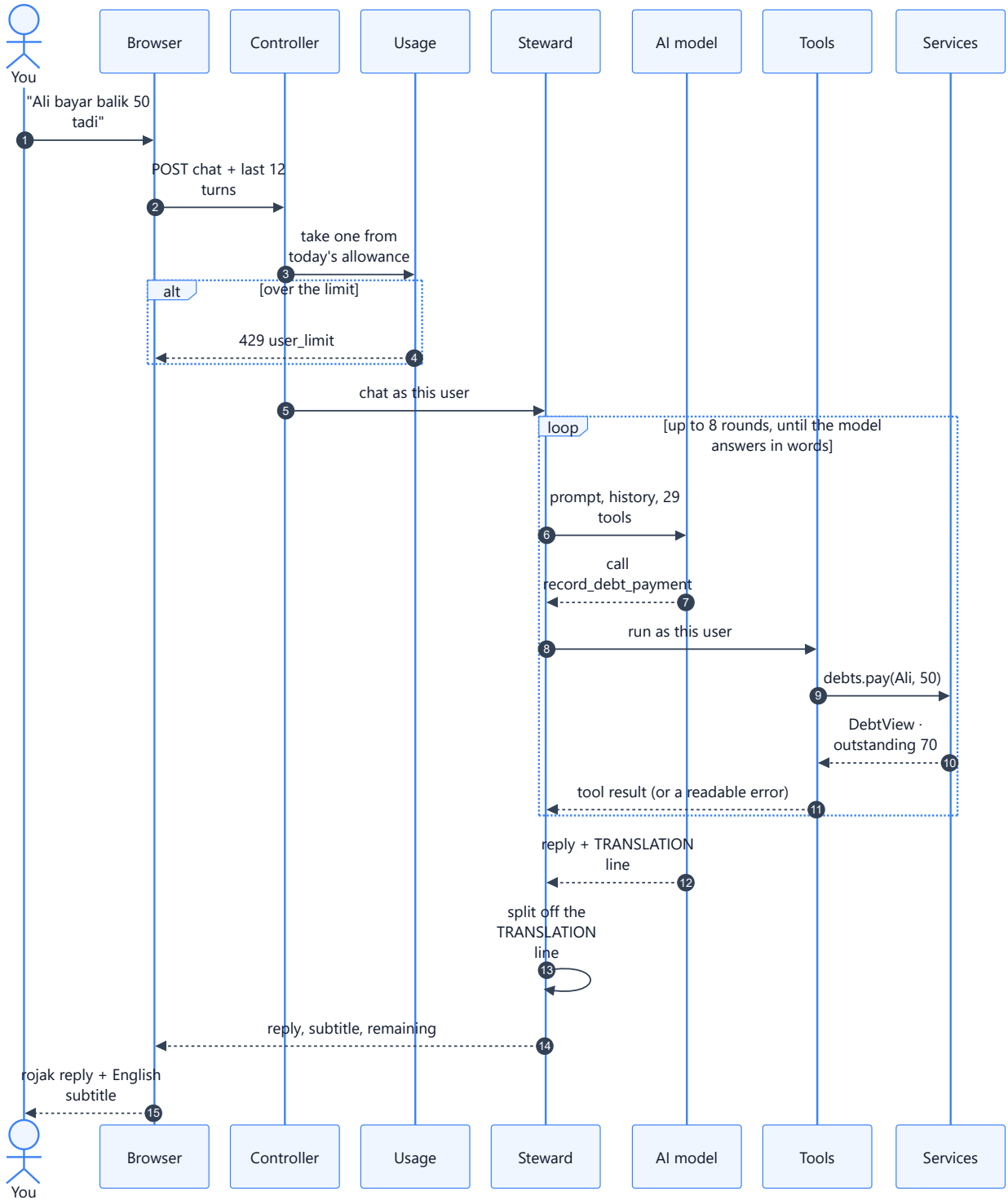
- **Look:** a female pixel-art portrait (`frontend/src/steward/Portrait.tsx`): burgundy cap with a gold feather, gold spectacles, teal coat, auburn braids. She blinks, rolls her eyes up while thinking and moves her mouth while talking (`useMood`).
- **Voice in text:** warm, a little playful, a light medieval flavour (the odd *thee* or *'tis*; in Malay a touch of classical courtesy like *hamba*, *tuan hamba*). Replies are a sentence or two, short enough to be read aloud on a small speaker.
- **Language:** the user mixes English and Malay. Gaby answers in **the same mix and proportion** the user used: English to English, rojak to rojak. Malay food and place names are never translated.
- **Screen text is English.** Only her spoken/chat reply is rojak. Each final reply ends with a `TRANSLATION:` line, one plain English sentence, which the server strips off and returns as `subtitle`. Labels on the device and her daily line are English too.

FIGURE 34 · CHARACTER



How a message is answered

FIGURE 35 · HOW A MESSAGE IS ANSWERED



The reply in that example: "*Baik tuan hamba, Ali dah bayar RM50, tinggal RM70 lagi.*" with the subtitle "*Ali paid back RM 50; RM 70 is still owed.*"

- **Stateless.** The server keeps no conversation. The browser sends the last few turns (`history-turns` , 12) with each message, so a redeploy or a second tab never loses or mixes anything.

- **Tools are thin wrappers over the real services.** Gaby's actions go through the same ownership checks, validation and XP as the screens. She can't do anything the user couldn't, and her actions count towards streaks.
- **A backstop of 8 tool rounds** stops a runaway loop; a normal answer takes one to three.
- **Tool failures are readable.** An `ApiException` (e.g. "Only RM 70.00 is left to pay") goes back to the model as an error result, and she explains it in character.
- The system prompt includes today's date and weekday, so "esok" and "last Tuesday" resolve correctly.

Tools

Area	Tools
Overview	`get_overview` (the character sheet)
Money	`list_accounts`, `list_categories`, `list_entries`, `add_entry`, `delete_entry`
Budgets	`get_budgets`, `set_budget`
Recurring	`list_recurring`, `add_recurring` (with `auto` true/false), `confirm_recurring` (confirm, change the amount, or `skip`)
Debts	`list_debts`, `add_debt`, `record_debt_payment` (by person's name; the oldest open debt if several)
Quests	`list_quests`, `add_quest`, `contribute_to_quest`
Calendar	`list_events`, `add_event`, `delete_event`
Meals	`get_meals`, `add_meal` (estimates calories if not given), `set_calorie_goal`
Training	`list_workouts`, `add_workout`, `mark_workout_done`
Workshop	`list_ideas`, `add_idea`, `set_idea_status`

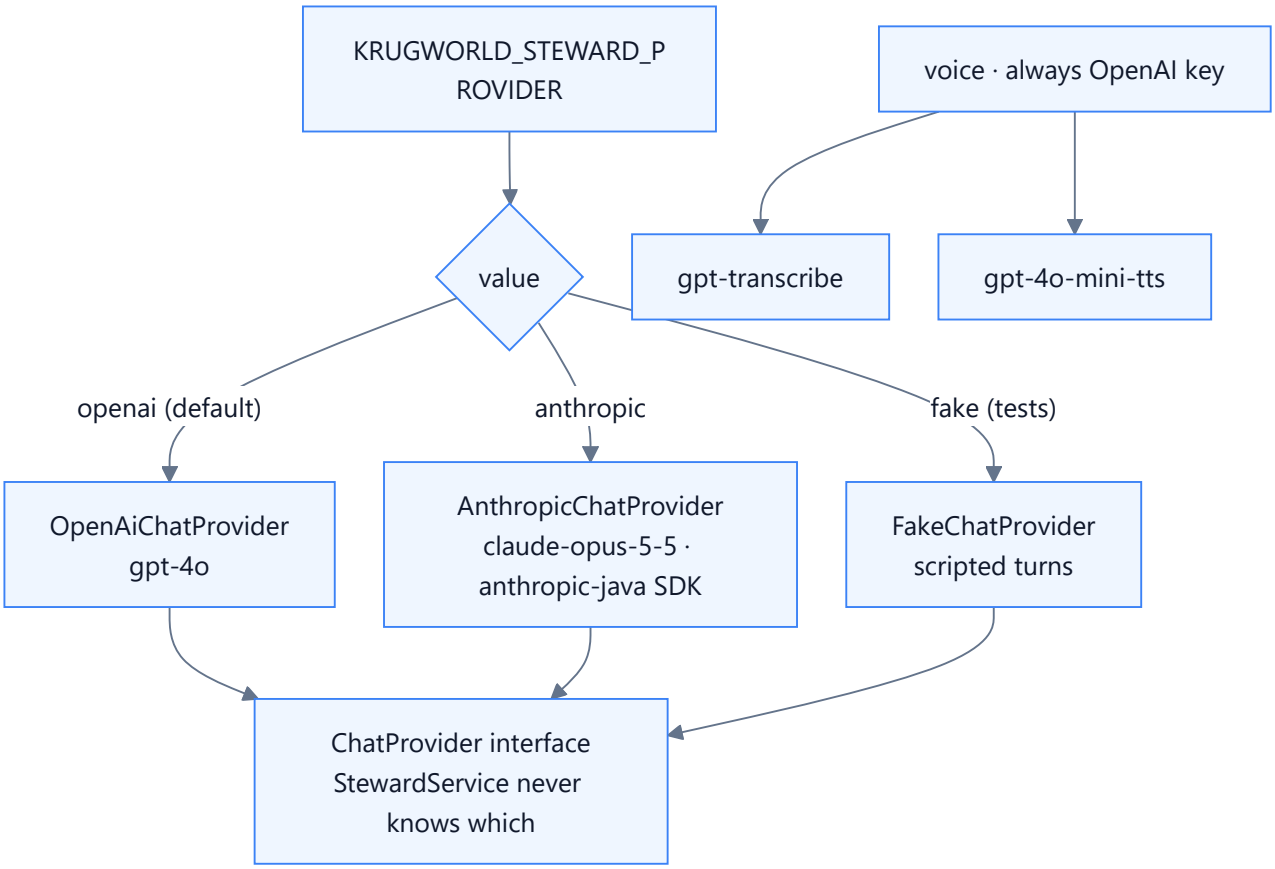
Things people name (accounts, categories, quests, people) are looked up by name, ignoring case; when a name doesn't match, the error lists the valid choices so the model can recover. Things people don't name (entries, events, ideas) use ids from the `list_*` tools.

To add a tool: add a `StewardTool` in `StewardTools.define()` with a JSON-schema (`Schema.object/props/string/...`) and a handler calling an existing service, then extend `StewardIntegrationTest` with a scripted `FakeChatProvider` turn.

Providers and configuration

OpenAI is the default for everything (brain, speech-to-text, text-to-speech), so one key covers it all. Claude is available by switching one variable.

FIGURE 36 · PROVIDERS AND CONFIGURATION



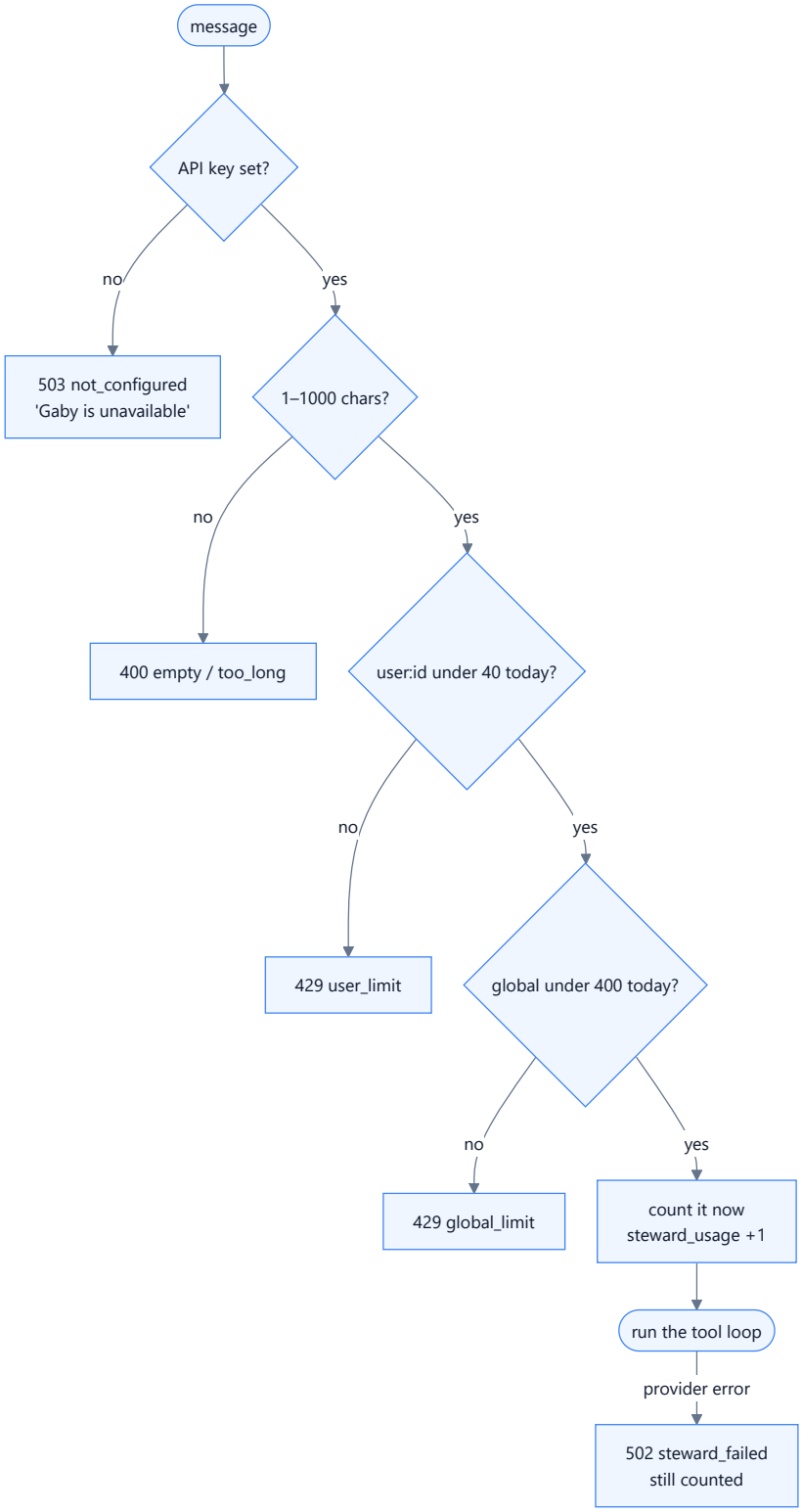
Variable	Default	Meaning
`KRUGWORLD_STEWARD_PROVIDER`	`openai`	`openai` or `anthropic`
`OPENAI_API_KEY`		brain, ears and voice
`KRUGWORLD_STEWARD_OPENAI_MODEL`	`gpt-4o`	chat model
`ANTHROPIC_API_KEY`		when the provider is `anthropic`
`KRUGWORLD_STEWARD_MODEL`	`claude-opus-5-5`	Claude model (official `anthropic-java` SDK)
`KRUGWORLD_STEWARD_EFFORT`	`low`	Claude effort
`KRUGWORLD_STEWARD_DAILY_LIMIT`	`40`	messages per account per day
`KRUGWORLD_STEWARD_GLOBAL_DAILY_LIMIT`	`400`	messages for the whole app per day
`KRUGWORLD_TRANSCRIBE_MODEL`	`gpt-transcribe`	speech-to-text
`KRUGWORLD_TRANSCRIBE_LANGUAGE`	blank (auto-detect)	`ms` forces Malay; auto works well for rojak
`KRUGWORLD_SPEECH_MODEL`	`gpt-4o-mini-tts`	text-to-speech
`KRUGWORLD_SPEECH_VOICE`	`coral`	her voice (chosen by the owner)

With no key, the Steward page says Gaby is unavailable (`503 not_configured`) instead of failing.

Spend guards

A server-held key plus open registration needs a cap. `StewardUsage` keeps per-account and global counters per day in `steward_usage` (so a mid-day redeploy doesn't reset them). A message counts when accepted, not when it succeeds, so a failing provider can't become a free retry loop. Over the limit: `429 user_limit` or `429 global_limit`. Messages are capped at 1000 characters.

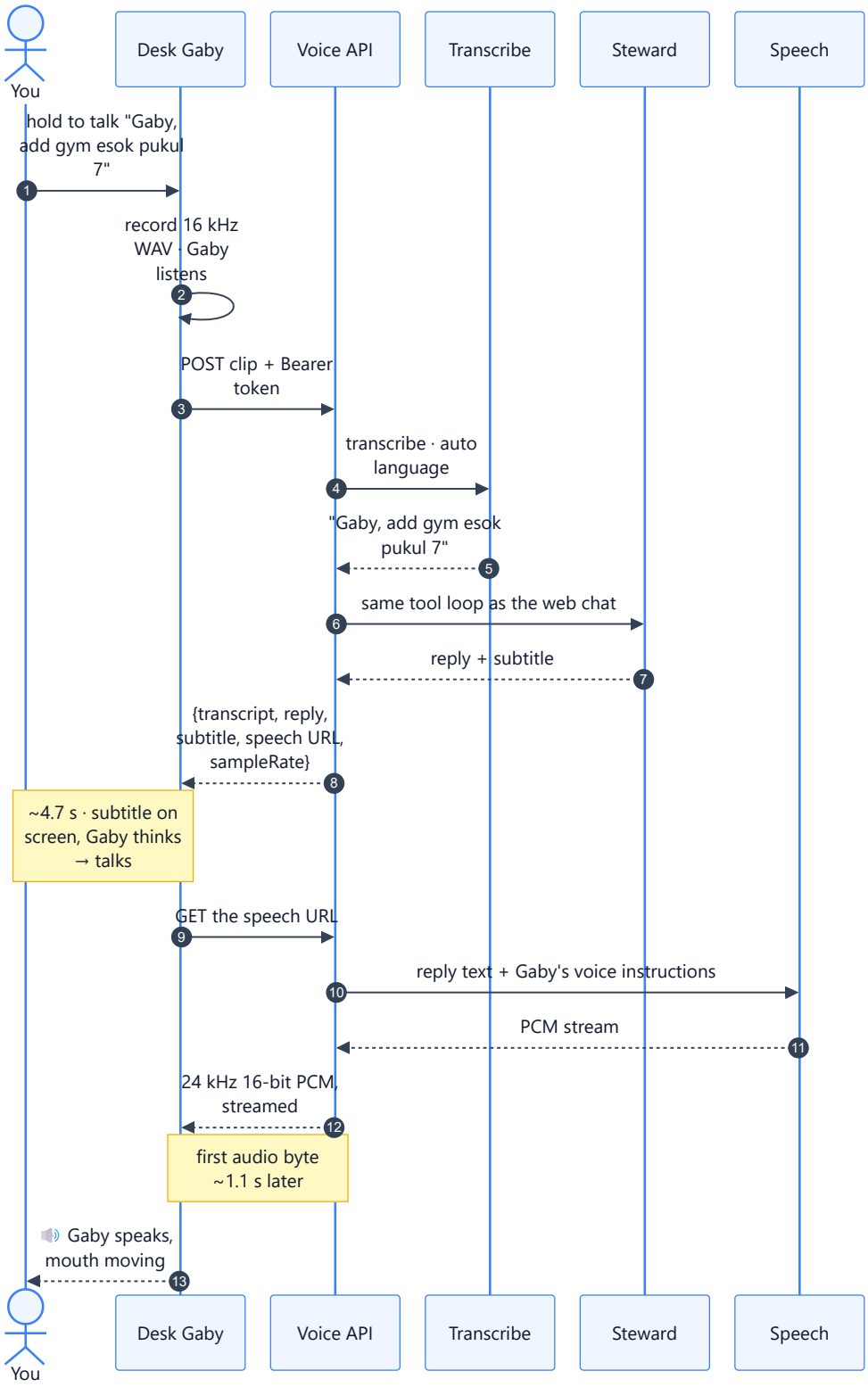
FIGURE 37 · SPEND GUARDS



Voice (desk device)

Tap-to-talk on the ESP32: the device records a WAV clip and posts it to `/api/device/voice`. The server transcribes it (`gpt-transcribe`), runs the same `StewardService` (there is no second assistant), returns the text reply and subtitle, and streams her speech as 24 kHz 16-bit PCM from `gpt-4o-mini-tts` with voice instructions for Gaby's accent. Measured on 2026-10-12: about 4.7 s to the text reply and 1.1 s more to the first audio byte. Details: Desk Device API.

FIGURE 38 · VOICE (DESK DEVICE)



On the device's home screen Gaby also has a **daily line** (`GabyLine`): picked by plain rules from the user's day (most urgent first, a greeting otherwise), with no AI call, so refreshing it every minute costs nothing.

Testing

Tests never call a real provider. `IntegrationTest` boots the app with `krugworld.steward.provider=fake`; `FakeChatProvider` replays scripted turns (including tool calls), and `FakeSpeechProvider` stands in for transcription and speech. No network, no cost, deterministic.

CHAPTER 08

Desk Device API

The ESP32 desk Gaby and its token-based API

What the ESP32 firmware talks to. Everything is under `/api/device/`, and except for pairing every call needs `Authorization: Bearer <token>`. No cookies and no CSRF token are involved. Errors are the usual `{code, message}` JSON.

1. Pairing (once)

The user clicks **Settings** → **Desk Gaby** → **Pair a device** and gets a six-digit code, valid for 10 minutes. The device sends it:

```
POST /api/device/pair
Content-Type: application/json

{"code": "482913", "name": "Desk Gaby"}
```

```
{"deviceId": 3, "token": "kgd_…", "username": "Krugger"}
```

Store the token in NVS (flash). A wrong or expired code returns `401 bad_code`. Each wrong guess also uses up one of every live code's 5 attempts, so the digits can't be brute-forced. A 401 on any later call means the device was removed in Settings, so go back to the pairing screen.

2. Glance screen

```
GET /api/device/summary
```

```
{
  "date": "2026-10-12", "username": "Krugger",
  "level": 4, "title": "Squire", "xp": 610, "levelXp": 110, "levelXpNeeded": 300, "streak": 6,
  "gold": 2450.30, "spentThisMonth": 812.40,
  "budget": {"left": 388.00, "total": 1200.00, "daysLeft": 19, "tightest": "Food", "tightestLeft": 42.50},
  "calories": 1350, "calorieGoal": 2000,
  "workouts": [{"id": 7, "name": "Push-ups", "done": false}],
  "events": [{"date": "2026-10-13", "time": "14:30", "title": "Dentist"}],
  "voiceReady": true, "voiceRemaining": 38
}
```

`budget` is `null` when no budgets are set, and `time` is `null` for all-day events. Polling once a minute is plenty.

Tick a workout (`false` unticks it):

```
POST /api/device/workouts/7/done
{"done": true}
```

3. Tap to talk

Record while the button is held, then send the WAV clip as the raw body (not multipart). 16 kHz, 16-bit mono is ideal. The clip must be between about 0.2 s and 1 MB (about 30 s).

```
POST /api/device/voice
Content-Type: audio/wav

<wav bytes>
```

```
{
  "transcript": "spent 12 on nasi lemak tadi",
  "reply": "'Tis written: twelve ringgit for nasi lemak.",
  "subtitle": "Logged RM 12.00 under Food.",
  "speech": "/api/device/voice/speech/5b0c…",
  "sampleRate": 24000,
  "remaining": 37
}
```

Show `subtitle` on screen (fall back to `reply` when it's null), then fetch `speech` and play it as it streams in:

```
GET /api/device/voice/speech/5b0c…
→ application/octet-stream, X-Audio-Format: pcm_s16le;rate=24000;channels=1
```

The audio is raw PCM, ready for I2S to the ES8311. Each speech link works **once**, within 2 minutes.

Gaby remembers the last few turns per device for 5 quiet minutes, so a follow-up like "pukul 7" works after she asks "what hour?". To start over:

```
DELETE /api/device/voice/conversation
```

Each voice message counts against the same daily allowance as typed chat. Errors: `400 clip_too_short`, `413 clip_too_long`, `422 heard_nothing`, `429 user_limit`, `502 steward_failed`, `503 not_configured`.

Try it without the device

```
CODE=482913   # from Settings
TOKEN=$(curl -s localhost:8085/api/device/pair -H 'Content-Type: application/json' \
  -d "{\"code\":\"$CODE\"}" | jq -r .token)
curl -s localhost:8085/api/device/summary -H "Authorization: Bearer $TOKEN" | jq
curl -s localhost:8085/api/device/voice -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: audio/wav' --data-binary @question.wav | tee reply.json | jq
curl -s "localhost:8085$(jq -r .speech reply.json)" -H "Authorization: Bearer $TOKEN" -o reply.pcm
ffplay -f s16le -ar 24000 -ac 1 reply.pcm
```

CHAPTER 09

Development

Running locally, tests, and adding a feature

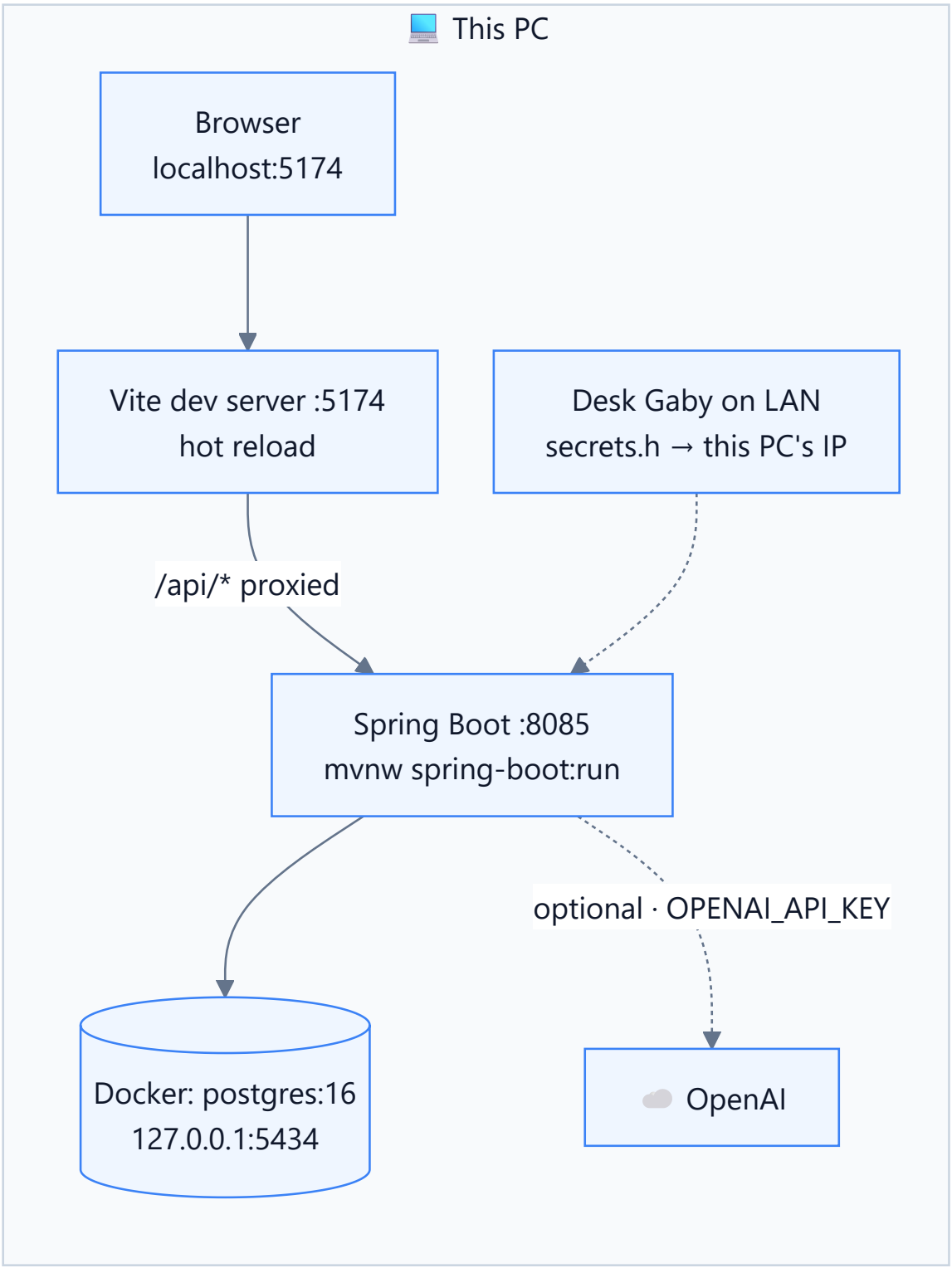
First-time setup on a machine (JDK 17, Node 20+, Docker Desktop): see New PC Setup. This page is the day-to-day.

Run it locally

```
docker compose up -d db                # Postgres on 127.0.0.1:5434
cd backend && ./mvnw spring-boot:run   # API on :8085 (Windows: .\mvnw.cmd spring-boot:run)
cd frontend && npm run dev             # app on :5174, proxies /api to :8085
```

Open http://localhost:5174 and register an account (local only).

FIGURE 39 · RUN IT LOCALLY



Port	What	Set in
5434	local Postgres	`docker-compose.yml`, `application.properties`
8085	backend (`PORT` overrides)	`application.properties`
5174	Vite dev server and its `/api` proxy target	`frontend/vite.config.ts`

If 5174 is taken by another project, run Vite on another port; the proxy still points at 8085: `npm run dev -- --port 5175 --strictPort`.

Optional environment for Gaby locally: `OPENAI_API_KEY` (read it from the key file outside the repo; never commit or print it).

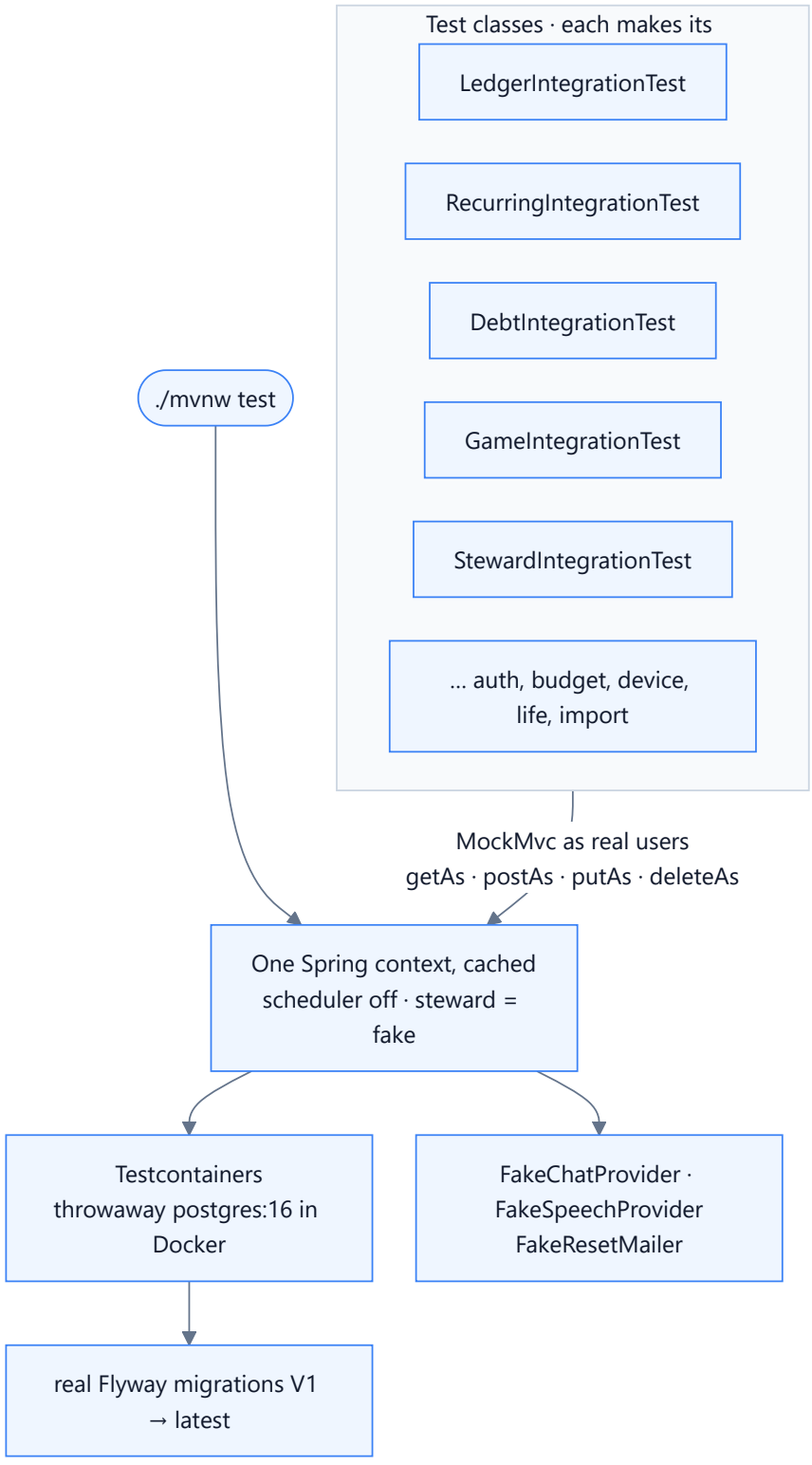
Tests

```
cd backend && ./mvnw test        # needs Docker running
cd frontend && npm run lint      # must stay at zero warnings
cd frontend && npx tsc -b        # type check
```

Backend tests are **integration tests against real Postgres**: `IntegrationTest` starts one Testcontainers `postgres:16` shared by every test class, runs the real Flyway migrations, and drives the API through MockMvc as real users (`newUser()`, `getAs`, `postAs`, …). Tests never assume an empty database; each makes its own uniquely named users. The scheduler is off and Gaby runs on scripted fakes, so tests are offline and free.

Testcontainers is pinned to **1.21.4**: older versions fail against Docker 29.

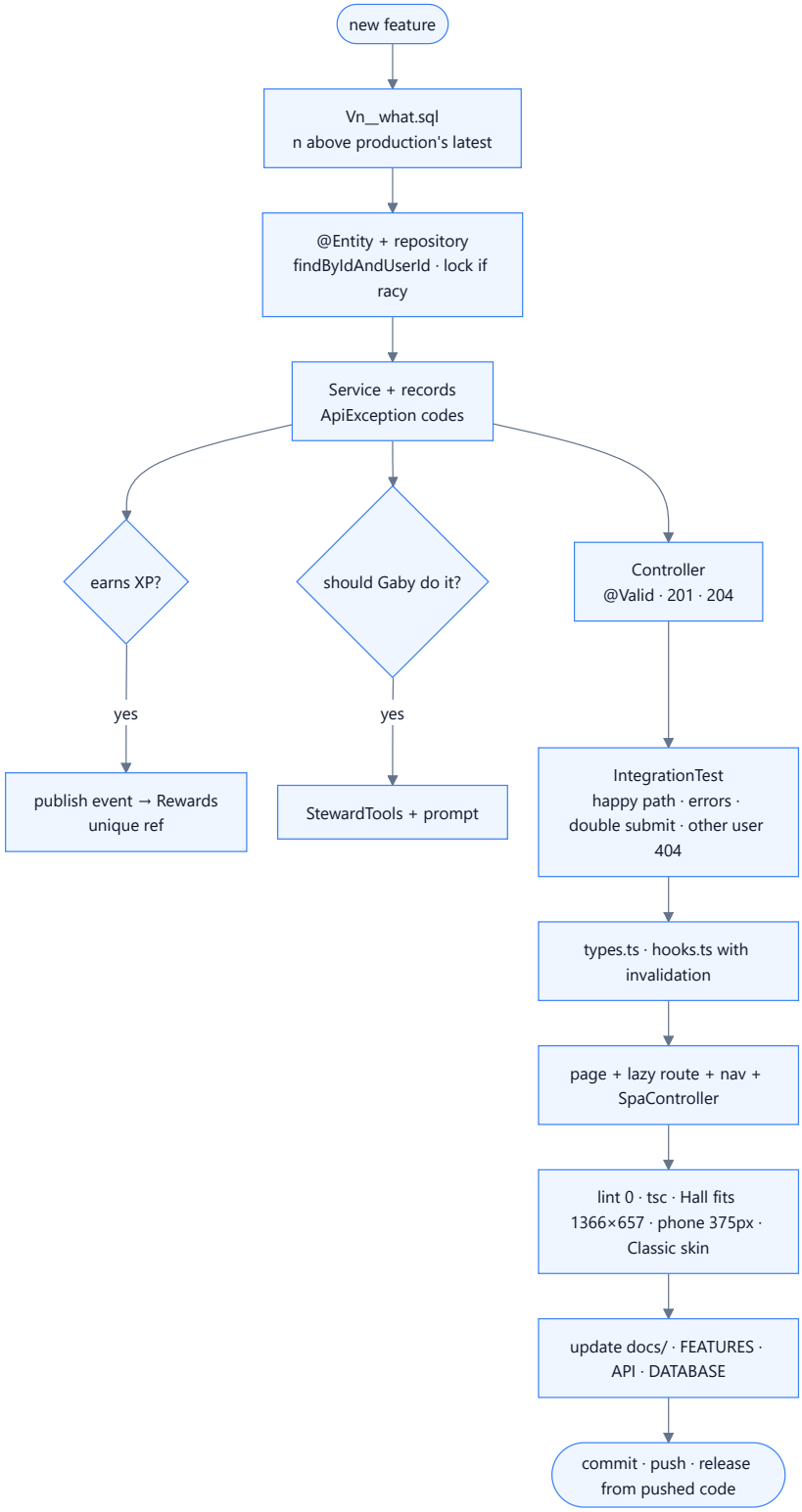
FIGURE 40 · TESTS



CI (`.github/workflows/ci.yml`) runs lint, the frontend build and all backend tests on every push to `main` and on pull requests.

Adding a feature: checklist

FIGURE 41 · ADDING A FEATURE: CHECKLIST



Backend

1. **Migration**: `V<n>__what.sql` with the next number **from production** (see Database). Constraints and indexes in SQL; a comment explaining why.

2. **Entity** (`@Entity` , matching columns exactly; Hibernate validates) and a repository with `findByIdAndUserId` . Lock the row (`@Lock(PESSIMISTIC_WRITE)`) where two quick requests could race (confirming, paying).
3. **Service** with request/view records, `@Transactional` , user id first on every method, `ApiException` for rule failures (stable `code` , human `message`).
4. **Controller**: thin, `@Valid` bodies, `201` for creates, `204` for deletes.
5. **Rewards**: publish an event and award in `game/Rewards` via `XpLedger.award` with a unique `ref` ; never write XP from the feature itself.
6. **Gaby**: add tools in `StewardTools` if she should be able to do it, and mention the feature in the system prompt's list.
7. **Tests**: an `…IntegrationTest` covering the happy path, each rule's error code, idempotency (double submit) and another user's id returning 404.

Frontend

1. Types in `api/types.ts` , hooks in `api/hooks.ts` (or `lifeHooks.ts`) that invalidate every query the write affects.
2. A page in `pages/` , loaded with `page(() => import(...))` in `App.tsx` .
3. A nav entry in `components/nav.ts` and the route in `web/SpaController` (or a refresh on that URL is a 404).
4. Styles only through the skin variables (`var(--accent)` , `var(--hp)` , …), and check the *Classic Krugger* skin.
5. Check the Hall still fits 1366×657 if you touched it, plus a phone width (375 px).

Code conventions

- Money: `BigDecimal` / `NUMERIC(14,2)` ; the frontend sends `"12.50"` strings via `money()` and displays with `rm()` .
- Dates are local calendar dates (`LocalDate` , `"2026-10-23"`), built from local parts in the browser (`lib/format.ts`), never `toISOString()` .
- Read time through the injected `Clock` , never `LocalDate.now()` without it.
- Never annotate a `String` column with `@Lob` (it breaks Postgres reads outside a transaction).
- Comments explain *why*, not what.
- Optimistic UI only where a wrong guess is harmless and easy to roll back (`onMutate` + restore in `onError`).
- For a React Query callback that must run even if the component unmounts after the refetch (e.g. a toast after the last item disappears), await `mutateAsync` instead of passing `onSuccess` to `mutate` .

Useful local tricks

- `GET /api/status` shows the applied schema version.
- Reset the local database completely: `docker compose down -v` then `up -d db` (deletes all local data).
- If you renumber a migration that already ran **locally**, Flyway will refuse to start on your local database. Either reset it as above, or delete those rows from `flyway_schema_history` and drop what they created. Never do this on production.

CHAPTER 10

New PC Setup

Getting a fresh machine ready

Everything you need to go from the zip to a running app.

1. Install the tools

Tool	Version	Why
Java JDK	17 (e.g. Eclipse Temurin 17)	The backend is Spring Boot 3 on Java 17
Node.js	20 or newer	Builds and runs the React frontend
Docker Desktop	any recent	Local PostgreSQL, and the backend tests start a throwaway Postgres in Docker
Git	any	The zip includes the full history

Maven is **not** needed: the repo has the Maven wrapper (`backend/mvnw` , `mvnw.cmd`).

After installing the JDK, make sure `JAVA_HOME` points at it (`java -version` should say 17).

2. Unzip and check

Unzip anywhere, e.g. `Documents\Projects\krugworld` , then:

```
cd krugworld
git log --oneline | head
```

You should see the commit history.

3. Start the local database

```
docker compose up -d db
```

Postgres runs on `127.0.0.1:5434` (user/password/database all `krugworld` , local only). It starts empty; Flyway creates every table on the backend's first start.

4. Run the backend (API on port 8085)

```
cd backend
./mvnw spring-boot:run          # Windows PowerShell: .\mvnw.cmd spring-boot:run
```

Wait for "Started KrugWorldApplication". Check: http://localhost:8085/api/status

5. Run the frontend (app on port 5174)

```
cd frontend
npm install
npm run dev
```

Open http://localhost:5174 and **create an account** (the database is new, so the old demo account doesn't exist here).

6. Optional: let Gaby (the AI Steward) talk

Set an OpenAI key before starting the backend:

```
$env:OPENAI_API_KEY = "sk-..."
.\mvnw.cmd spring-boot:run
```

Without a key, Gaby's page simply says she's resting.

7. Run the tests

```
cd backend && ./mvnw test        # 62 tests; needs Docker running
cd frontend && npm run build && npm run lint
```

8. Build the deployable jar

```
.\build.ps1                      # builds frontend + backend into deploy\krugworld.jar
```

`build.ps1` falls back to a JDK inside `..\KruggerTavern\.buildtools` when `JAVA_HOME` isn't set. On a new PC, just set `JAVA_HOME` (step 1) and it uses that.

Ports used on the old PC

These were picked to avoid other projects on that machine. Change them if needed:

What	Port	Where it's set
Postgres	5434	`docker-compose.yml` , `application.properties`
API	8085	`application.properties` (`PORT` env var overrides)
Frontend dev server	5174	`frontend/vite.config.ts` (also the `/api` proxy target)

Continuing with Claude Code

Open the `krugworld` folder in Claude Code. `CLAUDE.md` at the repo root holds the project briefing (decisions, status, next steps), so a new session starts with context.

CHAPTER 11

Operations

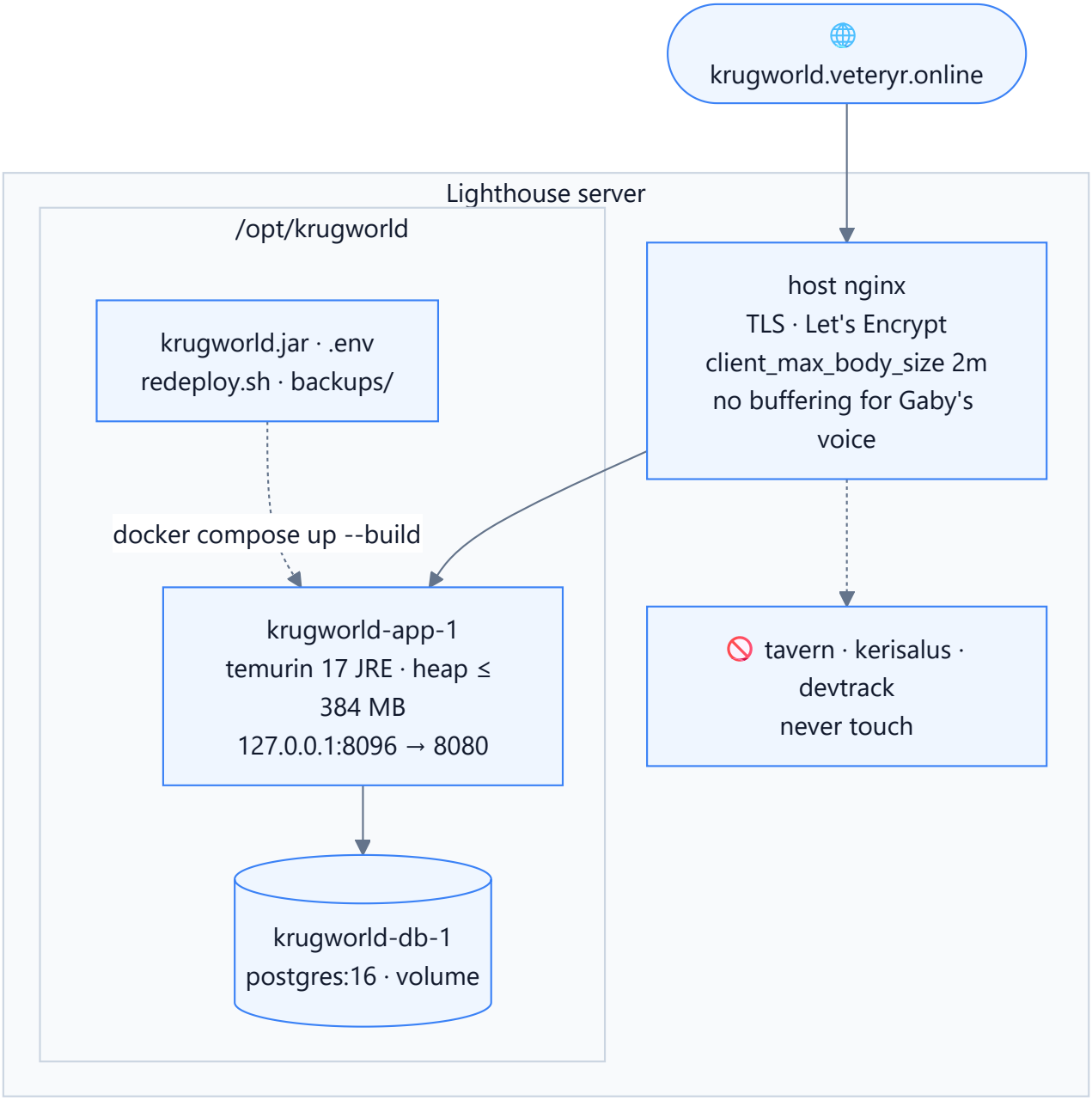
Releases, troubleshooting, rollback, backups, lessons

Running the live site: releasing, checking, backing up and recovering. The first-time server setup (DNS, nginx, certbot, `.env`) is in First-time Deployment.

Where it runs

Address	https://krugworld.veteryr.online (Let's Encrypt, auto-renew)
Server	Tencent Lighthouse, `ubuntu@43.156.150.155`, **shared** with the Tavern, kerisalus and devtrack: never touch their containers
Folder	`/opt/krugworld` (`docker-compose.yml`, `Dockerfile`, `.env`, `krugworld.jar`, `redeploy.sh`, `backups/`)
Containers	`krugworld-app-1` (the jar on `eclipse-temurin:17-jre`, heap capped at 384 MB) and `krugworld-db-1` (`postgres:16`)
Port	app on `127.0.0.1:8096`, behind the host nginx
Secrets	`/opt/krugworld/.env`: `POSTGRES_PASSWORD`, `OPENAI_API_KEY`, optional SMTP settings
SSH	`ssh -i ~/.ssh/krugworld_lighthouse ubuntu@43.156.150.155`. Each PC has its own key; add a new PC's public key to `~/.ssh/authorized_keys` through the Lighthouse console's OrcaTerm

FIGURE 42 · WHERE IT RUNS



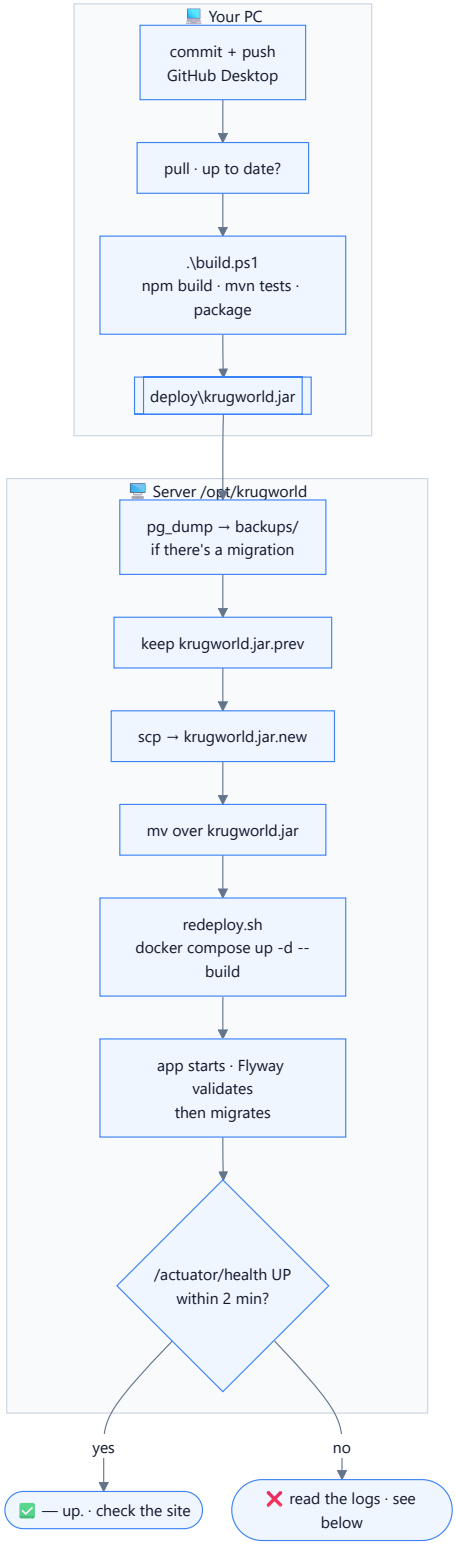
How a release works

The server never builds anything (it's small and shared). A release is:

1. **build the jar** on a PC (frontend build + backend tests + package),
2. **copy** it to `/opt/krugworld/krugworld.jar`,
3. **run** `redeploy.sh` on the server: it rebuilds the app image around the jar, restarts the container, and waits until `/actuator/health` says `UP` (up to 2 minutes).

Flyway applies new migrations when the app starts, so there's no manual database step. `redeploy.sh` doesn't pull from GitHub; it deploys whatever jar is in the folder.

FIGURE 43 · HOW A RELEASE WORKS



Release checklist

1. **Release only from code that is on GitHub.** Commit and push first; on the PC you release from, pull so it has everything. A jar built from an out-of-date checkout silently removes features that were live (see *Lessons* below).
2. If the release adds a migration, check its number is above production's:

```
ssh -i ~/.ssh/krugworld_lighthouse ubuntu@43.156.150.155 \
  "cd /opt/krugworld && docker compose exec -T db psql -U krugworld -d krugworld -Atc 'SELECT
max(version::int) FROM flyway_schema_history'"
```

3. Build: `.\build.ps1` (or in Git Bash: `cd frontend && npm run build`, then `cd backend && ./mvnw -q package`, then copy `backend/target/krugworld-0.1.0.jar` to `deploy/krugworld.jar`).
4. Back up the database (below) if the release has a migration.
5. Keep the current jar as the rollback, upload the new one beside it, then swap:

```
ssh -i $HOME\.ssh\krugworld_lighthouse ubuntu@43.156.150.155 "cp /opt/krugworld/krugworld.jar
/opt/krugworld/krugworld.jar.prev"
scp -i $HOME\.ssh\krugworld_lighthouse deploy\krugworld.jar
ubuntu@43.156.150.155:/opt/krugworld/krugworld.jar.new
ssh -i $HOME\.ssh\krugworld_lighthouse ubuntu@43.156.150.155 "mv /opt/krugworld/krugworld.jar.new
/opt/krugworld/krugworld.jar && /opt/krugworld/redeploy.sh"
```

   The `.new` name means a broken upload is never the jar that starts.
6. Expect `Waiting for KrugWorld..... — up.` Then open the site (Ctrl+F5 past the PWA cache) and check the Hall and anything the release changed.

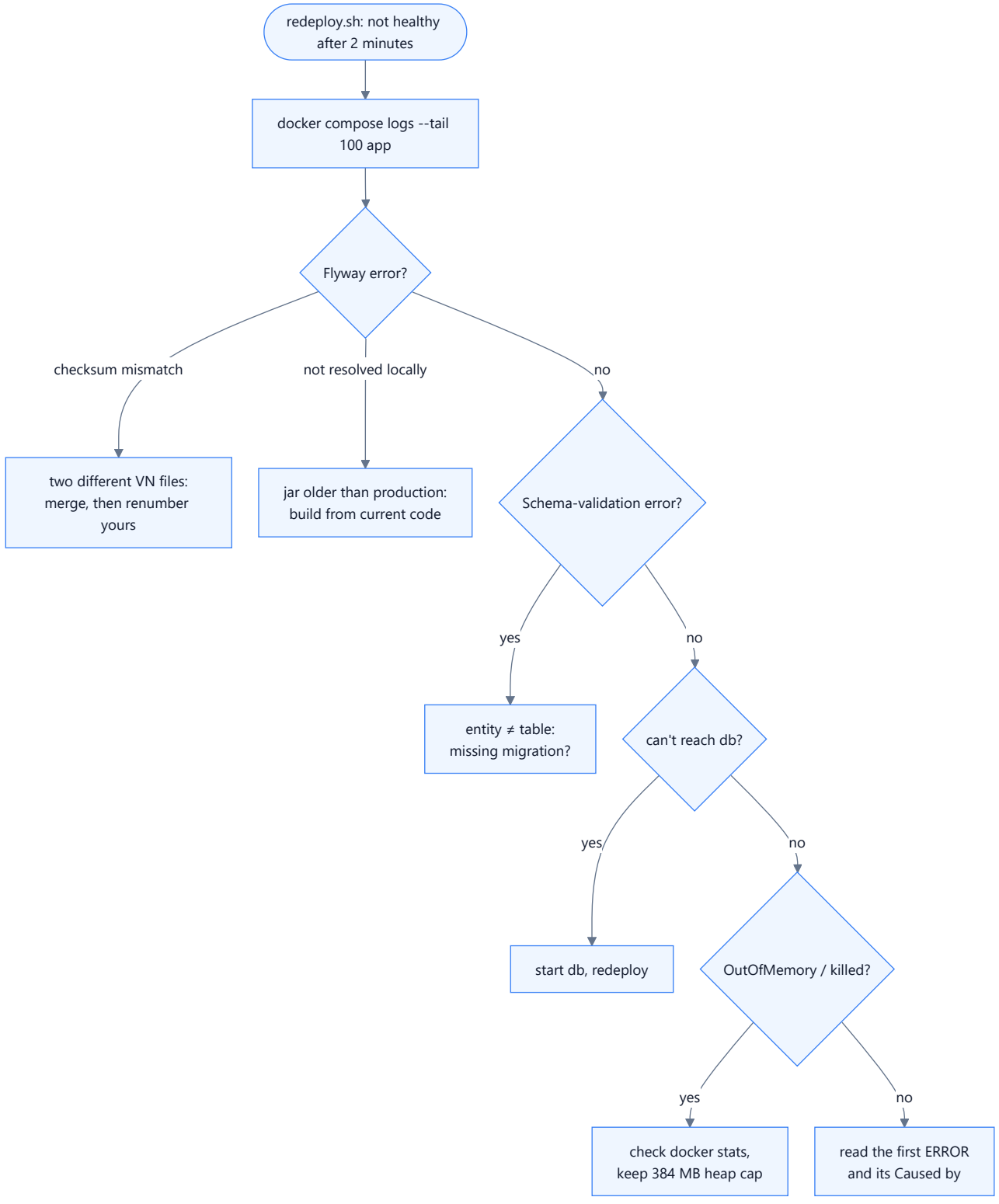
When `redeploy.sh` says "not healthy"

```
ssh -i ~/.ssh/krugworld_lighthouse ubuntu@43.156.150.155 "cd /opt/krugworld && docker compose logs --tail 100
app"
```

Log says	Meaning	Fix
`Migration checksum mismatch for migration version N`	the jar has a different `VN` file from the one that already ran	rebuild from code that has production's `VN` (pull/merge), renumber your new migration above it
`Detected applied migration not resolved locally`	production ran a migration this jar doesn't have	the jar is older than production: build from up-to-date code
`Schema-validation: missing column/table`	an entity doesn't match the database	a migration is missing from the jar, or an entity is wrong
connection refused to `db`	the database container is down	`docker compose up -d db`, then `redeploy.sh`

A failed start changes nothing in the database: Flyway validates before it migrates, and Hibernate validates before anything runs. If the site must be up right away, roll back to `krugworld.jar.prev` first and investigate after.

FIGURE 44 · WHEN REDEPLOY.SH SAYS NOT HEALTHY



Rolling back

If the old jar was kept (checklist step 5):

```
ssh -i ~/.ssh/krugworld_lighthouse ubuntu@43.156.150.155 "cp /opt/krugworld/krugworld.jar.prev
/opt/krugworld/krugworld.jar && /opt/krugworld/redeploy.sh"
```

An older jar normally starts on the newer schema a release left behind. Flyway treats migrations newer than any it knows as "future" and lets them be, and Hibernate ignores tables and columns it doesn't map. It breaks only if the new migration added something the old code trips over, like a `NOT NULL` column without a default that old inserts leave out; then restore the backup taken before the release. (A *gap* is different: if production has a migration in the middle that the jar lacks, as with V10 on 2026-10-23, Flyway refuses. That's the "not resolved locally" row above.)

Without a kept jar: rebuild the previous version from git (`git checkout <commit>` , build, deploy), which is another reason to only release pushed commits.

Backups

Before any release with a migration, and whenever you like:

```
ssh -i ~/.ssh/krugworld_lighthouse ubuntu@43.156.150.155 \
  "cd /opt/krugworld && docker compose exec -T db pg_dump -U krugworld krugworld | gzip > backups/before-
<what>-\$(date +%F-%H%M).sql.gz"
```

Copy one down to your PC now and then:

```
scp -i $HOME\.ssh\krugworld_lighthouse ubuntu@43.156.150.155:/opt/krugworld/backups/<file>.sql.gz .
```

Restore (this **replaces** the live data; stop the app first):

```
cd /opt/krugworld
docker compose stop app
gunzip -c backups/<file>.sql.gz | docker compose exec -T db psql -U krugworld -d krugworld --single-transaction
\
  -c 'DROP SCHEMA public CASCADE; CREATE SCHEMA public;' -f -
docker compose start app
```

Configuration on the server

`.env` is read by `docker compose` . After editing it, apply with `docker compose up -d app` (a plain `restart` doesn't re-read `.env`). Keep one line per key: a pasted key with a stray newline breaks the HTTP header it's sent in.

Variable	Purpose
`POSTGRES_PASSWORD`	database password (app and db containers)
`OPENAI_API_KEY`	Gaby (see Gaby, the Steward for the other `KRUGWORLD_*` settings)
`SMTP_HOST` , `SMTP_PORT` , `SMTP_USERNAME` , `SMTP_PASSWORD` , `MAIL_FROM` , `PUBLIC_URL`	password reset by email (port 587; Tencent blocks 25)

Health and monitoring

- `https://krugworld.veteryr.online/actuator/health` → `{"status":"UP"}`
- `https://krugworld.veteryr.online/api/status` → the applied schema version
- `docker compose ps` / `docker compose logs -f app` on the server
- Mail health is excluded from `/actuator/health`, so a dead SMTP server can't block a redeploy.

Lessons

2026-10-23: deploying from a checkout that was behind production. On 22 Oct, four releases (push notifications with `V10__push.sql`, meal money, one language, push urgency) were deployed from one PC without being pushed to GitHub. The next day another PC built from GitHub, added its own `V10`, and deployed. Flyway refused to start (checksum mismatch on V10), so the site went down, though no data was touched, and the previous jar had been overwritten, so there was nothing to roll back to. The fix was to redeploy from the PC that had the live code.

FIGURE 45 · LESSONS

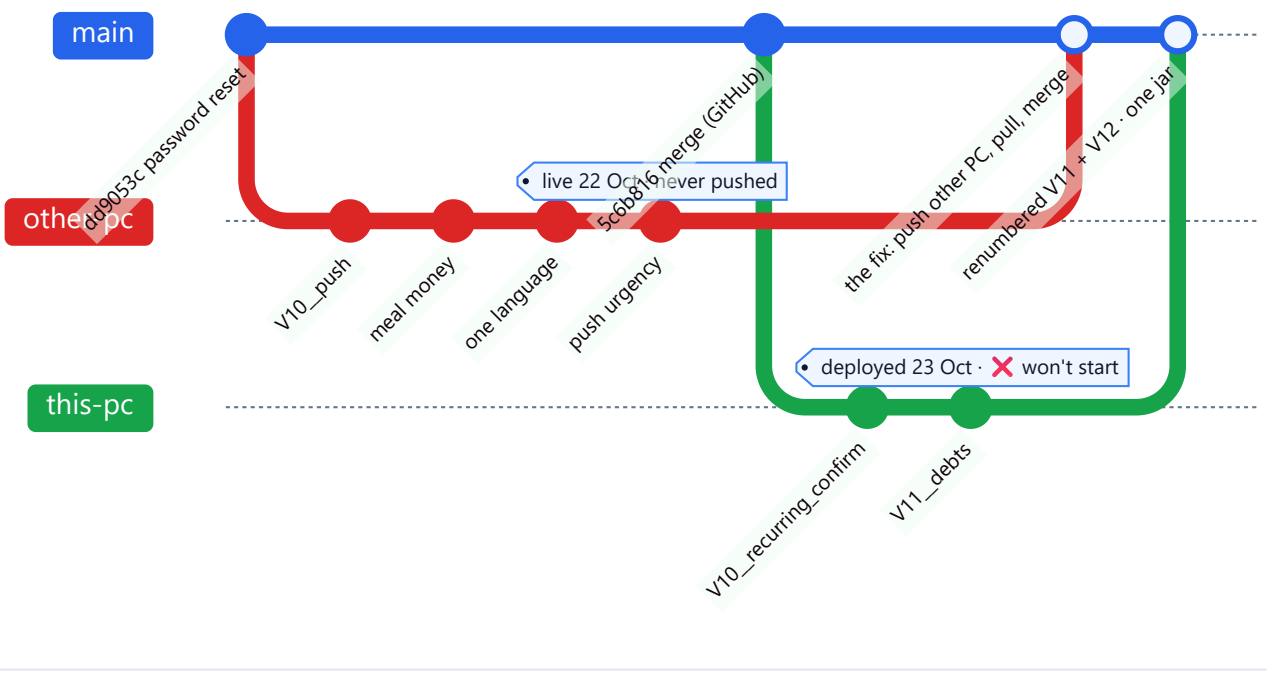
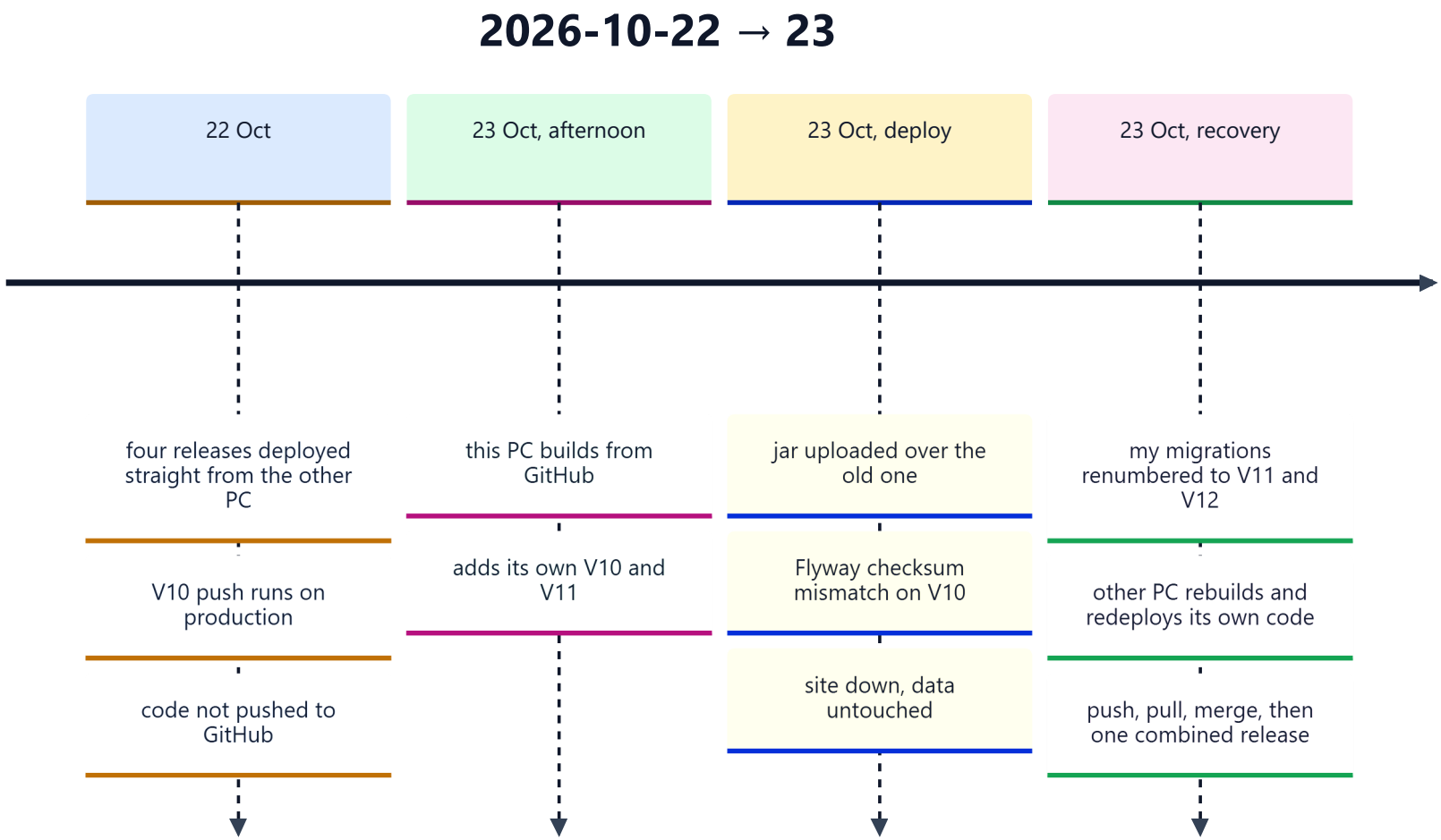


FIGURE 46 · LESSONS

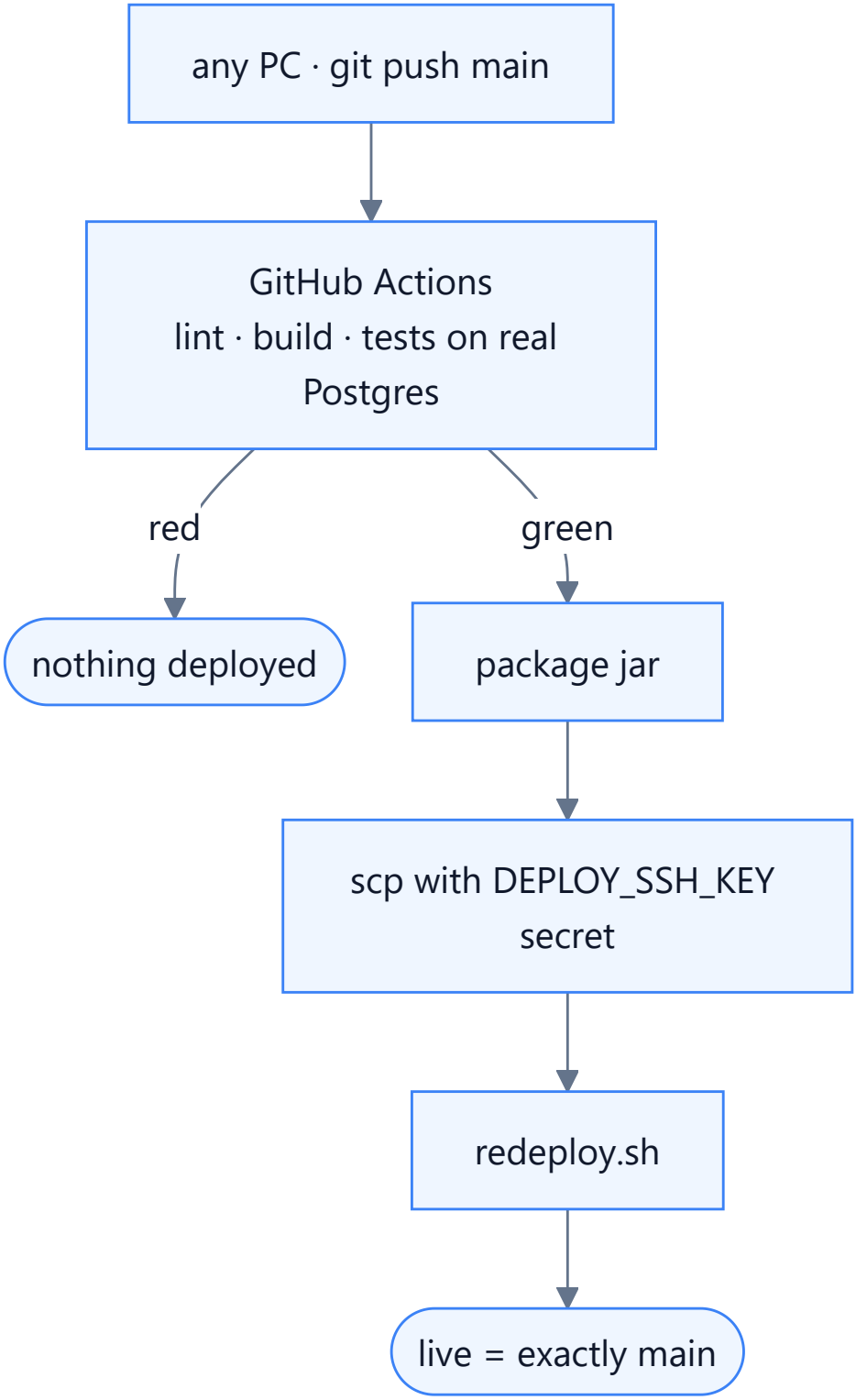


What changed afterwards:

- release only pushed code, from an up-to-date checkout (checklist step 1);
- number migrations from production's latest version (checklist step 2);
- keep `krugworld.jar.prev` as an instant rollback (checklist step 5).

A next step worth doing: let GitHub Actions build the jar after the tests pass and deploy it (a deploy SSH key as a repository secret). Then there's only one place a release can come from.

FIGURE 47 · LESSONS



CHAPTER 12

First-time Deployment

Setting up the server from nothing

KrugWorld ships as **one jar** that serves both the React app and the API, run in Docker next to its own Postgres, behind the host's nginx. This matches how Krugger's Tavern runs on the shared Lighthouse box, and lives alongside it without touching it.

	Tavern (existing)	KrugWorld
Folder	`/opt/kruggers-tavern/`	`/opt/krugworld/`
App port (localhost only)	8090	**8096**
Database	own `postgres:16` container	own `postgres:16` container
Address	tavern.veteryr.online	https://krugworld.veteryr.online (live since 2026-10-15)

1. Build (on your PC)

```
.\build.ps1
```

Builds the React app, runs all backend tests (Docker must be running), and writes `deploy\krugworld.jar`.

2. First time on the server

1. **DNS:** add an `A` record `krugworld.veteryr.online → 43.156.150.155`.
2. **Check port 8096 is free:** `ss -ltn | grep 8096` should print nothing.
3. **Copy the deploy folder up** (from the repo root on your PC):

   ```
   scp -i $HOME\.ssh\krugworld_lighthouse -r deploy ubuntu@43.156.150.155:/tmp/krugworld
   ssh -i $HOME\.ssh\krugworld_lighthouse ubuntu@43.156.150.155 "sudo mv /tmp/krugworld /opt/krugworld && sudo chown -R ubuntu /opt/krugworld"
   ```

4. **Secrets:** on the server, `cp /opt/krugworld/.env.example /opt/krugworld/.env` and set `POSTGRES_PASSWORD` (`openssl rand -base64 24`). For the Steward, set `OPENAI_API_KEY` (the default provider; `KRUGWORLD_STEWARD_PROVIDER=anthropic` plus `ANTHROPIC_API_KEY` switches its brain to Claude). After editing `.env`, apply it with `docker compose up -d app` (a plain `restart` does not re-read `.env`). Check the file stays one line per key: a pasted key with a stray newline breaks the header it's sent in.
5. **Start it:** `cd /opt/krugworld && chmod +x redeploy.sh && ./redeploy.sh`
6. **nginx + HTTPS:**

```
sudo cp /opt/krugworld/nginx-krugworld.conf /etc/nginx/sites-available/krugworld
sudo ln -s /etc/nginx/sites-available/krugworld /etc/nginx/sites-enabled/krugworld
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d krugworld.veteryr.online
```

3. Every later release

```
.\build.ps1
scp -i $HOME\.ssh\krugworld_lighthouse deploy\krugworld.jar ubuntu@43.156.150.155:/opt/krugworld/krugworld.jar
ssh -i $HOME\.ssh\krugworld_lighthouse ubuntu@43.156.150.155 "/opt/krugworld/redeploy.sh"
```

Database changes need no manual step: Flyway applies any new `V<n>__*.sql` migrations when the app starts.

Password reset by email (optional)

Without this, a forgotten passphrase can only be fixed by hand in the database. To turn on "Forgot your passphrase?":

1. Pick a mailbox to send from. With Gmail: turn on 2-step verification, then create an **App password** at https://myaccount.google.com/apppasswords. Use that 16-character password, never your real one.
2. On the server, add to `/opt/krugworld/.env` (one line per key, no stray newlines):

   ```
   SMTP_HOST=smtp.gmail.com
   SMTP_PORT=587
   SMTP_USERNAME=you@gmail.com
   SMTP_PASSWORD=the-app-password
   MAIL_FROM=you@gmail.com
   PUBLIC_URL=https://krugworld.veteryr.online
   ```

3. Apply it: `cd /opt/krugworld && docker compose up -d app`.
4. Each person adds their email in **Settings → Recovery email** (new accounts can add it when registering). Accounts with no email can't use the reset.

Port 587 is used on purpose: Tencent Cloud blocks outbound port 25.

4. Bring your Tavern ledger across

Log in to KrugWorld, open **Settings → Import from Krugger's Tavern**, and follow the three steps there (it reads Tavern's `/api/kv` export).

Backups

```
docker compose -f /opt/krugworld/docker-compose.yml exec -T db pg_dump -U krugworld krugworld >
krugworld-$(date +%F).sql
```

Notes from the first real deploy (2026-10-15)

- **SSH key:** `~/.ssh/krugworld_lighthouse` (ed25519, made on the Windows PC). Its public half is in `ubuntu`'s `~/.ssh/authorized_keys` on the server. On a new PC, make a new key and add its `.pub` line the same way (Lighthouse console → OrcaTerm).
- **Building on Windows:** if `build.ps1` stops on a JVM warning ("Sharing is only supported for boot loader classes..."), that's PowerShell 5.1 treating any stderr as an error, not a failed build. Run the same steps from Git Bash instead: `cd frontend && npm run build`, then `cd backend && ./mvnw -q package`, then copy `backend/target/krugworld-0.1.0.jar` to `deploy/krugworld.jar`. Set `JAVA_HOME` to an installed JDK if the Tavern's portable one isn't there.
- **Swap the jar safely:** upload as `krugworld.jar.new`, then `mv` it over the old one and run `redeploy.sh`, so a half-finished upload is never the jar that starts.
- **nginx for Desk Gaby:** `client_max_body_size 2m` (voice clips) and `proxy_buffering off` on `/api/device/voice/speech/` so her voice streams instead of arriving all at once. Both are in `nginx-krugworld.conf`; certbot added the 443 block.
- **Fresh DNS names:** a home router may cache "no such name" for a while after the record is created. Public resolvers (`nslookup krugworld.veteryr.online 8.8.8.8`) show the truth; the desk device falls back to them on its own.